Programming Language Foundations
Appunti

Alex "Morbidelli WhatsApp" Basta

Contents

Chapter 1 **Operational Semantics** Page

1.1 Properties of programs
Inductive Sets — • Inductive Proofs —
1.2 Large-step Operational Semantics
1.3 Equivalence of semantics
1.4 A simple imperative language IMP
Small-step operational semantics —
1.5 Large-step operational semantics for IMP
Command equivalence —

Chapter 2 **Denotational Semantics** Page

2.1 A Denotational Semantics for IMP
2.2 Fixed Points
2.3 Denotation for Loops
2.4 Kleene's Fixed Point Theorem
2.5 Reasoning using Denotational Semantics

Chapter 3 **Axiomatic Semantics** Page

3.1 Introduction to axiomatic semantics
3.2 Assertions
3.3 Satisfaction and Validity
3.4 Hoare Logic
3.5 Soundness and Completeness
3.6 Example: Factorial
3.7 Decorated Programs
Informal Rules — • Decorating a Program —
3.8 Weakest Preconditions

Chapter 4 **Separation Logic** Page

4.1 IMP with Heaps
4.2 Big-Step Operational Semantics
4.3 The Rule of Constancy
4.4 Heap Assertions
4.5 Valid Triples
4.6 The Frame Rule

4.7 Small Axioms for Heap Commands
4.8 Example: Swap
4.9 Linked Lists
4.10 Example: Disposing a List
4.11 List Segments
4.12 Example: List Append
4.13 The Magic Wand

Chapter 5 Lambda Calculus Page

5.1 Syntax
Variable binding and α-equivalence — • Higher-order functions —
5.2 Semantics
β-equivalence — • Call-by-value — • Call-by-name —
5.3 Lambda calculus evaluation
5.4 Confluence
5.5 Substitution
5.6 λ-calculus encodings
Booleans — • Church numerals —
5.7 Nontermination
5.8 Recursion
Recursion Removal Trick — • Fixed point combinators —
5.9 de Bruijn Notation
5.10 Definitional translation
Evaluation contexts — • Multi-argument functions and currying —
5.11 Products and let
5.12 Laziness
5.13 Adequacy of translation

Chapter 6 Type Systems Page

6.1 Simply-typed lambda calculus
The typing relation — • Type soundness — • Preservation — • Progress — • Uniqueness of Types — • Type Completeness? — • Termination —
6.2 Advanced Types
Booleans and Conditionals — • What Goes Wrong? — • Products — • Derived Forms — • Sums — • Extending the Proofs — • Fixed Points — • Exceptions —
6.3 Reference Types
6.4 Recursive Types
6.5 Equirecursive Types
The μ constructor — • Typing rules —
6.6 Isorecursive Types
Typing rules — • Operational semantics — • Example —
6.7 Equirecursive vs. Isorecursive
6.8 Numbers as a Recursive Type
6.9 Self-Application and Ω
6.10 Untyped to Typed λ-Calculus

Chapter 7 Polymorphism

Page

7.1 Parametric Polymorphism

7.2 System F
Syntax and operational semantics — • Type system —

7.3 Programming in System F
Self application — • Church numerals — • Products and sums —

7.4 Existential Types
Syntax and semantics — • Typing — • Example: counters — • Encoding existentials —

7.5 Metatheory

7.6 Type Erasure

7.7 Parametricity

7.8 Polymorphism in OCaml

7.9 Type Inference
Constraint-based typing — • Unification — • The occurs check —

7.10 Let-Polymorphism
Typing rules — • Inference — • The value restriction — • Weak type variables —

7.11 Summary

Chapter 1

Operational Semantics

Definition 1.0.1: Multi-step evaluation

Binary relation indicated with $\rightarrow^*$ which is the reflexive and transitive closure of the small-step evaluation.

Note that the transitive rule is a single step and a multi step, although it can be proved that it allows concatenation of single stepps.

1.1 Properties of programs

- Determinism: suppose that for any given configuration, if this can step in different ways these are equal
- Termination: evaluation terminates. Formally, for the multi step there is a configuration that it can multi step to without other possible steps.
- Soundness: if we run a program long enough we should get a final integer, but it's not ture because stores are partial function and if a variable isn't defined we're cooked.

Definition 1.1.1: Free Variables

Given the function $fvs(e)$ defined inductively as:

$$fvs(e) := \begin{cases} \emptyset & \text{if } e \text{ is a variable} \\ fvs(e_1) \cup \{x\} & \text{if } e = (\lambda x.e_1) \\ fvs(e_1) \cup fvs(e_2) & \text{if } e = (e_1 e_2) \end{cases}$$

Note that in $x := x + 1; y := x$ we still need to add x to the free variables because it's in the rhs of the first assignment.

Definition 1.1.2: Well-Formedness

For any environment σ and expression e, if all free variables in e are in σ, then e is well-formed under σ.

Two properties that imply soundness:

Definition 1.1.3: Progress

Every well-formed configuration is either an `Int` or can step to another configuration.

Definition 1.1.4: Preservation

If a configuration is well-formed, any following configuration it stepps to will also be well-formed.

with these lenmas and termination we can show that a well-formaed configuration is sound.

1.1.1 Inductive Sets

A bit of maths before the proof

Definition 1.1.5: Inductive Set

A set S is **inductive** if:

1. $\emptyset \in S$, and
2. for every set x, if $x \in S$, then $x \cup \{x\} \in S$.

We have used inductive setts with the inductive rules (remember that $\longrightarrow$ is a relation, so a set of tuples) and BNF grammar, free-variables and multistep. Another example is the natural numbers using the successor definition.

1.1.2 Inductive Proofs

If we want to prove something about an inductive set (that can be infinite by closing under the inductive rules) we can use the induction principle.
The base cases are all the axioms.
Sometimes the properties are defined over multiple inductive setts, so we need to chose one.

1.2 Large-step Operational Semantics

In the last lecture we defined a semantics for our language of arithmetic expressions using a small-step evaluation relation $\rightarrow \subseteq \textbf{Config} \times \textbf{Config}$ (and its reflexive and transitive closure $\rightarrow^*$). In this lecture we will explore an alternative approach—*large-step* operational semantics—which yields the final result of evaluating an expression directly.

Defining a large-step semantics boils down to specifying a relation $\Downarrow$ that captures the evaluation of an expression. The $\Downarrow$ relation has the following type:

$$\Downarrow \subseteq (\textbf{Store} \times \textbf{Exp}) \times (\textbf{Store} \times \textbf{Int}).$$

We write $\langle \sigma, e \rangle \Downarrow \langle \sigma', n \rangle$ to indicate that $((\sigma, e), (\sigma', n)) \in \Downarrow$. In other words, the expression e with store σ evaluates in one big step to the final store σ' and integer n.

We define the relation $\Downarrow$ inductively, using inference rules:

$$\frac{}{\langle \sigma, n \rangle \Downarrow \langle \sigma, n \rangle}\ \text{INT} \qquad \frac{n = \sigma(x)}{\langle \sigma, x \rangle \Downarrow \langle \sigma, n \rangle}\ \text{VAR}$$

Note:

Note that in the big step semantics, we need an inference rule for every valid configuration, hence the INT rule.

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \qquad \langle \sigma', e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle \qquad n = n_1 + n_2}{\langle \sigma, e_1 + e_2 \rangle \Downarrow \langle \sigma'', n \rangle}\ \text{ADD}$$

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \qquad \langle \sigma', e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle \qquad n = n_1 \times n_2}{\langle \sigma, e_1 * e_2 \rangle \Downarrow \langle \sigma'', n \rangle}\ \text{MUL}$$

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \qquad \langle \sigma'[x \mapsto n_1], e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle}{\langle \sigma, x := e_1; e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle}\ \text{ASSGN}$$

Example 1.2.1 (Big-step proof)

To illustrate the use of these rules, consider the following proof tree, which shows that evaluating $\langle \sigma, foo := 3; foo * bar \rangle$ using a store σ such that $\sigma(bar) = 7$ yields $\sigma' = \sigma[foo \mapsto 3]$ and 21 as a result:

$$\dfrac{\dfrac{}{\langle \sigma, 3 \rangle \Downarrow \langle \sigma, 3 \rangle}\ \text{INT} \qquad \dfrac{\dfrac{}{\langle \sigma', foo \rangle \Downarrow \langle \sigma', 3 \rangle}\ \text{VAR} \qquad \dfrac{}{\langle \sigma', bar \rangle \Downarrow \langle \sigma', 7 \rangle}\ \text{VAR}}{\langle \sigma', foo * bar \rangle \Downarrow \langle \sigma', 21 \rangle}\ \text{MUL}}{\langle \sigma, foo := 3; foo * bar \rangle \Downarrow \langle \sigma', 21 \rangle}\ \text{ASSGN}$$

A closer look to this structure reveals the relation between small step and large-step evaluation: a depth-first traversal of the large-step proof tree yields the sequence of one-step transitions in small-step evaluation.

Note:

Big step semantics don't capture the computational intricacys (if not well-formed it's undefined instead of blocking), and it strugles with concurrency. For this reason it can be used toghether with small step for the critical parts.

Note:

Also, small step semantics can be used to precysely count computation ticks (cslib 4 lean) for tighter complexity bounds.

1.3 Equivalence of semantics

A natural question to ask is whether the small-step and large-step semantics are equivalent. The next theorem answers this question affirmatively.

Theorem 1.3.1 Equivalence of semantics

For all expressions e, stores σ and σ', and integers n we have:

$$\langle \sigma, e\rangle \Downarrow \langle \sigma', n\rangle \quad \text{if and only if} \quad \langle \sigma, e\rangle \to^* \langle \sigma', n\rangle$$

To streamline the proof, we will work with the following definition of the multi-step relation:

$$\frac{}{\langle \sigma, e\rangle \to^* \langle \sigma, e\rangle}\ \text{REFL} \qquad \frac{\langle \sigma, e\rangle \to \langle \sigma', e'\rangle \qquad \langle \sigma', e'\rangle \to^* \langle \sigma'', e''\rangle}{\langle \sigma, e\rangle \to^* \langle \sigma'', e''\rangle}\ \text{TRANS}$$

Proof Sketch: We show each direction separately.

$\Longrightarrow$**:** We want to prove that the following property P holds for all expressions $e \in$ **Exp**:

$$P(e) \triangleq \forall \sigma, \sigma' \in \textbf{Store}.\ \forall n \in \textbf{Int}.\ \langle \sigma, e\rangle \Downarrow \langle \sigma', n\rangle \implies \langle \sigma, e\rangle \to^* \langle \sigma', n\rangle$$

We proceed by structural induction on e. We have to consider each of the possible axioms and inference rules for constructing an expression.

Case $e = x$**:** Assume that $\langle \sigma, x\rangle \Downarrow \langle \sigma', n\rangle$. That is, there is some derivation in the large-step operational semantics whose conclusion is $\langle \sigma, x\rangle \Downarrow \langle \sigma, n\rangle$. There is only one rule whose conclusion matches the configuration $\langle \sigma, x\rangle$: the large-step rule VAR. Thus, we have $n = \sigma(x)$ and $\sigma' = \sigma$. By the small-step rule VAR, we also have $\langle \sigma, x\rangle \to \langle \sigma, n\rangle$. By the REFL and TRANS rules, we conclude that $\langle \sigma, x\rangle \to^* \langle \sigma, n\rangle$, which finishes the case.

Case $e = n$**:** Assume that $\langle \sigma, n\rangle \Downarrow \langle \sigma', n'\rangle$. There is only one rule whose conclusion matches $\langle \sigma, n\rangle$: the large-step rule INT. Thus, we have $n' = n$ and $\sigma' = \sigma$ and so $\langle \sigma, n\rangle \to^* \langle \sigma, n\rangle$ by the REFL rule.

Case $e = e_1 + e_2$**:** This is an inductive case. We want to prove that if $P(e_1)$ and $P(e_2)$ hold, then $P(e)$ also holds. Let's write out $P(e_1)$, $P(e_2)$, and $P(e)$ explicitly.

$$P(e_1) = \forall n, \sigma, \sigma'.\ \langle \sigma, e_1\rangle \Downarrow \langle \sigma', n\rangle \implies \langle \sigma, e_1\rangle \to^* \langle \sigma', n\rangle$$

$$P(e_2) = \forall n, \sigma, \sigma'.\ \langle \sigma, e_2\rangle \Downarrow \langle \sigma', n\rangle \implies \langle \sigma, e_2\rangle \to^* \langle \sigma', n\rangle$$

$$P(e) = \forall n, \sigma, \sigma'.\ \langle \sigma, e_1 + e_2\rangle \Downarrow \langle \sigma', n\rangle \implies \langle \sigma, e_1 + e_2\rangle \to^* \langle \sigma', n\rangle$$

Assume that $P(e_1)$ and $P(e_2)$ hold. Also assume that there exist σ, σ' and n such that $\langle \sigma, e_1 + e_2\rangle \Downarrow \langle \sigma', n\rangle$. We need to show that $\langle \sigma, e_1 + e_2\rangle \to^* \langle \sigma', n\rangle$.
We assumed that $\langle \sigma, e_1 + e_2\rangle \Downarrow \langle \sigma', n\rangle$. This means that there is some derivation whose conclusion is $\langle \sigma, e_1 + e_2\rangle \Downarrow \langle \sigma', n\rangle$. By inspection, we see that only one rule has a conclusion of this form: the ADD rule. Thus, the last rule used in the derivation was ADD and it must be the case that $\langle \sigma, e_1\rangle \Downarrow \langle \sigma'', n_1\rangle$ and $\langle \sigma'', e_2\rangle \Downarrow \langle \sigma', n_2\rangle$ hold for some n_1 and n_2 with $n = n_1 + n_2$.
By the induction hypothesis $P(e_1)$, as $\langle \sigma, e_1\rangle \Downarrow \langle \sigma'', n_1\rangle$, we must have $\langle \sigma, e_1\rangle \to^* \langle \sigma'', n_1\rangle$. Likewise, by induction hypothesis $P(e_2)$, we have $\langle \sigma'', e_2\rangle \to^* \langle \sigma', n_2\rangle$. By Lemma 1 below, we have,

$$\langle \sigma, e_1 + e_2\rangle \to^* \langle \sigma'', n_1 + e_2\rangle,$$

and by another application of Lemma 1 we have:

$$\langle \sigma'', n_1 + e_2\rangle \to^* \langle \sigma', n_1 + n_2\rangle$$

Then, using the small-step ADD rule and the multi-step TRANS rule, we have:

$$\frac{\dfrac{n = n_1 + n_2}{\langle \sigma', n_1 + n_2\rangle \to \langle \sigma', n\rangle}\ \text{ADD} \qquad \dfrac{}{\langle \sigma', n\rangle \to^* \langle \sigma', n\rangle}\ \text{REFL}}{\langle \sigma', n_1 + n_2\rangle \to^* \langle \sigma', n\rangle}\ \text{TRANS}$$

Finally, by two applications of Lemma 2, we obtain $\langle \sigma, e_1 + e_2\rangle \to^* \langle \sigma', n\rangle$, which finishes the case.

Case $e = e_1 * e_2$**.** Similar to case for $e_1 + e_2$ above.

Case $e = x := e_1; e_2$**.** Assume that $\langle \sigma, x := e_1; e_2 \rangle \Downarrow \langle \sigma', n \rangle$. There is only one rule for which the conclusion matches the configuration, the ASSGN rule. Thus we have that $\langle \sigma, e_1 \rangle \Downarrow \langle \sigma'', n_1 \rangle$ and $\langle \sigma''[x \mapsto n_1], e_2 \rangle \Downarrow \langle \sigma', n$. By using the IHs and the Lemma 3 below, done.

$\Longleftarrow$**:** We proceed by induction on the derivation of $\langle \sigma, e \rangle \to^* \langle \sigma', n \rangle$ with a case analysis on the last rule used.

Case Refl: Then $e = n$ and $\sigma' = \sigma$. We immediately have $\langle \sigma, n \rangle \Downarrow \langle \sigma, n \rangle$ by the large-step rule INT.

Case Trans: Then $\langle \sigma, e \rangle \to \langle \sigma'', e'' \rangle$ and $\langle \sigma'', e'' \rangle \to^* \langle \sigma', n \rangle$. In this case, the induction hypothesis gives $\langle \sigma'', e'' \rangle \Downarrow \langle \sigma', n \rangle$. The result follows from Lemma 3 below. ✆

> **Lenma 1.3.1**
>
> If $\langle \sigma, e \rangle \to^* \langle \sigma', n \rangle$, then the following hold:
>
> - $\langle \sigma, e + e_2 \rangle \to^* \langle \sigma', n + e_2 \rangle$
> - $\langle \sigma, e * e_2 \rangle \to^* \langle \sigma', n * e_2 \rangle$
> - $\langle \sigma, n_1 + e \rangle \to^* \langle \sigma', n_1 + n \rangle$
> - $\langle \sigma, n_1 * e \rangle \to^* \langle \sigma', n_1 * n \rangle$
> - $\langle \sigma, x := e; e_2 \rangle \to^* \langle \sigma', x := n; e_2 \rangle$

: By induction on $\langle \sigma, e \rangle \to^* \langle \sigma', n \rangle$: **Case Refl**: then $e = n$ and $\sigma' = \sigma$, done by substitution. **Case Trans**: then $\langle \sigma, e \rangle \to \langle \sigma'', e' \rangle$ and $\langle \sigma'', e' \rangle \to^* \langle \sigma', n \rangle$. By using the TRANS rule and the IH, by setting the intermediate step appropriately and using the relative single-step rules for the operations, the proof is done. ✆

> **Lenma 1.3.2**
>
> If $\langle \sigma, e \rangle \to^* \langle \sigma', e' \rangle$ and $\langle \sigma', e' \rangle \to^* \langle \sigma'', e'' \rangle$, then $\langle \sigma, e \rangle \to^* \langle \sigma'', e'' \rangle$.

: By induction on the first relation: **Refl**: done by substitution. **Trans**: By the conditions of the rule, IH and the second relation, we have that $\langle \sigma^*, e^* \rangle \to^* \langle \sigma'', e'' \rangle$. We can use the other condition of the step rule to prove the conclusion (using the step rule). ✆

> **Lenma 1.3.3**
>
> If $\langle \sigma, e \rangle \to \langle \sigma'', e'' \rangle$ and $\langle \sigma'', e'' \rangle \Downarrow \langle \sigma', n \rangle$, then $\langle \sigma, e \rangle \Downarrow \langle \sigma', n \rangle$.

Proof. By induction on e, quite a long proof and I can't be bothered. Just use the single-step rules to find σ'', e'', and by this deduce the big-step rule used. ✆

1.4 A simple imperative language IMP

We will now consider a more realistic programming language, one where we can assign values to variables and execute control constructs such as **if** and **while**. The syntax for this imperative language, called IMP, is as follows:

$$\begin{array}{rl} \text{arithmetic expressions} & a \in \mathbf{Aexp} \quad a ::= x \mid n \mid a_1 + a_2 \mid a_1 \times a_2 \\ \text{Boolean expressions} & b \in \mathbf{Bexp} \quad b ::= \mathbf{true} \mid \mathbf{false} \mid a_1 < a_2 \\ \text{commands} & c \in \mathbf{Com} \quad c ::= \mathbf{skip} \mid x := a \mid c_1; c_2 \mid \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \mid \mathbf{while}\ b\ \mathbf{do}\ c \end{array}$$

1.4.1 Small-step operational semantics

We'll first give a small-step operational semantics for IMP. The configurations in this language are of the form $\langle \sigma, c\rangle$, $\langle \sigma, b\rangle$, and $\langle \sigma, a\rangle$, where σ is a store. The final configurations are of the form $\langle \sigma, \mathbf{skip}\rangle$ for commands, $\langle \sigma, \mathbf{true}\rangle$ and $\langle \sigma, \mathbf{false}\rangle$ for Boolean expressions, and $\langle \sigma, n\rangle$ for arithmetic expressions. There are three different small-step operational semantics relations: one each of the syntactic categories.

$$\rightarrow_{\mathbf{Com}} \subseteq (\mathbf{Store} \times \mathbf{Com}) \times (\mathbf{Store} \times \mathbf{Com})$$

$$\rightarrow_{\mathbf{Bexp}} \subseteq (\mathbf{Store} \times \mathbf{Bexp}) \times (\mathbf{Store} \times \mathbf{Bexp})$$

$$\rightarrow_{\mathbf{Aexp}} \subseteq (\mathbf{Store} \times \mathbf{Aexp}) \times (\mathbf{Store} \times \mathbf{Aexp})$$

For brevity, we will overload the symbol $\rightarrow$ and use it to refer to all of these relations. Which relation is being used will be clear from context. The evaluation rules for arithmetic and Boolean expressions are similar to the ones we've seen before. However, note that since the arithmetic expressions no longer contain assignment, arithmetic and Boolean expressions can not update the store. But to streamline everything we keep it as an output. In the small-step we'll fix the fact that the store doesn't change, but this can be altered if needed using σ'.

Arithmetic expressions

$$\frac{n = \sigma(x)}{\langle \sigma, x\rangle \rightarrow \langle \sigma, n\rangle}$$

$$\frac{\langle \sigma, a_1\rangle \rightarrow \langle \sigma, a_1'\rangle}{\langle \sigma, a_1 + a_2\rangle \rightarrow \langle \sigma, a_1' + a_2\rangle} \qquad \frac{\langle \sigma, a_2\rangle \rightarrow \langle \sigma, a_2'\rangle}{\langle \sigma, n + a_2\rangle \rightarrow \langle \sigma, n + a_2'\rangle} \qquad \frac{p = n + m}{\langle \sigma, n + m\rangle \rightarrow \langle \sigma, p\rangle}$$

$$\frac{\langle \sigma, a_1\rangle \rightarrow \langle \sigma, a_1'\rangle}{\langle \sigma, a_1 \times a_2\rangle \rightarrow \langle \sigma, a_1' \times a_2\rangle} \qquad \frac{\langle \sigma, a_2\rangle \rightarrow \langle \sigma, a_2'\rangle}{\langle \sigma, n \times a_2\rangle \rightarrow \langle \sigma, n \times a_2'\rangle} \qquad \frac{p = n \times m}{\langle \sigma, n \times m\rangle \rightarrow \langle \sigma, p\rangle}$$

Boolean expressions

$$\frac{\langle \sigma, a_1\rangle \rightarrow \langle \sigma, a_1'\rangle}{\langle \sigma, a_1 < a_2\rangle \rightarrow \langle \sigma, a_1' < a_2\rangle} \qquad \frac{\langle \sigma, a_2\rangle \rightarrow \langle \sigma, a_2'\rangle}{\langle \sigma, n < a_2\rangle \rightarrow \langle \sigma, n < a_2'\rangle}$$

$$\frac{n < m}{\langle \sigma, n < m\rangle \rightarrow \langle \sigma, \mathbf{true}\rangle} \qquad \frac{n \geqslant m}{\langle \sigma, n < m\rangle \rightarrow \langle \sigma, \mathbf{false}\rangle}$$

Commands

$$\frac{\langle \sigma, a\rangle \rightarrow \langle \sigma, a'\rangle}{\langle \sigma, x := a\rangle \rightarrow \langle \sigma, x := a'\rangle} \qquad \frac{}{\langle \sigma, x := n\rangle \rightarrow \langle \sigma[x \mapsto n], \mathbf{skip}\rangle}$$

$$\frac{\langle \sigma, c_1\rangle \rightarrow \langle \sigma', c_1'\rangle}{\langle \sigma, c_1; c_2\rangle \rightarrow \langle \sigma', c_1'; c_2\rangle} \qquad \frac{}{\langle \sigma, \mathbf{skip}; c_2\rangle \rightarrow \langle \sigma, c_2\rangle}$$

For **if** commands, we reduce the test until we get **true** or **false** and then we execute the appropriate branch:

$$\frac{\langle \sigma, b\rangle \rightarrow \langle \sigma, b'\rangle}{\langle \sigma, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2\rangle \rightarrow \langle \sigma, \mathbf{if}\ b'\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2\rangle}$$

$$\frac{}{\langle \sigma, \textbf{if true then } c_1 \textbf{ else } c_2\rangle \to \langle \sigma, c_1\rangle} \qquad \frac{}{\langle \sigma, \textbf{if false then } c_1 \textbf{ else } c_2\rangle \to \langle \sigma, c_2\rangle}$$

For **while** loops, the above strategy doesn't work, because if we evaluate b we won't be able to re-evaluate it for the next loop. Instead, we use the following rule, which can be thought of as "unrolling" the loop, one iteration at a time.

$$\frac{}{\langle \sigma, \textbf{while } b \textbf{ do } c\rangle \to \langle \sigma, \textbf{if } b \textbf{ then } (c; \textbf{while } b \textbf{ do } c) \textbf{ else skip}\rangle}$$

Note:

With this new rule, induction on the structure of the term doesn't work! This is because instead of always getting smaller, the step is to a bigger c. Sometimes this is managable, but not always.

We can now take a concrete program and see how it executes under the above rules. Consider we execute the program

Example 1.4.1 (Small-step eval of IMP)

$$foo := 3;\ \textbf{while } foo < 4 \textbf{ do } foo := foo + 5$$

The execution works as follows:

$$\begin{aligned}
&\langle \sigma, foo := 3; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma', \textbf{skip}; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle && \text{where } \sigma' = \sigma[foo \mapsto 3] \\
\to\ &\langle \sigma', \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma', \textbf{if } foo < 4 \textbf{ then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma', \textbf{if } 3 < 4 \textbf{ then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma', \textbf{if true then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma', foo := foo + 5; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma', foo := 3 + 5; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma', foo := 8; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma'', \textbf{skip}; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle && \text{where } \sigma'' = \sigma'[foo \mapsto 8] \\
\to\ &\langle \sigma'', \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma'', \textbf{if } foo < 4 \textbf{ then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma'', \textbf{if } 8 < 4 \textbf{ then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma'', \textbf{if false then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma'', \textbf{skip}\rangle
\end{aligned}$$

where W is an abbreviation for the while loop **while** $foo < 4$ **do** $foo := foo + 5$.

1.5 Large-step operational semantics for IMP

We define large-step evaluation relations for arithmetic expressions, Boolean expressions, and commands. The relation for arithmetic expressions relates an arithmetic expression and store to the integer value that the expression evaluates to. For Boolean expressions, the final value is in **Bool** = {**true**, **false**}. For commands, the final value is a store.

$$\Downarrow_{\textbf{Aexp}} \subseteq (\textbf{Aexp} \times \textbf{Store}) \times \textbf{Int}$$

$$\Downarrow_{\textbf{Bexp}} \subseteq (\textbf{Bexp} \times \textbf{Store}) \times \textbf{Bool}$$

$$\Downarrow_{\textbf{Com}} \subseteq (\textbf{Com} \times \textbf{Store}) \times \textbf{Store}$$

Again, we overload the symbol $\Downarrow$ and use it for any of these three relations; which relation is intended will be clear from context. We also use infix notation, for example writing $\langle \sigma, c \rangle \Downarrow \sigma'$ if $(\langle \sigma, c \rangle, \sigma') \in \Downarrow_{\mathbf{Com}}$.

Note:

In this course, we're not interested in concrete syntax, so ambiguous AST aren't a problem. Braces can be used to explicit.

Arithmetic expressions.

$$\frac{}{\langle \sigma, n \rangle \Downarrow n} \qquad \frac{\sigma(x) = n}{\langle \sigma, x \rangle \Downarrow n}$$

$$\frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \qquad \langle \sigma, a_2 \rangle \Downarrow n_2 \qquad n = n_1 + n_2}{\langle \sigma, a_1 + a_2 \rangle \Downarrow n} \qquad \frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \qquad \langle \sigma, a_2 \rangle \Downarrow n_2 \qquad n = n_1 \times n_2}{\langle \sigma, a_1 \times a_2 \rangle \Downarrow n}$$

Boolean expressions.

$$\frac{}{\langle \sigma, \mathbf{true} \rangle \Downarrow \mathbf{true}} \qquad \frac{}{\langle \sigma, \mathbf{false} \rangle \Downarrow \mathbf{false}}$$

$$\frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \qquad \langle \sigma, a_2 \rangle \Downarrow n_2 \qquad n_1 < n_2}{\langle \sigma, a_1 < a_2 \rangle \Downarrow \mathbf{true}} \qquad \frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \qquad \langle \sigma, a_2 \rangle \Downarrow n_2 \qquad n_1 \geqslant n_2}{\langle \sigma, a_1 < a_2 \rangle \Downarrow \mathbf{false}}$$

Commands.

$$\frac{}{\langle \sigma, \mathbf{skip} \rangle \Downarrow \sigma} \text{ SKIP} \qquad \frac{\langle \sigma, a \rangle \Downarrow n}{\langle \sigma, x := a \rangle \Downarrow \sigma[x \mapsto n]} \text{ ASSGN}$$

$$\frac{\langle \sigma, c_1 \rangle \Downarrow \sigma' \qquad \langle \sigma', c_2 \rangle \Downarrow \sigma''}{\langle \sigma, c_1; c_2 \rangle \Downarrow \sigma''} \text{ SEQ}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \qquad \langle \sigma, c_1 \rangle \Downarrow \sigma'}{\langle \sigma, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \rangle \Downarrow \sigma'} \text{ IF-T} \qquad \frac{\langle \sigma, b \rangle \Downarrow \mathbf{false} \qquad \langle \sigma, c_2 \rangle \Downarrow \sigma'}{\langle \sigma, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \rangle \Downarrow \sigma'} \text{ IF-F}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \mathbf{false}}{\langle \sigma, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \sigma} \text{ WHILE-F}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \qquad \langle \sigma, c \rangle \Downarrow \sigma' \qquad \langle \sigma', \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \sigma''}{\langle \sigma, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \sigma''} \text{ WHILE-T}$$

Note:

The big-step semantics treat termination differently to small-step. To prove these two are equal, we could assume termination.

It's interesting to see that the rule for while loops does not rely on using an if command (as in the case of small-step semantics). Why does this rule work?

1.5.1 Command equivalence

The small-step operational semantics suggests that the loop **while** b **do** c should be equivalent to the command **if** b **then** $(c;$ **while** b **do** $c)$ **else skip**. Can we show that this indeed the case that the language is defined using the above large-step evaluation?

First, we need to to be more precise about what "equivalent commands" mean. Our formal model allows us to define this concept using large-step evaluations as follows. (One can write a similar definition using $\rightarrow^*$ in small-step semantics.)

Definition 1.5.1: Equivalence of commands

Two commands c and c' are equivalent (written $c \sim c'$) if, for any stores σ and σ', we have

$$\langle \sigma, c \rangle \Downarrow \sigma' \iff \langle \sigma, c' \rangle \Downarrow \sigma'.$$

We can now state and prove the claim that **while** b **do** c and **if** b **then** $(c;$ **while** b **do** $c)$ **else skip** are equivalent.

Theorem 1.5.1

For all $b \in$ **Bexp** and $c \in$ **Com** we have

$$\textbf{while } b \textbf{ do } c \ \sim \ \textbf{if } b \textbf{ then } (c; \textbf{while } b \textbf{ do } c) \textbf{ else skip}.$$

: Let W be an abbreviation for **while** b **do** c. We want to show that for all stores σ, σ', we have:

$$\langle \sigma, W \rangle \Downarrow \sigma' \quad \text{if and only if} \quad \langle \sigma, \textbf{if } b \textbf{ then } (c; W) \textbf{ else skip} \rangle \Downarrow \sigma'$$

For this, we must show that both directions ($\Longrightarrow$ and $\Longleftarrow$) hold. We'll show only direction $\Longrightarrow$; the other is similar.
Assume that σ and σ' are stores such that $\langle \sigma, W \rangle \Downarrow \sigma'$. It means that there is some derivation that proves this fact. Inspecting the evaluation rules, we see that there are two possible rules whose conclusions match this fact: While-F and While-T. We analyze each of them in turn.

While-F. The derivation must look like the following.

$$\frac{\begin{array}{c}\vdots_1 \\ \langle \sigma, b \rangle \Downarrow \textbf{false}\end{array}}{\langle \sigma, W \rangle \Downarrow \sigma} \text{ WHILE-F}$$

Here, we use $\vdots_1$ to refer to the derivation of $\langle \sigma, b \rangle \Downarrow \textbf{false}$. Note that in this case, $\sigma' = \sigma$.

We can use $\vdots_1$ to derive a proof tree showing that the evaluation of **if** b **then** $(c; W)$ **else skip** yields the same final state σ:

$$\frac{\begin{array}{c}\vdots_1 \\ \langle \sigma, b \rangle \Downarrow \textbf{false}\end{array} \qquad \frac{}{\langle \sigma, \textbf{skip} \rangle \Downarrow \sigma} \text{ SKIP}}{\langle \sigma, \textbf{if } b \textbf{ then } (c; W) \textbf{ else skip} \rangle \Downarrow \sigma} \text{ IF-F}$$

While-T. In this case, the derivation has the following form.

$$\frac{\begin{array}{c}\vdots_2 \\ \langle \sigma, b \rangle \Downarrow \textbf{true}\end{array} \qquad \begin{array}{c}\vdots_3 \\ \langle \sigma, c \rangle \Downarrow \sigma''\end{array} \qquad \begin{array}{c}\vdots_4 \\ \langle \sigma'', W \rangle \Downarrow \sigma'\end{array}}{\langle \sigma, W \rangle \Downarrow \sigma'} \text{ WHILE-T}$$

We can use subderivations $\vdots_2$, $\vdots_3$, and $\vdots_4$ to show that the evaluation of **if** b **then** $(c; W)$ **else skip** yields the same final state σ'.

$$\frac{\begin{array}{c}\vdots_2 \\ \langle \sigma, b \rangle \Downarrow \textbf{true}\end{array} \qquad \frac{\begin{array}{c}\vdots_3 \\ \langle \sigma, c \rangle \Downarrow \sigma''\end{array} \qquad \begin{array}{c}\vdots_4 \\ \langle \sigma'', W \rangle \Downarrow \sigma'\end{array}}{\langle \sigma, c; W \rangle \Downarrow \sigma'} \text{ SEQ}}{\langle \sigma, \textbf{if } b \textbf{ then } (c; W) \textbf{ else skip} \rangle \Downarrow \sigma'} \text{ IF-T}$$

Hence, we showed that in each of the two possible cases, the command **if** b **then** $(c; W)$ **else skip** evaluates to the same final state as the command W. ☺

Some questions:

- Can we write a program that doesn't terminate? yes, obv
- Is IMP Turing Complete? Kinda, we don't have a way of creating new state (it's statically allocated), so if we replace the unbound **Int** type with something like **Int64**, it wouldn't be complete

Chapter 2

Denotational Semantics

We have now seen two operational models for programming languages: small-step and large-step. In this lecture, we consider a different semantic model, called *denotational semantics*.
The idea in denotational semantics is to express the meaning of a program as the mathematical function that expresses what the program computes. We can think of an IMP program c as a function from stores to stores: given an initial store, the program produces a final store. For example, the program $foo := bar + 1$ can be thought of as a function that when given an input store σ, produces a final store σ' that is identical to σ except that it maps foo to the integer $\sigma(bar) + 1$; that is, $\sigma' = \sigma[foo \mapsto \sigma(bar) + 1]$. We will model programs as functions from input stores to output stores. As opposed to operational models, which tell us *how* programs execute, the denotational model shows us *what* programs compute.

2.1 A Denotational Semantics for IMP

For each program c, we write $C[\![c]\!]$ for the *denotation* of c, that is, the mathematical function that c represents:

$$C[\![c]\!] : \textbf{Store} \rightharpoonup \textbf{Store}.$$

Note that $C[\![c]\!]$ is actually a partial function (as opposed to a total function), both because the store may not be defined on the free variables of the program and because the program may not terminate for certain input stores. The function $C[\![c]\!]$ is not defined for non-terminating programs as they have no corresponding output stores.
We will write $C[\![c]\!]\sigma$ for the result of applying the function $C[\![c]\!]$ to the store σ. That is, if f is the function that $C[\![c]\!]$ denotes, then we write $C[\![c]\!]\sigma$ to mean the same thing as $f(\sigma)$.
We must also model expressions as functions, this time from stores to the values they represent. We will write $\mathcal{A}[\![a]\!]$ for the denotation of arithmetic expression a, and $\mathcal{B}[\![b]\!]$ for the denotation of boolean expression b.

$$\mathcal{A}[\![a]\!] : \textbf{Store} \rightharpoonup \textbf{Int} \qquad \mathcal{B}[\![b]\!] : \textbf{Store} \rightharpoonup \{\textbf{true}, \textbf{false}\}$$

Now we want to define these functions. To make it easier to write down these definitions, we will describe (partial) functions using sets of pairs. More precisely, we will represent a partial map $f : A \rightharpoonup B$ as a set of pairs $F = \{(a, b) \mid a \in A \text{ and } b = f(a) \in B\}$ such that, for each $a \in A$, there is at most one pair of the form $(a, _)$ in the set. Hence $(a, b) \in F$ is the same as $b = f(a)$.
We can now define denotations for IMP. We start with the denotations of expressions:

$$\begin{aligned}
\mathcal{A}[\![n]\!] &= \{(\sigma, n)\} \\
\mathcal{A}[\![x]\!] &= \{(\sigma, \sigma(x))\} \\
\mathcal{A}[\![a_1 + a_2]\!] &= \{(\sigma, n) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n = n_1 + n_2\}
\end{aligned}$$

$$\begin{aligned}
\mathcal{B}[\![\textbf{true}]\!] &= \{(\sigma, \textbf{true})\} \\
\mathcal{B}[\![\textbf{false}]\!] &= \{(\sigma, \textbf{false})\} \\
\mathcal{B}[\![a_1 < a_2]\!] &= \{(\sigma, \textbf{true}) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n_1 < n_2\} \\
&\quad \cup \{(\sigma, \textbf{false}) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n_1 \geqslant n_2\}
\end{aligned}$$

The denotations for commands are as follows:

$$\begin{aligned} C[\![\textbf{skip}]\!] &= \{(\sigma, \sigma)\} \\ C[\![x := a]\!] &= \{(\sigma, \sigma[x \mapsto n]) \mid (\sigma, n) \in \mathcal{A}[\![a]\!]\} \\ C[\![c_1; c_2]\!] &= \{(\sigma, \sigma') \mid \exists \sigma''.\ ((\sigma, \sigma'') \in C[\![c_1]\!] \wedge (\sigma'', \sigma') \in C[\![c_2]\!])\} \end{aligned}$$

All these can be expressed in function-style notation too.

Note that $C[\![c_1; c_2]\!] = C[\![c_2]\!] \circ C[\![c_1]\!]$, where $\circ$ is the composition of relations, defined as follows: if $R_1 \subseteq A \times B$ and $R_2 \subseteq B \times C$ then $R_2 \circ R_1 \subseteq A \times C$ is $R_2 \circ R_1 = \{(a, c) \mid \exists b \in B.\ (a, b) \in R_1 \wedge (b, c) \in R_2\}$. If $C[\![c_1]\!]$ and $C[\![c_2]\!]$ are total functions, then $\circ$ is function composition.

$$\begin{aligned} C[\![\textbf{if } b \textbf{ then } c_1 \textbf{ else } c_2]\!] = &\{(\sigma, \sigma') \mid (\sigma, \textbf{true}) \in \mathcal{B}[\![b]\!] \wedge (\sigma, \sigma') \in C[\![c_1]\!]\} \\ &\cup \{(\sigma, \sigma') \mid (\sigma, \textbf{false}) \in \mathcal{B}[\![b]\!] \wedge (\sigma, \sigma') \in C[\![c_2]\!]\} \end{aligned}$$

Note:

If the denotation of σ under b is undefined, then the function should also be undefined, hence the case completeness.

$$\begin{aligned} C[\![\textbf{while } b \textbf{ do } c]\!] = &\{(\sigma, \sigma) \mid (\sigma, \textbf{false}) \in \mathcal{B}[\![b]\!]\} \\ &\cup \{(\sigma, \sigma') \mid (\sigma, \textbf{true}) \in \mathcal{B}[\![b]\!] \wedge \exists \sigma''.\ ((\sigma, \sigma'') \in C[\![c]\!] \wedge (\sigma'', \sigma') \in C[\![\textbf{while } b \textbf{ do } c]\!])\} \end{aligned}$$

But now we have a problem: the last "definition" is not really a definition because it expresses $C[\![\textbf{while } b \textbf{ do } c]\!]$ in terms of itself! This is not a definition but a recursive equation. What we want is the solution to this equation.

Note:

The second invocation of while isn't completely circular because the state changes.

2.2 Fixed Points

We gave a recursive equation that the function $C[\![\textbf{while } b \textbf{ do } c]\!]$ must satisfy. To understand some of the issues involved, let's consider a simpler example. Consider the following equation for a function $f : \mathbb{N} \to \mathbb{N}$.

$$f(x) = \begin{cases} 0 & \text{if } x = 0 \\ f(x-1) + 2x - 1 & \text{otherwise} \end{cases} \tag{2.1}$$

This is not a definition for f, but rather an equation that we want f to satisfy. What function, or functions, satisfy this equation for f? The only solution to this equation is the function $f(x) = x^2$.
In general, there may be no solutions for a recursive equation (e.g., there are no functions $g : \mathbb{N} \to \mathbb{N}$ that satisfy the recursive equation $g(x) = g(x) + 1$), or multiple solutions (e.g., find two functions $g : \mathbb{R} \to \mathbb{R}$ that satisfy $g(x) = 4 \times g(\frac{x}{2})$).
We can compute solutions to such equations by building successive approximations. Each approximation is closer and closer to the solution. To solve the recursive equation for f, we start with the partial function $f_0 = \emptyset$ (i.e., f_0 is the empty relation; it is a partial function with the empty set for its domain). We compute successive

approximations using the recursive equation.

$$
\begin{aligned}
f_0 &= \emptyset \\
f_1 &= \begin{cases} 0 & \text{if } x = 0 \\ f_0(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0,0)\} \\
f_2 &= \begin{cases} 0 & \text{if } x = 0 \\ f_1(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0,0),(1,1)\} \\
f_3 &= \begin{cases} 0 & \text{if } x = 0 \\ f_2(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0,0),(1,1),(2,4)\} \\
&\vdots
\end{aligned}
$$

This sequence of successive approximations f_i gradually builds the function $f(x) = x^2$.
We can model this process of successive approximations using a higher-order function F that takes one approximation f_k and returns the next approximation f_{k+1}:

$$F : (\mathbb{N} \rightharpoonup \mathbb{N}) \to (\mathbb{N} \rightharpoonup \mathbb{N})$$

where

$$(F(f))(x) = \begin{cases} 0 & \text{if } x = 0 \\ f(x-1) + 2x - 1 & \text{otherwise} \end{cases}$$

A solution to the recursive equation (1) is a function f such that $f = F(f)$. In general, given a function $F : A \to A$, we have that $a \in A$ is a *fixed point* of F if $F(a) = a$. We also write $a = \text{fix}(F)$ to indicate that a is a least fixed point of F.
So the solution to the recursive equation (1) is a fixed-point of the higher-order function F. We can compute this fixed point iteratively, starting with $f_0 = \emptyset$ and at each iteration computing $f_{k+1} = F(f_k)$. The fixed point is the limit of this process:

$$
\begin{aligned}
f &= \text{fix}(F) \\
&= f_0 \cup f_1 \cup f_2 \cup f_3 \cup \ldots \\
&= \emptyset \cup F(\emptyset) \cup F(F(\emptyset)) \cup F(F(F(\emptyset))) \cup \ldots \\
&= \bigcup_{i \geqslant 0} F^i(\emptyset)
\end{aligned}
$$

2.3 Denotation for Loops

We can now write the correct denotation case for while loops as the fixed point of a higher-order function:

$$C[\![\textbf{while } b \textbf{ do } c]\!] = \text{fix}(F)$$

where

$$F(f) = \{(\sigma,\sigma) \mid (\sigma, \textbf{false}) \in \mathcal{B}[\![b]\!]\} \cup \{(\sigma,\sigma') \mid (\sigma, \textbf{true}) \in \mathcal{B}[\![b]\!] \wedge \exists \sigma''.\ (\sigma,\sigma'') \in C[\![c]\!] \wedge (\sigma'',\sigma') \in f\}$$

2.4 Kleene's Fixed Point Theorem

Next we'll prove Kleene's fixpoint theorem, which shows that the fixed point used to define the semantics of while commands exists.

Definition 2.4.1: Scott Continuity

A function F from U to U is said to be *Scott-continuous* if for every chain $X_1 \subseteq X_2 \subseteq \ldots$ we have $F(\bigcup_i X_i) = \bigcup_i F(X_i)$.

It is not hard to show that

Proposition 2.4.1 Monotonicity of Scott-continuous function

If F is Scott continuous, then it is also *monotonic*—that is, $X \subseteq Y$ implies $F(X) \subseteq F(Y)$.

: Assume $X \subseteq Y$, so $Y = Y \cup X$. By Scott-continuity

$$F(Y) = F(Y \cup X) = F(Y) \cup F(X)$$

So if $x \in F(X)$, then x is also in $F(Y)$. Not too sure if this is correct because is used a finite chain.

Theorem 2.4.1 Kleene Fixpoint

Let F be a Scott-continuous function. The least fixed point of F is $\bigcup_i F^i(\emptyset)$.

: Let $X = \bigcup_i F^i(\emptyset)$.
First, we will prove that X is a fixed point of F—that is, $F(X) = X$. We calculate as follows:

$$\begin{aligned}
F(X) &= F\left(\bigcup_i F^i(\emptyset)\right) && \text{By definition of } X \\
&= \bigcup_i F(F^i(\emptyset)) && \text{By Scott continuity} \\
&= \bigcup_i F^{i+1}(\emptyset) \\
&= \emptyset \cup \bigcup_i F^{i+1}(\emptyset) \\
&= F^0(\emptyset) \cup \bigcup_i F^{i+1}(\emptyset) \\
&= \bigcup_i F^i(\emptyset) \\
&= X
\end{aligned}$$

Second, we will show that X is the least fixed point of F. Suppose that Y is some other arbitrary fixed point of F. By induction, we can easily show that $F^i(\emptyset) \subseteq Y$ for all i. For the base case, i is 0 and we trivially have $F^0(\emptyset) = \emptyset \subseteq Y$. For the inductive case, we assume that $F^i(\emptyset) \subseteq Y$ and prove that $F^{i+1}(\emptyset) \subseteq Y$. By our inductive hypothesis and the fact that F is monotone, we have that $F(F^i(\emptyset)) \subseteq F(Y)$. As Y is a fixed point we also have $F(Y) = Y$ and so $F^{i+1}(\emptyset) \subseteq Y$.
Then, since every element of the chain

$$F^0(\emptyset) \subseteq F^1(\emptyset) \subseteq \ldots$$

is a subset of Y, immediately we have that their union, $X = \bigcup_i F^i(\emptyset) \subseteq Y$. Hence, X is the least (with respect to $\subseteq$) fixed point of F.

2.5 Reasoning using Denotational Semantics

One of the main advantages of using denotational semantics compared to operational semantics is that proofs of equivalence can be carried out directly by simply calculating the denotations of programs and then arguing that they are identical. This is in contrast to operational techniques, where one must reason explicitly about low-level transitions and derivations involving ad hoc abstract machines.
As an example, to show that **skip**; c and c; **skip** are equivalent, we can calculate as follows,

$$\begin{aligned}
C[\![\textbf{skip}; c]\!] &= \{(\sigma, \sigma'') \mid \exists \sigma'.\ (\sigma, \sigma') \in C[\![\textbf{skip}]\!] \wedge (\sigma', \sigma'') \in C[\![c]\!]\} \\
&= \{(\sigma, \sigma'') \mid (\sigma, \sigma'') \in C[\![c]\!]\} \\
&= \{(\sigma, \sigma'') \mid \exists \sigma'.\ (\sigma, \sigma') \in C[\![c]\!] \wedge (\sigma', \sigma'') \in C[\![\textbf{skip}]\!]\} \\
&= C[\![c; \textbf{skip}]\!]
\end{aligned}$$

using standard facts about partial functions, relations, and sets as convenient in the proof itself.
Next, consider the command $C[\![\textbf{while false do } c]\!]$. By the definition of the denotational semantics, this is equal to $\mathrm{fix}(F)$ where

$$F(f) = \{(\sigma, \sigma) \mid (\sigma, \textbf{false}) \in \mathcal{B}[\![b]\!]\} \cup \{(\sigma, \sigma'') \mid (\sigma, \textbf{true}) \in \mathcal{B}[\![b]\!] \wedge (\sigma, \sigma') \in C[\![c]\!] \wedge (\sigma', \sigma'') \in f\}$$

By the Kleene fixpoint theorem we have that $\mathrm{fix}(F) = \bigcup_i F^i(\emptyset)$. It is straightforward to show by induction that since the guard is false, for all i we have $F^i(\emptyset) = \{(\sigma, \sigma)\}$. It follows that $\mathrm{fix}(F) = \{(\sigma, \sigma)\}$, which is just $C[\![\textbf{skip}]\!]$.
As an exercise, calculate $C[\![\textbf{while true do skip}]\!]$ using the same technique.

Chapter 3

Axiomatic Semantics

3.1 Introduction to axiomatic semantics

Now we turn to the third and final main style of semantics, *axiomatic semantics*. The idea in axiomatic semantics is to define meaning in terms of logical specifications that programs satisfy. This is in contrast to operational models (which show *how* programs execute) and denotational models (which show *what* programs compute). This approach to reasoning about programs and expressing program semantics was originally proposed by Floyd and Hoare and was developed further by Dijkstra and Gries.
A common way to express program specifications is in terms of pre-conditions and post-conditions:

$$\{Pre\}\ c\ \{Post\}$$

where c is a program, and *Pre* and *Post* are formulas that describe properties of the program state, usually referred to as *assertions*. Such a triple is usually referred to as a *partial correctness specification* (or sometimes a "Hoare triple") and has the following meaning:

> "If *Pre* holds before executing c, and c terminates, then *Post* holds after c."

In other words, if we start with a store σ in which *Pre* holds and the execution of c with respect to σ terminates and yields a store σ', then *Post* holds in store σ'.
Pre-conditions and post-conditions can be regarded as interfaces or contracts between the program and its clients. They help users to understand what the program is supposed to yield without needing to understand how the program executes. Typically, programmers write them as comments for procedures and functions as documentation and to make it easier to maintain programs. Such specifications are especially useful for library functions for which the source code is often not available to the users. In this case, pre-conditions and post-conditions serve as contracts between the library developers and users of the library.
However, there is no guarantee that pre-conditions and post-conditions written informally in comments are correct: the comments describe the intent of the developer, but they do not give a guarantee of correctness. Axiomatic semantics addresses this problem. It shows how to rigorously describe partial correctness statements and how to establish correctness using formal reasoning.
Note that partial correctness specifications don't ensure that the program will terminate—this is why they are called "partial". In contrast, *total correctness statements* ensure that the program terminates whenever the precondition holds. Such statements are denoted using square brackets:

$$[\ Pre\ ]\ c\ [\ Post\ ]$$

meaning:

> "If *Pre* holds before c then c will terminate and *Post* will hold after c."

In general a pre-condition specifies what the program expects before execution and the post-condition specifies what guarantees the program provides (if the program terminates). Here is a simple example:

$$\{foo = 0 \wedge bar = i\}\ baz := 0;\ \textbf{while}\ foo \neq bar\ \textbf{do}\ (baz := baz - 2;\ foo := foo + 1)\ \{baz = -2i\}$$

It says that if the store maps *foo* to 0 and *bar* to i before execution, then, if the program terminates, the final store will map *baz* to $-2i$ (i.e., -2 times the initial value of *bar*). Note that i is a *logical variable* that doesn't occur in the program and is only used to express the initial value of *bar*. Such variables are sometimes called *ghost variables*.
This partial correctness statement is valid. That is, it is indeed the case that if we have any store σ such that $\sigma(foo) = 0$, and

$$C[\![baz := 0;\ \textbf{while}\ foo \neq bar\ \textbf{do}\ (baz := baz - 2;\ foo := foo + 1)]\!]\sigma = \sigma',$$

then $\sigma'(baz) = -2\sigma(bar)$.
Note that this is a partial correctness statement: if the pre-condition is true before c, and c terminates then the post-condition holds after c. There are some initial stores for which the program will not terminate.
The following total correctness statement is true.

$$[foo = 0 \wedge bar = i \wedge i \geqslant 0]\ baz := 0;\ \textbf{while}\ foo \neq bar\ \textbf{do}\ (baz := baz - 2;\ foo := foo + 1)\ [baz = -2i]$$

That is, if we start with a store σ that maps *foo* to 0 and *bar* to a non-negative integer, then the execution of the command will terminate in a final store σ' with $\sigma'(baz) = -2\sigma(bar)$.
The following partial correctness statement is not valid. (Why not?)

$$\{foo = 0 \wedge bar = i\}\ baz := 0;\ \textbf{while}\ foo \neq bar\ \textbf{do}\ (baz := baz + foo;\ foo := foo + 1)\ \{baz = i\}$$

In the rest of our discussion of axiomatic semantics we will focus almost exclusively on partial correctness assertions.

3.2 Assertions

Now we turn to the following issues:

- What logic do we use for writing assertions? That is, what can we express in pre-conditions and post-conditions?
- What does it mean that an assertion is valid? What does it mean that a partial correctness statement $\{Pre\}\ c\ \{Post\}$ is valid?
- How can we prove that a partial correctness statement is valid?

What can we say in pre-conditions and post-conditions? In the examples so far, we have used program variables, equality, logical variables (e.g., i), and conjunction ($\wedge$). What we allow in pre-conditions and post-conditions directly influences the sorts of program properties we can describe using partial correctness statements. We will use the set of logical formulas including comparisons between arithmetic expressions, standard logical operators (and, or, implication, negation), as well as quantifiers (universal and existential). Assertions may also introduce logical variables that are different than the variables appearing in the program.

$$\begin{aligned} i, j &\in \textbf{LVar} \\ a \in \textbf{Aexp} &::= x \mid i \mid n \mid a_1 + a_2 \mid a_1 \times a_2 \\ P, Q \in \textbf{Assn} &::= \textbf{true} \mid \textbf{false} \mid a_1 < a_2 \mid P_1 \wedge P_2 \mid P_1 \vee P_2 \mid P_1 \Rightarrow P_2 \mid \neg P \mid \forall i.\ P \mid \exists i.\ P \end{aligned}$$

Observe that the domain of boolean expressions **Bexp** is a subset of the domain of assertions. Notable additions over the syntax of boolean expressions are quantifiers ($\forall$ and $\exists$). For instance, one can express the fact that variable x divides variable y using existential quantification: $\exists i.\ x \times i = y$.

3.3 Satisfaction and Validity

Now we would like to describe what we mean by "assertion P holds in store σ". But to determine whether P holds or not, we need more than just the store σ (which maps program variables to their values); we also need to know the values of the logical variables. We describe those values using an *interpretation* I,

$$I : \textbf{LVar} \to \textbf{Int},$$

and define the function $\mathcal{A}_i[\![a]\!]$, which is like the denotation of expressions extended to logical variables in the obvious way:

$$\begin{aligned}
\mathcal{A}_i[\![n]\!](\sigma, I) &= n \\
\mathcal{A}_i[\![x]\!](\sigma, I) &= \sigma(x) \\
\mathcal{A}_i[\![i]\!](\sigma, I) &= I(i) \\
\mathcal{A}_i[\![a_1 + a_2]\!](\sigma, I) &= \mathcal{A}_i[\![a_1]\!](\sigma, I) + \mathcal{A}_i[\![a_2]\!](\sigma, I)
\end{aligned}$$

Now we can express the satisfiability of assertions as a relation $\sigma \models_I P$ read as "P is satisfied in store σ under interpretation I," or "store σ satisfies assertion P under interpretation I." We will write $\sigma \not\models_I P$ whenever $\sigma \models_I P$ doesn't hold.

$$\begin{aligned}
\sigma \models_I \textbf{true} &\quad \text{(always)} \\
\sigma \models_I a_1 < a_2 &\quad \text{if } \mathcal{A}_i[\![a_1]\!](\sigma, I) < \mathcal{A}_i[\![a_2]\!](\sigma, I) \\
\sigma \models_I P_1 \wedge P_2 &\quad \text{if } \sigma \models_I P_1 \text{ and } \sigma \models_I P_2 \\
\sigma \models_I P_1 \vee P_2 &\quad \text{if } \sigma \models_I P_1 \text{ or } \sigma \models_I P_2 \\
\sigma \models_I P_1 \Rightarrow P_2 &\quad \text{if } \sigma \not\models_I P_1 \text{ or } \sigma \models_I P_2 \\
\sigma \models_I \neg P &\quad \text{if } \sigma \not\models_I P \\
\sigma \models_I \forall i.\ P &\quad \text{if } \forall k \in \mathit{Int}.\ \sigma \models_{I[i \mapsto k]} P \\
\sigma \models_I \exists i.\ P &\quad \text{if } \exists k \in \mathit{Int}.\ \sigma \models_{I[i \mapsto k]} P
\end{aligned}$$

We can now say that an assertion P is *valid* (written $\models P$) if it is valid in any store, under any interpretation: $\forall \sigma, I.\ \sigma \models_I P$.

Having defined validity for individual assertions, we now turn to partial correctness statements. We say that a partial correctness statement $\{P\}\ c\ \{Q\}$ is *satisfied* in store σ and interpretation I, written $\sigma \models_I \{P\}\ c\ \{Q\}$, if:

$$\forall \sigma'.\ \text{if } \sigma \models_I P \text{ and } C[\![c]\!]\sigma = \sigma' \text{ then } \sigma' \models_I Q$$

Note that this definition depends on the execution of c in the initial store σ.

Finally, we can say that a partial correctness triple is *valid* (written $\models \{P\}\ c\ \{Q\}$), if it is valid in any store and interpretation:

$$\forall \sigma, I.\ \sigma \models_I \{P\}\ c\ \{Q\}.$$

Now we know what we mean when we say "assertion P holds" or "partial correctness statement $\{P\}\ c\ \{Q\}$ is valid."

3.4 Hoare Logic

How do we show that a partial correctness statement $\{P\}\ c\ \{Q\}$ holds? We know that $\{P\}\ c\ \{Q\}$ is valid if it holds for all stores and interpretations: $\forall \sigma, I.\ \sigma \models_I \{P\}\ c\ \{Q\}$. Furthermore, showing that $\sigma \models_I \{P\}\ c\ \{Q\}$ requires reasoning about the execution of command c (that is, $C[\![c]\!]$), as indicated by the definition of validity.

It turns out that there is an elegant way of deriving valid partial correctness statements, without having to reason about stores, interpretations, and the execution of c. We can use a set of inference rules and axioms, called *Hoare rules*, to directly derive valid partial correctness statements. The set of rules forms a proof system known as *Hoare logic*.

$$\frac{}{\{P\}\ \textbf{skip}\ \{P\}}\ \text{SKIP} \qquad \frac{}{\{P[a/x]\}\ x := a\ \{P\}}\ \text{ASSIGN}$$

$$\frac{\{P\}\ c_1\ \{R\} \qquad \{R\}\ c_2\ \{Q\}}{\{P\}\ c_1; c_2\ \{Q\}}\ \text{SEQ}$$

$$\frac{\{P \wedge b\}\ c_1\ \{Q\} \qquad \{P \wedge \neg b\}\ c_2\ \{Q\}}{\{P\}\ \textbf{if}\ b\ \textbf{then}\ c_1\ \textbf{else}\ c_2\ \{Q\}}\ \text{IF}$$

$$\frac{\{P \wedge b\}\ c\ \{P\}}{\{P\}\ \textbf{while}\ b\ \textbf{do}\ c\ \{P \wedge \neg b\}}\ \text{WHILE}$$

The assertion P in the rule for while loops is essentially a *loop invariant*; it is an assertion that holds before and after each iteration, as shown in the premise of the rule. Therefore, it is both a pre-condition for the loop (because it holds before the first iteration); and also a post-condition for the loop (because it holds after the last iteration). The fact that P is both a pre- and post-condition for the while loop is reflected in the conclusion of the rule.
There is one more rule, the rule of consequence, which allows to strengthen pre-conditions and weaken post-conditions:

$$\frac{\models (P \Rightarrow P') \qquad \{P'\}\ c\ \{Q'\} \qquad \models (Q' \Rightarrow Q)}{\{P\}\ c\ \{Q\}}\ \text{CONSEQUENCE}$$

These set of Hoare rules represent an inductive definition for a set of partial correctness statements $\{P\}\ c\ \{Q\}$. We will say that $\{P\}\ c\ \{Q\}$ is a *theorem in Hoare logic*, written $\vdash \{P\}\ c\ \{Q\}$, if we can build a finite proof tree for it.

3.5 Soundness and Completeness

At this point we have two kinds of partial correctness assertions:

- valid partial correctness statements $\models \{P\}\ c\ \{Q\}$, which hold for all stores and interpretations, according to the semantics of c, and
- Hoare logic theorems $\vdash \{P\}\ c\ \{Q\}$, that is, partial correctness statements that can be derived using the axioms and rules of Hoare logic.

The question is how do these sets relate to each other? More precisely, we have to answer two questions. First, is each Hoare logic theorem guaranteed to be valid partial correctness triple? In other words,

$$\text{does } \vdash \{P\}\ c\ \{Q\} \text{ imply } \models \{P\}\ c\ \{Q\}?$$

The answer is yes, and it shows that Hoare logic is *sound*. Soundness is important because it says that Hoare logic doesn't allow us to derive partial correctness assertions that actually don't hold. The proof of soundness requires induction on the derivations in $\vdash \{P\}\ c\ \{Q\}$ (we omit this proof).
The second question refers to the expressiveness and power of Hoare rules: can we always build a Hoare logic proof for each valid assertion? In other words,

$$\text{does } \models \{P\}\ c\ \{Q\} \text{ imply } \vdash \{P\}\ c\ \{Q\}?$$

The answer is a qualified yes: if $\models \{P\}\ c\ \{Q\}$ then there is a proof of $\vdash \{P\}\ c\ \{Q\}$ using the rules of Hoare logic, provided there are proofs for the validity of assertions that occur in the rule of consequence $\models (P \Rightarrow P')$ and $\models (Q' \Rightarrow Q)$. This result is known as the *relative completeness* of Hoare logic and is due to Cook (1974).

3.6 Example: Factorial

As an example illustrating how we can use Hoare logic to verify the correctness of a program, consider a program that computes the factorial of a number n:

$\{\mathtt{x} = n \wedge n > 0\}$
`y := 1;`
while `x > 0` **do** (
    `y := y * x;`
    `x := x - 1`
)
$\{\mathtt{y} = n!\}$

Because the derivation for this proof is somewhat large, we will go through the reasoning used to construct it step by step.

At the top level, the program is a sequence of an assignment and a loop. To use the SEQ rule, we need to find an assertion that holds after the assignment and before the loop. Examining the rule for while loops, we see that the assertion before the loop must be an invariant for the loop. Inspecting the loop we see that it builds the factorial up in $\mathtt{y}$ starting with n, then multiplying it by $n-1$, then $n-2$, etc. At each iteration, $\mathtt{x}$ contains the next value multiplied into $\mathtt{y}$, that is:

$$\mathtt{y} = n * (n-1) * \ldots * (\mathtt{x}+1)$$

If we multiply both sides of this equality by $\mathtt{x}!$ and re-write the equality we get $\mathtt{x}! * \mathtt{y} = n!$, which is an invariant for the loop. However, to make the proof go through, we need a slightly stronger invariant:

$$I = \mathtt{x}! * \mathtt{y} = n! \wedge \mathtt{x} \geqslant 0$$

Having identified a suitable loop invariant, let us take a step back and review where we are. We want to prove that our overall partial correctness specification is valid. To do this, we need to show two facts:

$$\{\mathtt{x} = n \wedge n > 0\}\ \mathtt{y} := \mathtt{1}\ \{I\} \tag{3.1}$$

$$\{I\}\ \textbf{while}\ \mathtt{x} > \mathtt{0}\ \textbf{do}\ (\mathtt{y} := \mathtt{y} * \mathtt{x};\ \mathtt{x} := \mathtt{x} - \mathtt{1})\ \{\mathtt{y} = n!\} \tag{3.2}$$

After showing that both (1) and (2) hold, we can use the rule SEQ to obtain the desired result.
To show (1), we use the ASSIGN axiom and obtain the following: $\{I[1/\mathtt{y}]\}\ \mathtt{y} := \mathtt{1}\ \{I\}$. Expanding this out, we obtain:

$$\{\mathtt{x}! * 1 = n! \wedge \mathtt{x} \geqslant 0\}\ \mathtt{y} := \mathtt{1}\ \{\mathtt{x}! * \mathtt{y} = n! \wedge \mathtt{x} \geqslant 0\}$$

With the following implication,

$$\mathtt{x} = n \wedge n > 0 \implies \mathtt{x}! * 1 = n! \wedge \mathtt{x} \geqslant 0,$$

(which can be shown by an easy calculation) we obtain (1) using the rule CONSEQUENCE.
Now let us prove (2). To use the WHILE rule, we need to show that I is an invariant for the loop:

$$\{I \wedge \mathtt{x} > 0\}\ \mathtt{y} := \mathtt{y} * \mathtt{x};\ \mathtt{x} := \mathtt{x} - \mathtt{1}\ \{I\} \tag{3.3}$$

We will show this by going backwards through the sequence of assignments:

$$\{(\mathtt{x}-1)! * \mathtt{y} = n! \wedge (\mathtt{x}-1) \geqslant 0\}\ \mathtt{x} := \mathtt{x} - \mathtt{1}\ \{I\} \tag{3.4}$$

$$\{(\mathtt{x}-1)! * \mathtt{y} * \mathtt{x} = n! \wedge (\mathtt{x}-1) \geqslant 0\}\ \mathtt{y} := \mathtt{y} * \mathtt{x}\ \{(\mathtt{x}-1)! * \mathtt{y} = n! \wedge (\mathtt{x}-1) \geqslant 0\} \tag{3.5}$$

Then, using the following implication:

$$I \wedge \mathtt{x} > 0 \implies (\mathtt{x}-1)! * \mathtt{y} * \mathtt{x} = n! \wedge (\mathtt{x}-1) \geqslant 0$$

we obtain (3) using CONSEQUENCE, (4), and (5). Thus, I is an invariant for the loop and so by WHILE we obtain,

$$\{I\}\ \textbf{while}\ \mathtt{x} > \mathtt{0}\ \textbf{do}\ (\mathtt{y} := \mathtt{y} * \mathtt{x};\ \mathtt{x} := \mathtt{x} - \mathtt{1})\ \{I \wedge \mathtt{x} \leqslant 0\}$$

To finish the proof, we just have to show

$$I \wedge \mathtt{x} \leqslant 0 \implies \mathtt{y} = n!$$

i.e.,

$$\mathtt{x}! * \mathtt{y} = n! \wedge \mathtt{x} \geqslant 0 \wedge \mathtt{x} \leqslant 0 \implies \mathtt{y} = n!$$

which holds as $\mathtt{x} \geqslant 0$ and $\mathtt{x} \leqslant 0$ implies $\mathtt{x} = 0$ and so $\mathtt{x}! = 1$. The result follows by CONSEQUENCE.
Chap ? in winscal has more info about this work plz

3.7 Decorated Programs

Doing a complete proof in Hoare Logic can feel overly verbose. The "core" of a proof consists of the preconditions and postconditions surrounding every command; with those, it's usually possible to infer the complete shape of the proof tree.

Decorating programs is a way to informally write down a Hoare logic proof of correctness. The idea is to insert assertion decorations between every "line" of the program. Using this informal evidence, you can reconstruct the full formal proof.

3.7.1 Informal Rules

We'll set out a few rules to check whether a decorated program represents a valid proof using *local consistency* checks. The rules, as you'll see, are informal reflections of the formal inference rules for Hoare logic.

Skip. For **skip**, the precondition and postcondition should be the same, like this:

$$\begin{array}{l}\{P\}\\ \textbf{skip}\\ \{P\}\end{array}$$

Sequence. For sequences, a new assertion R appears between the two commands. The two halves $\{P\}\ c_1\ \{R\}$ and $\{R\}\ c_2\ \{Q\}$ must be (recursively) locally consistent:

$$\begin{array}{l}\{P\}\\ c_1;\\ \{R\}\\ c_2\\ \{Q\}\end{array}$$

Assignment. Assignments are locally consistent when the precondition is the same as the postcondition except that it substitutes the assigned expression in for the variable:

$$\begin{array}{l}\{P[a/x]\}\\ x := a\\ \{P\}\end{array}$$

Conditions. An **if** is locally consistent when both branches are locally consistent after adding the branch condition to each:

$$\begin{array}{l}\{P\}\\ \textbf{if } b \textbf{ then}\\ \quad \{P \wedge b\}\\ \quad c_1\\ \quad \{Q\}\\ \textbf{else}\\ \quad \{P \wedge \neg b\}\\ \quad c_2\\ \quad \{Q\}\\ \{Q\}\end{array}$$

Loops. A **while** command should be decorated with a loop invariant:

$$\begin{array}{l}\{P\}\\ \textbf{while } b \textbf{ do}\\ \quad \{P \wedge b\}\\ \quad c\\ \quad \{P\}\\ \{P \wedge \neg b\}\end{array}$$

Implication. To capture the CONSEQUENCE rule, you can always write a (valid) implication to connect two commands:

$$\begin{array}{l}\{P\} \Rightarrow\\ \{Q\}\end{array}$$

Note:

It's possible to give a less redundant definition of these rules (less decorated), but this won't have the property of having a pre and post condition for every command.

3.7.2 Decorating a Program

These informal rules tell you how to verify whether a decorated program is correct, but they don't by themselves tell you how to construct those decorations (and thereby a proof). You can do this almost mechanically—except for loop invariants, which still require some creativity to concoct.
Here's an example program:

$$\textbf{while } (0 < \mathtt{y}) \textbf{ do } (\ \mathtt{x} := \mathtt{x} + 1;\ \mathtt{y} := \mathtt{y} - 1\ )$$

The first step is to decide what we want to prove about this program. So we write a precondition and postcondition for the whole program. Intuitively, the program adds y onto the initial value of the variable x, so we'll assert that:

$$\begin{array}{l}\{\mathtt{x} = m \wedge \mathtt{y} = n \wedge 0 \leqslant n\}\\ \textbf{while } (0 < \mathtt{y}) \textbf{ do } (\\ \quad \mathtt{x} := \mathtt{x} + 1;\\ \quad \mathtt{y} := \mathtt{y} - 1\\ )\\ \{\mathtt{x} = m + n\}\end{array}$$

This program is a while command, so the next step is to come up with a loop invariant. We'll set up the structure first. We need an assertion I where we can "connect" the loop's precondition and postcondition to our overall precondition and postcondition using implications, like this:

$$\begin{array}{l}\{\mathtt{x} = m \wedge \mathtt{y} = n \wedge 0 \leqslant n\} \Rightarrow\\ \{I\}\\ \textbf{while } (0 < \mathtt{y}) \textbf{ do } (\\ \quad \{I \wedge 0 < y\}\\ \quad \mathtt{x} := \mathtt{x} + 1;\\ \quad \mathtt{y} := \mathtt{y} - 1\\ \quad \{I\}\\ )\\ \{I \wedge 0 \not< \mathtt{y}\} \Rightarrow\\ \{\mathtt{x} = m + n\}\end{array}$$

On every iteration of the loop, the variable x is the sum we want, $m + n$, less the current value of y, which is the number of iterations remaining. So we'll define the invariant I like this:

$$I ::= (\mathtt{x} = m + n - \mathtt{y}) \wedge 0 \leqslant \mathtt{y}$$

The top implication follows because $n - \mathtt{y} = 0$, and the bottom implication is valid because $0 \not< \mathtt{y}$ and $0 \leqslant \mathtt{y}$ together imply $\mathtt{y} = 0$.
To finish decorating the program, we need an assertion between the two lines in the body of the loop. By the rule for assignments, we know that this must be $I[\mathtt{y} - 1/\mathtt{y}]$, or:

$$P_1 ::= (\mathtt{x} = m + n - (\mathtt{y} - 1)) \wedge (0 \leqslant \mathtt{y} - 1)$$

To make the assignment to x locally consistent, then, its precondition must be $P_1[\mathtt{x} + 1/\mathtt{x}]$, or:

$$P_2 ::= (\mathtt{x} + 1 = m + n - (\mathtt{y} - 1)) \wedge (0 \leqslant \mathtt{y} - 1)$$

It's straightforward to see that the precondition at the top of the loop, $I \wedge 0 < \mathtt{y}$, implies this new assertion P_2. Now we can write our complete decorated program using these definitions:

$\{\mathrm{x} = m \wedge \mathrm{y} = n \wedge 0 \leqslant n\} \Rightarrow$
$\{I\}$
while $(0 < \mathrm{y})$ **do** (
    $\{I \wedge 0 < y\} \Rightarrow$
    $\{P_2\}$
    $\mathrm{x} := \mathrm{x} + 1;$
    $\{P_1\}$
    $\mathrm{y} := \mathrm{y} - 1$
    $\{I\}$
)
$\{I \wedge 0 \not< \mathrm{y}\} \Rightarrow$
$\{\mathrm{x} = m + n\}$

3.8 Weakest Preconditions

Previously, we discussed the issue of completeness—i.e., whether it is possible to derive every valid partial correctness specification using the axioms and rules of Hoare logic. Unfortunately, if treated as a pure deductive system, Hoare logic cannot be complete. To see why, consider the following partial correctness specifications:

$$\{\textbf{true}\}\ \textbf{skip}\ \{P\} \qquad \{\textbf{true}\}\ c\ \{\textbf{false}\}$$

The first is valid if and only if the assertion P is valid while the second is valid if and only if the command c does not halt.
It turns out that the culprit is the CONSEQUENCE rule,

$$\frac{\models (P \Rightarrow P') \qquad \{P'\}\ c\ \{Q'\} \qquad \models (Q' \Rightarrow Q)}{\{P\}\ c\ \{Q\}}\ \text{CONSEQUENCE}$$

which includes two premises about the validity of implications between the assertions involved. Proving those implications for our two examples would require proving arbitrary logical predicates P and proving that arbitrary IMP programs c terminate. By Gödel's incompleteness theorem, we know that there is no consistent mathematical proof system that can do the former. Therefore, Hoare logic cannot be complete in that sense.
On the other hand, Hoare logic does enjoy this property:

Theorem 3.8.1 Relative Completeness of Hoare Logic
$\forall P, Q \in \textbf{Assn}, c \in \textbf{Com}.\ \models \{P\}\ c\ \{Q\}$ implies $\vdash \{P\}\ c\ \{Q\}$.

This result states that Hoare logic is no more incomplete than the language of assertions—i.e., if we had an oracle that could decide the validity of assertions, then we could decide the validity of partial correctness specifications. In other words, for any valid Hoare triple, there exists a proof in Hoare logic—as long as you have some external way to decide the validity of the premises to the CONSEQUENCE rule.

Cook's proof of relative completeness depends on the notion of *weakest liberal preconditions*. Given a command c and a postcondition Q, the weakest liberal precondition is the weakest assertion P such that $\{P\}\ c\ \{Q\}$ is valid. Here, "weakest" means that any other valid precondition implies P. That is, P most accurately describes input states for which c either does not terminate or ends up in a state satisfying Q.

Definition 3.8.1: Weakest Liberal Precondition

Formally, an assertion P is a weakest liberal precondition of c and Q if:

$$\forall \sigma, I.\ \sigma \models_I P \iff (C[\![c]\!]\,\sigma) \text{ is undefined } \vee\ (C[\![c]\!]\sigma) \models_I Q$$

Note:
The interpretation is the same to check P and Q, meaning the program can't change the set of logical variables.

We write $\mathrm{wlp}(c, Q)$ for the weakest liberal precondition of command c and postcondition Q. From left-to-right, the formula above states that $\mathrm{wlp}(c, Q)$ is a valid precondition: $\models \{P\}\ c\ \{Q\}$. The right-to-left implication says it

is the weakest valid precondition: if another assertion R satisfies $\models \{R\}\ c\ \{Q\}$, then R implies P. It can be shown that weakest liberal preconditions are unique modulo equivalence.
We can calculate the weakest liberal precondition of a command as follows:

$$\begin{aligned}
\mathrm{wlp}(\textbf{skip}, P) &= P \\
\mathrm{wlp}(x := a, P) &= P[a/x] \\
\mathrm{wlp}((c_1; c_2), P) &= \mathrm{wlp}(c_1, \mathrm{wlp}(c_2, P)) \\
\mathrm{wlp}(\textbf{if } b \textbf{ then } c_1 \textbf{ else } c_2, P) &= (b \implies \mathrm{wlp}(c_1, P)) \wedge (\neg b \implies \mathrm{wlp}(c_2, P))
\end{aligned}$$

The definition of $\mathrm{wlp}(\textbf{while } b \textbf{ do } c, P)$ is slightly more complicated—it encodes the weakest liberal precondition for each iteration of the loop. To give the intuition, first define the weakest liberal precondition for a loop that terminates in i steps as follows:

$$\begin{aligned}
F_0(P) &= \textbf{true} \\
F_{i+1}(P) &= (\neg b \implies P) \wedge (b \implies \mathrm{wlp}(c, F_i(P)))
\end{aligned}$$

We can then express the weakest liberal precondition using an infinitary conjunction:

$$\mathrm{wlp}(\textbf{while } b \textbf{ do } c, P) = \bigwedge_i F_i(P)$$

See Winskel Chapter 7 for the details of how to encode the weakest liberal precondition for a while loop as an ordinary assertion.
To check that our definition is correct, we can prove (how?) that it yields a valid partial correctness specification:

Lenma 3.8.1 Correctness of Weakest Preconditions

$$\forall c \in \textbf{Com}, Q \in \textbf{Assn}.\ \models \{\mathrm{wlp}(c, Q)\}\ c\ \{Q\} \text{ and } \forall R \in \textbf{Assn}.\ \models \{R\}\ c\ \{Q\} \text{ implies } (R \implies \mathrm{wlp}(c, Q))$$

It is not hard to prove that it also yields a provable specification:

Lenma 3.8.2 Provability of Weakest Preconditions
$\forall c \in \textbf{Com}, Q \in \textbf{Assn}.\ \vdash \{\mathrm{wlp}(c, Q)\}\ c\ \{Q\}$

Relative completeness follows by a simple argument:

Sketch: Let c be a command and let P and Q be assertions such that the partial correctness specification $\{P\}\ c\ \{Q\}$ is valid. By Lemma 1 we have $\models P \implies \mathrm{wlp}(c, Q)$. By Lemma 2 we have $\vdash \{\mathrm{wlp}(c, Q)\}\ c\ \{Q\}$. We conclude $\vdash \{P\}\ c\ \{Q\}$ using the CONSEQUENCE rule.

Chapter 4

Separation Logic

So far, we have looked at a version of Hoare logic that supports reasoning about IMP programs in terms of pre- and post-conditions on stores. In this lecture, we consider a language with pointers and dynamic memory allocation. We define a big-step operational semantics for the language as a relation on program states (σ, h) with a store and a heap. We show how Hoare logic fails to support modular reasoning due to aliasing between pointers. Finally, we introduce *separation logic*, which extends Hoare logic's assertions with predicates on heaps and provides a *frame* rule that supports modular proofs.

4.1 IMP with Heaps

As a first step, we extend IMP with standard commands for manipulating values on the heap. Note: in this lecture we will write e for arithmetic expressions rather than a, as it is more intuitive:

$$\begin{aligned} c \ ::= \ & \textbf{skip} \mid x := e \mid c_1; c_2 \mid \textbf{if } b \textbf{ then } c_1 \textbf{ else } c_2 \mid \textbf{while } b \textbf{ do } c \\ & \mid x := \textbf{new}(e) \mid \textbf{free}(e) \mid x := *e \mid *e_1 := e_2 \end{aligned}$$

The new commands in this language include:

- $x := \textbf{new}(e)$, which evaluates e to a value v, allocates storage for v on the heap, and assigns the pointer to that storage to x in the store;
- $x := *e$, which evaluates e to a pointer p, loads the value v associated with p from the heap, and assigns v to x in the store;
- $*e_1 := e_2$, which evaluates e_1 to a pointer p and e_2 to a value v, and then stores v at the location associated with p on the heap; and
- $\textbf{free}(e)$, which evaluates e to a pointer p and deallocates the heap storage associated with p.

4.2 Big-Step Operational Semantics

To define the semantics of this extension of IMP, we will need states that keep track both of the local variables as well as the heap. We will model states as pairs (σ, h) where

- $\sigma \in \text{Var} \to \mathbb{Z}$ is the store, which for simplicity we take to be a total function, and
- $h \in \text{Addr} \rightharpoonup_{\text{fin}} \mathbb{Z}$ is the heap, a finite partial function.

Here $\text{Addr} \subseteq \mathbb{Z} \setminus \{0\}$ is the set of addresses. Pointers are just integers, so they can be stored in variables, stored on the heap, and computed with arithmetic expressions. We exclude 0 so that it can serve as the null pointer: no cell is ever stored at address 0. As usual, we write $\text{dom}(h)$ for the domain of the heap and $h[p \mapsto v]$ for the heap that maps p to v and otherwise behaves like h. In the formal semantics, commands evaluate in one big step to an outcome ω, which is either a final state or a memory error:

$$\langle \sigma, h, c \rangle \Downarrow \omega \qquad\qquad \omega \ ::= \ (\sigma', h') \mid \text{err}$$

while arithmetic and boolean expressions remain pure,

$$\langle \sigma, e \rangle \Downarrow v$$

Basic commands

$$\frac{}{\langle \sigma, h, \mathbf{skip} \rangle \Downarrow (\sigma, h)}\ \textsc{Skip} \qquad \frac{\langle \sigma, e \rangle \Downarrow v}{\langle \sigma, h, x := e \rangle \Downarrow (\sigma[x \mapsto v], h)}\ \textsc{Assign}$$

$$\frac{\langle \sigma, h, c_1 \rangle \Downarrow (\sigma_1, h_1) \qquad \langle \sigma_1, h_1, c_2 \rangle \Downarrow \omega}{\langle \sigma, h, c_1; c_2 \rangle \Downarrow \omega}\ \textsc{Seq}$$

Conditionals and loops

$$\frac{\langle \sigma, b \rangle \Downarrow \text{true} \qquad \langle \sigma, h, c_1 \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \rangle \Downarrow \omega}\ \textsc{If-True} \qquad \frac{\langle \sigma, b \rangle \Downarrow \text{false} \qquad \langle \sigma, h, c_2 \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \rangle \Downarrow \omega}\ \textsc{If-False}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{false}}{\langle \sigma, h, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow (\sigma, h)}\ \textsc{While-False}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{true} \qquad \langle \sigma, h, c \rangle \Downarrow (\sigma_1, h_1) \qquad \langle \sigma_1, h_1, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \omega}\ \textsc{While-True}$$

Heap commands

$$\frac{\langle \sigma, e \rangle \Downarrow p \qquad p \in \mathrm{dom}(h) \qquad h(p) = v}{\langle \sigma, h, x := *e \rangle \Downarrow (\sigma[x \mapsto v], h)}\ \textsc{Load} \qquad \frac{\langle \sigma, e_1 \rangle \Downarrow p \qquad \langle \sigma, e_2 \rangle \Downarrow v \qquad p \in \mathrm{dom}(h)}{\langle \sigma, h, *e_1 := e_2 \rangle \Downarrow (\sigma, h[p \mapsto v])}\ \textsc{Store}$$

$$\frac{\langle \sigma, e \rangle \Downarrow v \qquad p \in \mathrm{Addr} \setminus \mathrm{dom}(h)}{\langle \sigma, h, x := \mathbf{new}(e) \rangle \Downarrow (\sigma[x \mapsto p], h[p \mapsto v])}\ \textsc{New} \qquad \frac{\langle \sigma, e \rangle \Downarrow p \qquad p \in \mathrm{dom}(h)}{\langle \sigma, h, \mathbf{free}(e) \rangle \Downarrow (\sigma, h|_{\mathrm{dom}(h) \setminus \{p\}})}\ \textsc{Free}$$

Here $h|_A$ is the restriction of h to the addresses in A. Note that New may choose any unused address p, so the semantics is nondeterministic.

Memory errors A load, store, or free at an address that is not allocated produces a memory error, and errors propagate through sequencing and loops:

$$\frac{\langle \sigma, e \rangle \Downarrow p \qquad p \notin \mathrm{dom}(h)}{\langle \sigma, h, x := *e \rangle \Downarrow \mathrm{err}}\ \textsc{Load-Err} \qquad \frac{\langle \sigma, e_1 \rangle \Downarrow p \qquad p \notin \mathrm{dom}(h)}{\langle \sigma, h, *e_1 := e_2 \rangle \Downarrow \mathrm{err}}\ \textsc{Store-Err}$$

$$\frac{\langle \sigma, e \rangle \Downarrow p \qquad p \notin \mathrm{dom}(h)}{\langle \sigma, h, \mathbf{free}(e) \rangle \Downarrow \mathrm{err}}\ \textsc{Free-Err} \qquad \frac{\langle \sigma, h, c_1 \rangle \Downarrow \mathrm{err}}{\langle \sigma, h, c_1; c_2 \rangle \Downarrow \mathrm{err}}\ \textsc{Seq-Err}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{true} \qquad \langle \sigma, h, c \rangle \Downarrow \mathrm{err}}{\langle \sigma, h, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \mathrm{err}}\ \textsc{While-Err}$$

The rules for Seq, If, and While-True above already pass along an error from their last premise. Since $0 \notin \mathrm{Addr}$, any load, store, or free at address 0 produces err. No other command can fail: the store is total, so expressions always evaluate, and we assume that Addr is infinite, so New can always find an unused address in the finite heap. Thanks to the outcome err, a program that crashes is now distinguishable from one that runs forever, which has no derivation at all.

4.3 The Rule of Constancy

The following rule is admissible in standard Hoare logic:

$$\frac{\vdash \{P\}\ c\ \{Q\} \qquad \mathrm{fv}(R) \cap \mathrm{mod}(c) = \emptyset}{\vdash \{P \wedge R\}\ c\ \{Q \wedge R\}}\ \textsc{Constancy}$$

Here $\mathrm{fv}(R)$ are the free variables of R and $\mathrm{mod}(c)$ are the variables that c may modify. For example, $\mathrm{fv}(y = 0) = \{y\}$ and $\mathrm{mod}(x := x + 1) = \{x\}$. Intuitively, the rule of constancy captures a form of local reasoning. It allows us to first prove a simple Hoare triple and then extend it to a more complicated triple by conjoining the same predicate to the pre- and post-conditions. For instance, using this rule we can strengthen

$$\{x > 0\}\ x := x + 1\ \{x > 1\}$$

to

$$\{x > 0 \wedge y = 0\}\ x := x + 1\ \{x > 1 \wedge y = 0\}$$

Free and Modified Variables The function $\mathrm{fv}(P)$ returns the set of program variables that occur free in assertion P.

$$\begin{aligned}
\mathrm{fv}(n) = \mathrm{fv}(\mathrm{true}) = \mathrm{fv}(\mathrm{false}) &= \emptyset \\
\mathrm{fv}(x) &= \{x\} \\
\mathrm{fv}(a_1 + a_2) = \mathrm{fv}(a_1 \times a_2) = \mathrm{fv}(a_1 \leq a_2) &= \mathrm{fv}(a_1) \cup \mathrm{fv}(a_2) \\
\mathrm{fv}(P \wedge Q) = \mathrm{fv}(P \vee Q) = \mathrm{fv}(P \Rightarrow Q) &= \mathrm{fv}(P) \cup \mathrm{fv}(Q) \\
\mathrm{fv}(\neg P) = \mathrm{fv}(\forall i.\, P) = \mathrm{fv}(\exists i.\, P) &= \mathrm{fv}(P)
\end{aligned}$$

The quantifier cases do not remove anything, because i is a logical variable, not a program variable. The function $\mathrm{mod}(c)$ returns the set of program variables modified by command c.

$$\begin{aligned}
\mathrm{mod}(\mathbf{skip}) = \mathrm{mod}(*e_1 := e_2) = \mathrm{mod}(\mathbf{free}(e)) &= \emptyset \\
\mathrm{mod}(x := e) = \mathrm{mod}(x := *e) = \mathrm{mod}(x := \mathbf{new}(e)) &= \{x\} \\
\mathrm{mod}(c_1; c_2) = \mathrm{mod}(\mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2) &= \mathrm{mod}(c_1) \cup \mathrm{mod}(c_2) \\
\mathrm{mod}(\mathbf{while}\ b\ \mathbf{do}\ c) &= \mathrm{mod}(c)
\end{aligned}$$

Issues Related to Aliasing Unfortunately, the rule of constancy is not sound when we extend our language of assertions with heap predicates. For example, suppose that we add an assertion $e_1 \hookrightarrow e_2$, which says that the heap contains a cell at address e_1 holding e_2 (and possibly other cells as well). Now consider trying to use the rule of constancy to add a "constant fact" about the heap. We might first prove the following triple:

$$\{\, x \hookrightarrow - \,\}\ *x := 4\ \{\, x \hookrightarrow 4 \,\}.$$

and then use the rule of constancy to obtain:

$$\{\, x \hookrightarrow - \ \wedge\ y \hookrightarrow 3 \,\}\ *x := 4\ \{\, x \hookrightarrow 4 \ \wedge\ y \hookrightarrow 3 \,\}.$$

Here $\mathrm{mod}(*x := 4) = \emptyset$, so the side condition of the rule is satisfied trivially. However, if x and y are aliases for each other, then the postcondition is false. Of course, it is possible to complicate the pre- and post-condition to capture disjointness predicates, but this quickly becomes impractical—in principle, every pointer on the heap might be aliased to every other pointer, which leads to a combinatorial explosion. Such complications motivated the development of separation logic.

4.4 Heap Assertions

Before defining separation logic, let us extend our language of assertions with heap predicates:

$$H, J, K \ ::=\ \mathrm{emp} \mid [P] \mid e_1 \hookrightarrow e_2 \mid H_1 \star H_2 \mid H_1 \mathrel{-\!\!*} H_2 \mid H_1 \mathrel{-\!\!\!\wedge} H_2 \mid H_1 \mathrel{-\!\!\!\vee} H_2 \mid \forall i.\, H \mid \exists i.\, H$$

Intuitively, these predicates can be understood as follows.

- emp : heap is empty,
- $[P]$: heap is empty and P holds,
- $e_1 \hookrightarrow e_2$: heap consists of exactly one cell with pointer e_1 and value e_2 (this is stricter than the informal reading of $\hookrightarrow$ in the previous section),

- $H_1 \star H_2$: heap splits into two disjoint pieces, one satisfying H_1 and the other H_2,
- $H_1 \mathbin{\curlywedge} H_2$ and $H_1 \mathbin{\curlyvee} H_2$ are heap assertion versions of the standard boolean connectives, and
- $\forall i.\, H$ and $\exists i.\, H$ are heap assertion versions of the standard quantifiers.

More formally, we can model their semantics as follows:

$$
\begin{array}{ll}
(\sigma, h) \models_I \mathrm{emp} & \text{if } h = \emptyset \\
(\sigma, h) \models_I [P] & \text{if } h = \emptyset \wedge \sigma \models_I P \\
(\sigma, h) \models_I e_1 \hookrightarrow e_2 & \text{if } h = \{(p, v)\} \text{ where } \langle \sigma, e_1 \rangle_I \Downarrow p \text{ and } \langle \sigma, e_2 \rangle_I \Downarrow v \\
(\sigma, h) \models_I H_1 \star H_2 & \text{if } \exists h_1, h_2.\ h = h_1 \uplus h_2 \wedge (\sigma, h_1) \models_I H_1 \wedge (\sigma, h_2) \models_I H_2 \\
(\sigma, h) \models_I H_1 \mathbin{\curlywedge} H_2 & \text{if } (\sigma, h) \models_I H_1 \text{ and } (\sigma, h) \models_I H_2 \\
(\sigma, h) \models_I H_1 \mathbin{\curlyvee} H_2 & \text{if } (\sigma, h) \models_I H_1 \text{ or } (\sigma, h) \models_I H_2 \\
(\sigma, h) \models_I \forall i.\, H & \text{if } (\sigma, h) \models_{I[i \mapsto v]} H \text{ for all } v \\
(\sigma, h) \models_I \exists i.\, H & \text{if } (\sigma, h) \models_{I[i \mapsto v]} H \text{ for some } v
\end{array}
$$

Here $h_1 \uplus h_2$ is the union of heaps h_1 and h_2 with disjoint domains (it is undefined if they overlap). Free variables extend to heap assertions in the obvious way, e.g., $\mathrm{fv}(e_1 \hookrightarrow e_2) = \mathrm{fv}(e_1) \cup \mathrm{fv}(e_2)$, $\mathrm{fv}(\mathrm{emp}) = \emptyset$, and $\mathrm{fv}([P]) = \mathrm{fv}(P)$. Quantifiers bind logical variables (never program variables), which range over integers, including addresses, and over finite lists of integers; I maps each to a value of the right kind. We typically write i, j, a, b for integers and L for lists. We will often abbreviate $e \hookrightarrow - \triangleq \exists i.\, e \hookrightarrow i$. As heaps only contain addresses, $0 \hookrightarrow v$ is unsatisfiable for every v: there is no heap whose single cell is at address 0. Similarly, predicates like $x \hookrightarrow 5 \star x \hookrightarrow 5$ are unsatisfiable as the separating conjunction operator, $\star$, enforces disjointness—even when the two cells would hold the same value.

Equivalence and entailment. Two heap assertions are equivalent, written $H_1 \equiv H_2$, if they hold in exactly the same states:

$$H_1 \equiv H_2 \triangleq \forall \sigma, h, I.\ (\sigma, h) \models_I H_1 \iff (\sigma, h) \models_I H_2.$$

(Note that it is not enough for H_1 and H_2 to be both valid or both invalid—for example, $x \hookrightarrow 1$ and $x \hookrightarrow 2$ are both invalid, but they are certainly not equivalent.) Similarly, H_1 *entails* H_2, written $H_1 \Rrightarrow H_2$, if every state satisfying H_1 also satisfies H_2. Separating conjunction satisfies several familiar algebraic laws:

$$
\begin{array}{rl}
H \star J \equiv J \star H & \text{(commutativity)} \\
(H \star J) \star K \equiv H \star (J \star K) & \text{(associativity)} \\
H \star \mathrm{emp} \equiv H & \text{(unit)}
\end{array}
$$

However, $\star$ and $\mathbin{\curlywedge}$ behave differently. In general, $[P] \star H \not\equiv [P] \mathbin{\curlywedge} H$: the left-hand side says that P holds and H describes the whole heap, while the right-hand side requires the heap to be empty. For instance, $[\mathrm{true}] \star x \hookrightarrow 1$ is satisfiable, but $[\mathrm{true}] \mathbin{\curlywedge} x \hookrightarrow 1$ is not. When H is emp, the two do coincide: $[P] \star \mathrm{emp} \equiv [P] \mathbin{\curlywedge} \mathrm{emp}$. In practice, $[P] \star H$ is the idiomatic way to state a pure fact P alongside a heap assertion H, and we will use entailments such as $[P] \star H \Rrightarrow H$ freely.

4.5 Valid Triples

Now we define the notion of valid triples. We use the standard definition for separation logic, which is partial correctness with *fault avoidance*, following Reynolds and O'Hearn: a valid triple guarantees that the program never produces a memory error, and that if it terminates, the postcondition holds. The explicit outcome err is what makes this possible. If memory errors simply got stuck, a crashing program would have no derivation, just like a program that runs forever, and a partial correctness triple would hold vacuously for it. With fault avoidance, {emp} **free**(x) {false} is not valid, because **free**(x) produces err on the empty heap. On the other hand, {emp} **while** true **do skip** {false} is valid: the loop never crashes and never terminates. As usual for partial correctness, termination must be proved separately if it is needed.

Definition 4.5.1: Valid Triple

A triple is *valid*, written $\models \{H\}\ c\ \{J\}$, if for all σ, h, and I with $\sigma, h \models_I H$:

- $\langle \sigma, h, c \rangle \Downarrow \mathrm{err}$ is not derivable, and
- every σ', h' with $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$ satisfies $\sigma', h' \models_I J$.

Because New makes the semantics nondeterministic, both conditions quantify over all executions: a valid triple rules out an error for every possible choice of fresh addresses. Note that the precondition H describes the entire heap in which c runs, and c must not produce an error in any such heap. So a valid triple implicitly says that c only accesses cells described by H—its *footprint*. For example, $\{x \hookrightarrow 3\}\ *x := 4\ \{x \hookrightarrow 4\}$ is valid, but $\{\mathrm{emp}\}\ *x := 4\ \{\mathrm{emp}\}$ is not, because $*x := 4$ produces err on the empty heap.

4.6 The Frame Rule

A major appeal of separation logic is that it supports the so-called frame rule:

$$\frac{\vdash \{H\}\ c\ \{J\} \qquad \mathrm{fv}(K) \cap \mathrm{mod}(c) = \emptyset}{\vdash \{H \star K\}\ c\ \{J \star K\}}\ \textsc{Frame}$$

The rule captures modular reasoning about the heap. Intuitively, we first prove a specification for c using only the "heap" footprint it needs. Then the command can be framed in any larger context obtained by combining the footprint with a disjoint heap.

Soundness. Why is the frame rule sound? Write $h \perp h_0$ when h and h_0 have disjoint domains. The proof rests on four facts, the first three about the semantics (each proved by induction on derivations) and the last about assertions (proved by induction on the assertion):

- *Safety monotonicity*: if $\langle \sigma, h, c \rangle \Downarrow \mathrm{err}$ is not derivable, then neither is $\langle \sigma, h \uplus h_0, c \rangle \Downarrow \mathrm{err}$, for any $h_0 \perp h$. (If a load, store, or free finds its address in h, it also finds it in $h \uplus h_0$.)
- *Frame property*: if $\langle \sigma, h, c \rangle \Downarrow \mathrm{err}$ is not derivable and $\langle \sigma, h \uplus h_0, c \rangle \Downarrow (\sigma', h')$, then $h' = h'_1 \uplus h_0$ for some h'_1 with $\langle \sigma, h, c \rangle \Downarrow (\sigma', h'_1)$. In other words, a run in the larger heap leaves h_0 untouched and behaves like a run in h.
- *Unmodified variables*: if $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$, then $\sigma'(y) = \sigma(y)$ for every $y \notin \mathrm{mod}(c)$.
- *Coincidence* (also called the agreement or relevance lemma): if σ and σ' agree on $\mathrm{fv}(K)$, then $\sigma, h \models_I K$ if and only if $\sigma', h \models_I K$. That is, an assertion depends on the store only through its free variables.

The interesting case of the frame property is New. The run in $h \uplus h_0$ allocates some $p \notin \mathrm{dom}(h \uplus h_0)$. Since $\mathrm{dom}(h) \subseteq \mathrm{dom}(h \uplus h_0)$, this p is also unused in h, so the run in h alone can pick the same p, and the results match: $(h \uplus h_0)[p \mapsto v] = h[p \mapsto v] \uplus h_0$. This is exactly why New is allowed to choose any unused address. With a deterministic allocator, say $p = 1 + \max \mathrm{dom}(h)$, the two runs could allocate different addresses, and the frame rule would be unsound: on the empty heap the allocator returns 1, so $\{\mathrm{emp}\}\ x := \mathbf{new}(1)\ \{[x = 1] \star x \hookrightarrow 1\}$ would be valid, but framing in $1 \hookrightarrow 5$ gives a false triple, because the allocator would now return 2. **Theorem (Soundness of Frame).** If $\models \{H\}\ c\ \{J\}$ and $\mathrm{fv}(K) \cap \mathrm{mod}(c) = \emptyset$, then $\models \{H \star K\}\ c\ \{J \star K\}$. *Proof.* Suppose $\sigma, h \models_I H \star K$. Then $h = h_1 \uplus h_0$ with $\sigma, h_1 \models_I H$ and $\sigma, h_0 \models_I K$. By the premise, $\langle \sigma, h_1, c \rangle \Downarrow \mathrm{err}$ is not derivable, so by safety monotonicity neither is $\langle \sigma, h, c \rangle \Downarrow \mathrm{err}$. Now suppose $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$. By the frame property, $h' = h'_1 \uplus h_0$ with $\langle \sigma, h_1, c \rangle \Downarrow (\sigma', h'_1)$, so $\sigma', h'_1 \models_I J$ by the premise. By the unmodified-variables fact, σ' agrees with σ on $\mathrm{fv}(K)$, since $\mathrm{fv}(K) \cap \mathrm{mod}(c) = \emptyset$; so by coincidence we still have $\sigma', h_0 \models_I K$. Hence $\sigma', h' \models_I J \star K$. ∎

Why fault avoidance? The proof uses the absence of errors in an essential way. Suppose instead that memory errors simply got stuck, and triples were ordinary partial correctness statements. Then $\{\mathrm{emp}\}\ \mathbf{free}(x)\ \{\mathrm{emp}\}$ would be valid vacuously, since $\mathbf{free}(x)$ has no result on the empty heap. But framing in $x \hookrightarrow 3$ would give $\{x \hookrightarrow 3\}\ \mathbf{free}(x)\ \{x \hookrightarrow 3\}$, which is false: the free succeeds and the cell is gone. Fault avoidance rules out the first triple, because it forces a command to own every cell it touches.

Baking in the frame rule. Some modern presentations, such as Charguéraud's, take a different approach and build the frame rule into the definition of triples. There, $\{H\}\ c\ \{J\}$ is defined to be valid if, for every K with $\mathrm{fv}(K) \cap \mathrm{mod}(c) = \emptyset$, the triple $\{H \star K\}\ c\ \{J \star K\}$ is valid in the sense above. The frame rule is then sound by construction, at the cost of a less direct definition.

4.7 Small Axioms for Heap Commands

We can define "small" axioms for the heap-manipulating commands as follows.

$$\frac{x \notin \mathrm{fv}(e_1) \cup \mathrm{fv}(e_2)}{\{\, e_1 \hookrightarrow e_2 \,\}\ x := *e_1\ \{\, [x = e_2] \star e_1 \hookrightarrow e_2 \,\}}\ \textsc{Load} \qquad \frac{}{\{\, e_1 \hookrightarrow - \,\}\ *e_1 := e_2\ \{\, e_1 \hookrightarrow e_2 \,\}}\ \textsc{Store}$$

$$\frac{x \notin \mathrm{fv}(e)}{\{\, \mathrm{emp} \,\}\ x := \mathbf{new}(e)\ \{\, x \hookrightarrow e \,\}}\ \textsc{Alloc} \qquad \frac{}{\{\, e \hookrightarrow - \,\}\ \mathbf{free}(e)\ \{\, \mathrm{emp} \,\}}\ \textsc{Free}$$

In Alloc, the program variable x ends up holding the address of the new cell, so $x \hookrightarrow e$ describes that cell. The side condition is needed because e is interpreted in the final store, where x has been overwritten. A full treatment of separation logic has other proof rules similar to Hoare logic. In the examples below we also use the rules for sequencing and consequence, where consequence is stated using entailment ($\Rrightarrow$) between heap assertions, and the familiar invariant rule for loops:

$$\frac{\{[b] \star I\}\ c\ \{I\}}{\{I\}\ \mathbf{while}\ b\ \mathbf{do}\ c\ \{[\neg b] \star I\}}\ \textsc{While}$$

Note how the loop test is combined with the invariant using $\star$: $[b] \star I$ says that b holds and I describes the heap. Using $[b] \wedge I$ instead would be wrong: it would also require an empty heap.

4.8 Example: Swap

The following program swaps the contents of the cells at p and q:

$$t := *p;\ u := *q;\ *p := u;\ *q := t$$

A natural specification is the following, where a and b are logical variables:

$$\{\, p \hookrightarrow a \star q \hookrightarrow b \,\}\ \mathrm{swap}\ \{\, p \hookrightarrow b \star q \hookrightarrow a \,\}.$$

We can prove it using the small axioms, framing away the cells (and pure facts) each command does not touch. Each use of Frame is allowed, since the frame never mentions the assigned variable:

$\{\, p \hookrightarrow a \star q \hookrightarrow b \,\}$
$t := *p;$ Load, framing $q \hookrightarrow b$
$\{\, [t = a] \star p \hookrightarrow a \star q \hookrightarrow b \,\}$
$u := *q;$ Load, framing $[t = a] \star p \hookrightarrow a$
$\{\, [t = a] \star [u = b] \star p \hookrightarrow a \star q \hookrightarrow b \,\}$
$*p := u;$ Store, framing everything but $p \hookrightarrow a$
$\{\, [t = a] \star [u = b] \star p \hookrightarrow u \star q \hookrightarrow b \,\}$
$*q := t$ Store, framing everything but $q \hookrightarrow b$
$\{\, [t = a] \star [u = b] \star p \hookrightarrow u \star q \hookrightarrow t \,\} \Rrightarrow$
$\{\, p \hookrightarrow b \star q \hookrightarrow a \,\}$

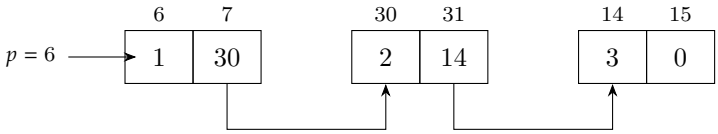
(Before each Store we also weaken $p \hookrightarrow a$ to $p \hookrightarrow -$, and similarly for q.)

Is this the only specification? No! Because $\star$ forces the two cells to be disjoint, the specification above says nothing about the case where p and q are aliases. In that case the program is the identity, which we can specify as follows:

$$\{\, [p = q] \star p \hookrightarrow a \,\}\ \mathrm{swap}\ \{\, [p = q] \star p \hookrightarrow a \,\}.$$

4.9 Linked Lists

Representation. We represent a linked list as a sequence of nodes. The node at address p occupies two consecutive cells: its value is stored at p and the address of the next node is stored at $p+1$. We use 0 as the null pointer; recall that $0 \notin \text{Addr}$, so 0 is never the address of a cell. For example, the following heap holds the list $[1,2,3]$ at address 6:



As a finite map, it is $h = \{6 \mapsto 1,\ 7 \mapsto 30,\ 30 \mapsto 2,\ 31 \mapsto 14,\ 14 \mapsto 3,\ 15 \mapsto 0\}$. Nodes need not be adjacent or in order; only the pointers matter. (Our **new**(e) allocates a single cell. A real language would allocate a whole node at once, but we will not need allocation below.)

A list predicate. To reason about lists, we define a heap predicate $\text{list}(p, L)$, meaning "p points to a list holding the mathematical list L," by recursion on L:

$$\begin{aligned} \text{list}(p, [\,]) &\triangleq [p = 0] \\ \text{list}(p, v :: L) &\triangleq \exists q.\ p \hookrightarrow v \star (p+1) \hookrightarrow q \star \text{list}(q, L) \end{aligned}$$

Unfolding the definition step by step:

$$\begin{aligned} & \text{list}(6, [1,2,3]) \\ \equiv\ & \exists q_1.\ 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star \text{list}(q_1, [2,3]) \\ \equiv\ & \exists q_1, q_2.\ 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star q_1 \hookrightarrow 2 \star (q_1+1) \hookrightarrow q_2 \star \text{list}(q_2, [3]) \\ \equiv\ & \exists q_1, q_2, q_3.\ 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star q_1 \hookrightarrow 2 \star (q_1+1) \hookrightarrow q_2 \star q_2 \hookrightarrow 3 \star (q_2+1) \hookrightarrow q_3 \star [q_3 = 0] \end{aligned}$$

The heap above satisfies the last line, with $q_1 = 30$, $q_2 = 14$, and $q_3 = 0$.

What $\star$ rules out. Because each unfolding claims new cells that are disjoint from all the others, and the list must end at 0, no list L makes $\text{list}(p, L)$ describe a cyclic structure. Similarly, $\text{list}(p_1, L_1) \star \text{list}(p_2, L_2)$ guarantees that the two lists share no nodes: a shared tail would have to belong to both halves of the heap. In Hoare logic we would need explicit "acyclic" and "disjoint" predicates to say these things. Here $\star$ gives us both for free.

Further Reading

Interested students are referred to two canonical treatments: Reynolds's lecture notes, or Charguéraud's habilitation thesis or textbook *Separation Logic Foundations* (Software Foundations, Volume 6), which this lecture is loosely based on.

In proofs, it will often be convenient to have structural rules that allow us to rearrange assertions in various ways. The Exists rule lets us reason about a command under an existential quantifier: if c takes H to J for every value of the logical variable i, then it takes $\exists i.\, H$ to $\exists i.\, J$.

$$\frac{\vdash \{H\}\ c\ \{J\}}{\vdash \{\exists i.\, H\}\ c\ \{\exists i.\, J\}}\ \textsc{Exists}$$

In the examples, we will use this rule constantly, because unfolding a recursive predicate will always introduces existentials.

4.10 Example: Disposing a List

The following program dispose frees every node of the list at p:

```
while (p ≠ 0) do (
  q := *(p + 1);
  free(p);
  free(p + 1);
  p := q
)
```

Its specification is $\{\text{list}(p, L)\}$ dispose $\{\text{emp}\}$, and a suitable loop invariant is $\exists L'.\, \text{list}(p, L')$: the heap always holds exactly the rest of the list. The precondition implies the invariant (take $L' = L$). For the loop body, the While rule requires us to show $\{[p \neq 0] \star \exists L'.\, \text{list}(p, L')\}$ body $\{\exists L'.\, \text{list}(p, L')\}$:

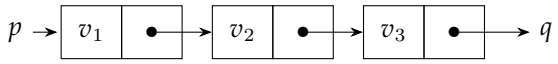
$\{(\exists L'.\, \text{list}(p, L')) \star [p \neq 0]\} \Rightarrow$ (unfold)
$\{\exists v, y, L''.\ p \hookrightarrow v \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\}$
$q := *(p + 1);$ (Load, Frame)
$\{\exists v, y, L''.\ [q = y] \star p \hookrightarrow v \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\}$
free(p); (Free, Frame)
$\{\exists y, L''.\ [q = y] \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\}$
free$(p + 1)$; (Free, Frame)
$\{\exists y, L''.\ [q = y] \star \text{list}(y, L'')\} \Rightarrow$
$\{\exists L'.\, \text{list}(q, L')\}$
$p := q$ (assignment)
$\{\exists L'.\, \text{list}(p, L')\}$

The first implication unfolds the list: since $p \neq 0$, the list cannot be empty (as $\text{list}(p, [\,])$ would force $p = 0$), so $L' = v :: L''$ for some v and L''. Each command is then verified under the existential using Exists, and with Frame to carry along the cells it does not touch. For example, the load uses the Load axiom on the single cell $(p+1) \hookrightarrow y$ and frames $p \hookrightarrow v \star \text{list}(y, L'')$; its side condition holds because q occurs in neither $p + 1$ nor y. After **free**(p), no cell mentions v any more, so we drop $\exists v$ by Consequence. The last implication repackages what remains as the invariant with q in place of p, and the assignment axiom then gives the invariant itself.
When the loop exits, we have $(\exists L'.\, \text{list}(p, L')) \star [p = 0]$. Since $0 \hookrightarrow v$ is unsatisfiable, $\text{list}(0, L')$ forces $L' = [\,]$, and so the heap is empty, as required. Because triples are partial correctness statements, we did not need to prove that the loop terminates (although it does, since the length of L' decreases on every iteration).

4.11 List Segments

For loops that traverse a list without destroying it, we need to describe the part of the list that has already been visited. A *list segment* from p to q holding L is a list that ends at q instead of at 0:

$$\begin{aligned} \text{lseg}(p, q, [\,]) &\triangleq [p = q] \\ \text{lseg}(p, q, v :: L) &\triangleq \exists r.\ p \hookrightarrow v \star (p+1) \hookrightarrow r \star \text{lseg}(r, q, L) \end{aligned}$$



$$\mathrm{lseg}(p, q, [v_1, v_2, v_3])$$

The following facts are each proved by induction on L:

$$\mathrm{list}(p, L) \equiv \mathrm{lseg}(p, 0, L)$$

$$\mathrm{lseg}(p, x, L) \star x \hookrightarrow v \star (x+1) \hookrightarrow y \Rightarrow \mathrm{lseg}(p, y, L +\!\!+ [v]) \qquad \text{(snoc)}$$

$$\mathrm{lseg}(p, y, L) \star \mathrm{list}(y, L') \Rightarrow \mathrm{list}(p, L +\!\!+ L') \qquad \text{(concat)}$$

Snoc extends a segment by one node at its end, and concat attaches a list to the end of a segment.

4.12 Example: List Append

The following program append, appends the list at p_2 to the list at p_1, leaving the result at r.

```
x := p1;
n := *(x + 1);
while (n ≠ 0) do (
   x := n;
   n := *(x + 1)
);
*(x + 1) := p2;
r := p1
```

In the program, the variable x always points at a node of the first list, and n holds that node's next pointer; the loop walks x to the last node, and the final store overwrites its null pointer with p_2.
For example, suppose the heap initially holds the lists $[1, 2]$ at address 6 and $[3, 4]$ at address 99:

address	6	7	22	23	46	47	99	100
before	1	46	4	0	2	0	3	22
after	1	46	4	0	2	99	3	22

with $p_1 = 6$ and $p_2 = 99$. Afterwards, $x = 46$ (the last node of the first list), $n = 0$, $r = 6$, and the heap holds the list $[1, 2, 3, 4]$ at r. Only one cell changes—the last node is modified from 0 to p_2.
The specification for this program

$$\{\mathrm{list}(p_1, L_1) \star \mathrm{list}(p_2, L_2) \star [p_1 \neq 0]\}\ \mathrm{append}\ \{\mathrm{list}(r, L_1 +\!\!+ L_2)\}.$$

Two aspects of this specification deserve comment. First, the pure fact $p_1 \neq 0$ is essential: if the first list is empty, then $x = 0$, so the program accesses address 1, which need not be allocated, and it may produce a memory error. Second, $\mathrm{list}(p_2, L_2)$ does not appear in the postcondition because its cells have become part of $\mathrm{list}(r, L_1 +\!\!+ L_2)$. The precondition's $\star$ guarantees that the two lists are disjoint, so appending cannot create a cycle.
To keep the proof readable, we abbreviate a node as

$$p{:}\langle v \mid q \rangle \ \triangleq\ p \hookrightarrow v \star (p+1) \hookrightarrow q.$$

The proof has three parts: the code before the loop, which establishes the invariant; the loop body, which preserves it; and the code after the loop, which establishes the postcondition.

Before the loop. We frame $\mathrm{list}(p_2, L_2)$, which this code does not touch. Since $p_1 \neq 0$, the first list is not empty, so we can unfold its first node:

$$
\begin{array}{l}
\{\, \mathrm{list}(p_1, L_1) \star [p_1 \neq 0] \,\} \;\Rightarrow \\
\{\, \exists v, y, L_b.\ [L_1 = v :: L_b] \star p_1{:}\langle v \mid y \rangle \star \mathrm{list}(y, L_b) \,\} \\
x := p_1; \\
\{\, \exists v, y, L_b.\ [L_1 = v :: L_b] \star [x = p_1] \star x{:}\langle v \mid y \rangle \star \mathrm{list}(y, L_b) \,\} \\
n := *(x + 1); \\
\{\, \exists v, y, L_b.\ [L_1 = v :: L_b] \star [x = p_1] \star [n = y] \star x{:}\langle v \mid y \rangle \star \mathrm{list}(y, L_b) \,\} \;\Rightarrow \\
\{\, \exists v, L_b.\ [L_1 = [\,] +\!\!+ v :: L_b] \star \mathrm{lseg}(p_1, x, [\,]) \star x{:}\langle v \mid n \rangle \star \mathrm{list}(n, L_b) \,\}
\end{array}
$$

Note that the pure fact $[L_1 = v :: L_b]$ must be carried through every assertion: it is the only connection between the logical variables v and L_b and the list L_1 in the specification. In the last step we use $\mathrm{lseg}(p_1, x, [\,]) \equiv [p_1 = x]$ and replace y by n. Generalizing $[\,]$ to an arbitrary L_a, and adding back the frame, suggests the loop invariant

$$I \;\triangleq\; \exists L_a, v, L_b.\ [L_1 = L_a +\!\!+ v :: L_b] \star \mathrm{lseg}(p_1, x, L_a) \star x{:}\langle v \mid n \rangle \star \mathrm{list}(n, L_b) \star \mathrm{list}(p_2, L_2).$$

The visited part of the first list is the segment from p_1 to x; x is the current node, holding v and pointing to n; and the rest of the first list is at n. The assertion we derived is exactly I with $L_a = [\,]$, so the code before the loop establishes the invariant.

The loop body. We must show $\{I \star [n \neq 0]\}\ x := n;\ n := *(x+1)\ \{I\}$. We again frame $\mathrm{list}(p_2, L_2)$. Since $n \neq 0$, the rest of the list is not empty, so we unfold $L_b = w :: L_c$:

$$
\begin{array}{l}
\{\, I \star [n \neq 0] \,\} \;\Rightarrow\; \text{(unfold)} \\
\{\, \exists L_a, v, w, m, L_c.\ [L_1 = L_a +\!\!+ v :: w :: L_c] \\
\qquad \star\, \mathrm{lseg}(p_1, x, L_a) \star x{:}\langle v \mid n \rangle \star n{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c) \,\} \;\Rightarrow\; \text{(snoc)} \\
\{\, \exists L_a, v, w, m, L_c.\ [L_1 = L_a +\!\!+ v :: w :: L_c] \\
\qquad \star\, \mathrm{lseg}(p_1, n, L_a +\!\!+ [v]) \star n{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c) \,\} \\
x := n; \qquad \text{(assignment)} \\
\{\, \exists L_a, v, w, m, L_c.\ [L_1 = L_a +\!\!+ v :: w :: L_c] \\
\qquad \star\, \mathrm{lseg}(p_1, x, L_a +\!\!+ [v]) \star x{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c) \,\} \\
n := *(x + 1) \qquad (\textsc{Load}, \textsc{Frame}) \\
\{\, \exists L_a, v, w, m, L_c.\ [L_1 = L_a +\!\!+ v :: w :: L_c] \star [n = m] \\
\qquad \star\, \mathrm{lseg}(p_1, x, L_a +\!\!+ [v]) \star x{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c) \,\} \;\Rightarrow \\
\{\, I \,\}
\end{array}
$$

The snoc step folds the current node into the visited segment. The final implication re-establishes I with $L_a +\!\!+ [v]$ for L_a, w for v, and L_c for L_b. This fact holds because $L_a +\!\!+ v :: w :: L_c = (L_a +\!\!+ [v]) +\!\!+ w :: L_c$, and $x{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c)$ becomes $x{:}\langle w \mid n \rangle \star \mathrm{list}(n, L_c)$ because $n = m$.

After the loop. When the loop exits, we have $I \star [n = 0]$. Since $n = 0$, the rest of the list must be empty, so $L_b = [\,]$ and hence $L_1 = L_a +\!\!+ [v]$:

$$
\begin{array}{l}
\{\, I \star [n = 0] \,\} \;\Rightarrow \\
\{\, \exists L_a, v.\ [L_1 = L_a +\!\!+ [v]] \star \mathrm{lseg}(p_1, x, L_a) \star x{:}\langle v \mid 0 \rangle \star \mathrm{list}(p_2, L_2) \,\} \\
*(x + 1) := p_2; \qquad (\textsc{Store}, \textsc{Frame}) \\
\{\, \exists L_a, v.\ [L_1 = L_a +\!\!+ [v]] \star \mathrm{lseg}(p_1, x, L_a) \star x{:}\langle v \mid p_2 \rangle \star \mathrm{list}(p_2, L_2) \,\} \;\Rightarrow\; \text{(snoc)} \\
\{\, \mathrm{lseg}(p_1, p_2, L_1) \star \mathrm{list}(p_2, L_2) \,\} \\
r := p_1 \qquad \text{(assignment)} \\
\{\, \mathrm{lseg}(r, p_2, L_1) \star \mathrm{list}(p_2, L_2) \,\} \;\Rightarrow\; \text{(concat)} \\
\{\, \mathrm{list}(r, L_1 +\!\!+ L_2) \,\}
\end{array}
$$

This establishes the postcondition and completes the proof.

4.13 The Magic Wand

In standard Hoare logic, the assignment axiom $\{P[e/x]\}\ x := e\ \{P\}$ works backwards: it computes the weakest precondition of an assignment by substitution. What is the weakest precondition of a heap store such as $*x := 5$ with respect to a postcondition Q? Substitution doesn't work, because Q may refer to the cell at x under a different name—e.g., an assertion such as $y \hookrightarrow 3$ may refers to the same cell when x and y are aliases. What we would like to say is: "I hold the cell at x, together with a promise: add $x \hookrightarrow 5$, and you get Q." Our existing assertion language has no way to express such a promise, so we add a new connective, called *magic wand*.

Definition. The *magic wand*, or *separating implication*, is defined by

$$(\sigma, h) \models_I H \mathrel{-\!\!\star} J \quad \text{iff for all } h' \perp h \text{ with } (\sigma, h') \models_I H, \text{ we have } (\sigma, h \uplus h') \models_I J.$$

Intuitively, $H \mathrel{-\!\!\star} J$ describes a heap that becomes a J-heap when you add an H-heap; it is sometimes described as "J with an H-shaped hole." The wand is to $\star$ as implication is to conjunction. In particular, it satisfies modus ponens and a currying law:

$$H \star (H \mathrel{-\!\!\star} J) \Rightarrow J \qquad\qquad (K \star H \Rightarrow J) \text{ iff } (K \Rightarrow H \mathrel{-\!\!\star} J)$$

For example, $\mathrm{emp} \mathrel{-\!\!\star} H \equiv H$, and any heap satisfying $y \hookrightarrow 4$ also satisfies

$$x \hookrightarrow 3 \mathrel{-\!\!\star} (x \hookrightarrow 3 \star y \hookrightarrow 4).$$

(If x and y are aliases, no disjoint heap satisfies $x \hookrightarrow 3$, so the wand holds vacuously.)

Weakest preconditions for heap commands. With the wand, we can state the weakest preconditions of the heap commands, where "weakest" is with respect to our fault-avoiding triples:

Command	**Weakest precondition for Q**
$*e_1 := e_2$	$(e_1 \hookrightarrow -) \star ((e_1 \hookrightarrow e_2) \mathrel{-\!\!\star} Q)$
$\mathbf{free}(e)$	$(e \hookrightarrow -) \star Q$
$x := \mathbf{new}(e)$	$\forall i.\ (i \hookrightarrow e) \mathrel{-\!\!\star} Q[i/x] \quad (i \text{ fresh})$

For a store, we must own the cell at e_1 (so that the store does not fault), and the rest of the heap must turn into a Q-heap once a cell $e_1 \hookrightarrow e_2$ is added. For allocation, the rest of the heap must turn into a Q-heap once any new cell $i \hookrightarrow e$ is added, with x holding its address i, because New may choose any unused address. For example, with $Q = x \hookrightarrow 5 \star y \hookrightarrow 7$,

$$\mathrm{wp}(*x := 5,\ Q) = x \hookrightarrow - \star (x \hookrightarrow 5 \mathrel{-\!\!\star} (x \hookrightarrow 5 \star y \hookrightarrow 7)) \iff x \hookrightarrow - \star y \hookrightarrow 7,$$

by currying, since $y \hookrightarrow 7 \star x \hookrightarrow 5 \Rightarrow x \hookrightarrow 5 \star y \hookrightarrow 7$. This is exactly what we would expect.

Backward rules from forward ones. We do not need new proof rules to use these weakest preconditions. For example, the backward rule for stores follows from the forward Store axiom. Let $K = (e_1 \hookrightarrow e_2) \mathrel{-\!\!\star} Q$. Since $\mathrm{mod}(*e_1 := e_2) = \emptyset$, the Store axiom and Frame give

$$\{e_1 \hookrightarrow - \star K\}\ *e_1 := e_2\ \{e_1 \hookrightarrow e_2 \star K\}.$$

By modus ponens, $e_1 \hookrightarrow e_2 \star ((e_1 \hookrightarrow e_2) \mathrel{-\!\!\star} Q) \Rightarrow Q$, so by Consequence,

$$\{(e_1 \hookrightarrow -) \star ((e_1 \hookrightarrow e_2) \mathrel{-\!\!\star} Q)\}\ *e_1 := e_2\ \{Q\}.$$

In other words, the forward small axioms together with Frame already suffice; the wand simply lets us state weakest preconditions.

Where the magic wand shows up. The wand plays a central role in modern program logics. In Iris, for example, the rules for weakest preconditions are stated using $\mathrel{-\!\!\star}$, much like the backward rules above. Viper supports magic wands in specifications. The wand is also useful for describing partial data structures: for instance, $\mathrm{list}(x, L_2) \mathrel{-\!\!\star} \mathrm{list}(p, L_1 +\!\!+ L_2)$ describes a list with a hole, which becomes a complete list once the list at x is supplied. On the other hand, the wand is notoriously hard to automate, which is one reason why most tools reason in the forward direction, often using symbolic execution.

Further Reading

Reynolds's lecture notes cover list segments, the structural rules (including the form of the EXISTS rule used here), and backward reasoning with the magic wand. Charguéraud's textbook *Separation Logic Foundations* (Software Foundations, Volume 6) takes as primitive a variant of the EXISTS rule with the existential only in the precondition (`triple_hexists`), and derives the form used here from it (`triple_hexists2`).

Chapter 5

Lambda Calculus

Lambda calculus (or λ-calculus) was introduced by Alonzo Church in the 1930s, and developed with his students Stephen Kleene and J. Barkley Rosser, to describe functions in an unambiguous and compact manner. Many real languages are based on the lambda calculus, including Lisp, Scheme, Haskell, and ML. A key characteristic of these languages is that functions are values, just like integers and booleans are values: functions can be used as arguments to functions, and can be returned from functions.

The name "lambda calculus" comes from the use of the Greek letter lambda (λ) in function definitions. (The letter lambda has no significance.) "Calculus" means a method of calculating by the symbolic manipulation of expressions.

Intuitively, a function is a rule for determining a value from an argument. Some examples of functions in mathematics are

$$f(x) = x^3$$
$$g(y) = y^3 - 2y^2 + 5y - 6.$$

5.1 Syntax

The pure λ-calculus contains just function definitions (called *abstractions*), variables, and function application (i.e., applying a function to an argument). If we add additional data types and operations (such as integers and addition), we have an *applied* λ-calculus. In the following text, we will sometimes assume that we have integers and addition in order to give more intuitive examples.

The syntax of the pure λ-calculus is defined as follows.

$e ::= x$	variable
$\mid \lambda x.\, e$	abstraction
$\mid e_1\ e_2$	application

An abstraction $\lambda x.\, e$ is a function: variable x is the *argument*, and expression e is the *body* of the function. Note that the function $\lambda x.\, e$ doesn't have a name. Assuming we have integers and arithmetic operations, the expression $\lambda x.\, x^2$ is a function that takes an argument x and returns the square of x.

An application $e_1\ e_2$ requires that e_1 is (or evaluates to) a function, and then applies the function to the expression e_2. For example, $(\lambda x.\, x^2)\ 5$ is, intuitively, equal to 25, the result of applying the squaring function $\lambda x.\, x^2$ to 5.

Here are some examples of lambda calculus expressions.[1]

$\lambda x.\, x$	a lambda abstraction called the *identity function*
$\lambda x.\, f\ (g\ x)$	another abstraction
$(\lambda x.\, x)\ 42$	an application
$\lambda y.\, \lambda x.\, x$	an abstraction that ignores its argument and returns the identity function

[1] Lambda expressions extend as far to the right as possible. For example $\lambda x.\, x\ \lambda y.\, y$ is the same as $\lambda x.\, x\ (\lambda y.\, y)$, and is not the same as $(\lambda x.\, x)\ (\lambda y.\, y)$. Application is left associative. For example $e_1\ e_2\ e_3$ is the same as $(e_1\ e_2)\ e_3$. In general, use parentheses to make the parsing of a lambda expression clear if you are in doubt.

5.1.1 Variable binding and α-equivalence

An occurrence of a variable in an expression is either bound or free. An occurrence of a variable x in a term is *bound* if there is an enclosing $\lambda x.\, e$; otherwise, it is *free*. A *closed* term is one in which all identifiers are bound. Consider the following term:

$$\lambda x.\, (x\ (\lambda y.\, y\ a)\ x)\ y$$

Both occurrences of x are bound, the first occurrence of y is bound, the a is free, and the last y is also free, since it is outside the scope of the λy.

If a program has some variables that are free, then you do not have a complete program as you do not know what to do with the free variables. Hence, a well formed program in lambda calculus is a closed term.

The symbol λ is a *binding operator*, as it binds a variable within some scope (i.e., some part of the expression): variable x is bound in e in the expression $\lambda x.\, e$.

The name of bound variables is not important. Consider the mathematical integrals $\int_0^7 x^2\, dx$ and $\int_0^7 y^2\, dy$. They describe the same integral, even though one uses variable x and the other uses variable y in their definition. The meaning of these integrals is the same: the bound variable is just a placeholder. In the same way, we can change the name of bound variables without changing the meaning of functions. Thus $\lambda x.\, x$ is the same function as $\lambda y.\, y$. Expressions e_1 and e_2 that differ only in the name of bound variables are called *α-equivalent*, sometimes written $e_1 \equiv_\alpha e_2$.

5.1.2 Higher-order functions

In lambda calculus, functions are values: functions can take functions as arguments and return functions as results. In the pure lambda calculus, every value is a function, and every closed program that terminates produces a function!

For example, the following function takes a function f as an argument, and applies it to the value 42.

$$\lambda f.\, f\ 42$$

This function takes an argument v and returns a function that applies its own argument (a function) to v.

$$\lambda v.\, \lambda f.\, (f\ v)$$

5.2 Semantics

5.2.1 β-equivalence

Application $(\lambda x.\, e_1)\ e_2$ applies the function $\lambda x.\, e_1$ to e_2. In some ways, we would like to regard the expression $(\lambda x.\, e_1)\ e_2$ as equivalent to the expression e_1 where every (free) occurrence of x is replaced with e_2. For example, we would like to regard $(\lambda x.\, x^2)\ 5$ as equivalent to 5^2.[2]

Using our notation, we would like expressions $(\lambda x.\, e_1)\ e_2$ and $e_1\{e_2/x\}$ to be equivalent. This equivalence, between $(\lambda x.\, e_1)\ e_2$ and $e_1\{e_2/x\}$, is called *β-equivalence*. Rewriting $(\lambda x.\, e_1)\ e_2$ into $e_1\{e_2/x\}$ is called a *β-reduction*. Given a lambda calculus expression, we may, in general, be able to perform β-reductions. This corresponds to executing a lambda calculus expression.

There may be more than one possible way to β-reduce an expression. Consider, for example, $(\lambda x.\, x{+}x)\ ((\lambda y.\, y)\ 5)$. We could use β-reduction to get either $((\lambda y.\, y)\ 5) + ((\lambda y.\, y)\ 5)$ or $(\lambda x.\, x + x)\ 5$. The order in which we perform β-reductions results in different semantics for the lambda calculus.

5.2.2 Call-by-value

Call-by-value (or CBV) semantics makes sure that functions are only called on values. That is, given an application $(\lambda x.\, e_1)\ e_2$, CBV semantics makes sure that e_2 is a value before calling the function.

So, what is a value? In the pure lambda calculus, any abstraction is a value. Remember, an abstraction $\lambda x.\, e$ is a function; in the pure lambda calculus, the only values are functions. In an applied lambda calculus with integers and arithmetic operations, values also include integers. Intuitively, a value is an expression that can not be reduced/executed/simplified any further.

[2] We write $e_1\{e_2/x\}$ to mean expression e_1 with all free occurrences of x replaced with e_2. There are several different notations to express this substitution, including $[x \mapsto e_2]e_1$ (used by Pierce), $[e_2/x]e_1$ (used by Mitchell), and $e_1[e_2/x]$ (used by Winskel).

We can give small step operational semantics for call-by-value execution of the lambda calculus. Here, v can be instantiated with any value (e.g., a function).

$$\frac{e_1 \to e_1'}{e_1\ e_2 \to e_1'\ e_2} \qquad \frac{e \to e'}{v\ e \to v\ e'} \qquad \frac{}{(\lambda x.\, e)\ v \to e\{v/x\}}\ \beta\text{-Reduction}$$

We can see from these rules that, given an application $e_1\ e_2$, we first evaluate e_1 until it is a value, then we evaluate e_2 until it is a value, and then we apply the function to the value—a β-reduction.
Let's consider some examples. (These examples use an applied lambda calculus that also includes reduction rules for arithmetic expressions.)

$$\begin{aligned}(\lambda x.\, \lambda y.\, y\ x)\ (5+2)\ \lambda x.\, x+1 &\to (\lambda x.\, \lambda y.\, y\ x)\ 7\ \lambda x.\, x+1\\ &\to (\lambda y.\, y\ 7)\ \lambda x.\, x+1\\ &\to (\lambda x.\, x+1)\ 7\\ &\to 7+1\\ &\to 8\end{aligned}$$

$$\begin{aligned}(\lambda f.\, f\ 7)\ ((\lambda x.\, x\ x)\ \lambda y.\, y) &\to (\lambda f.\, f\ 7)\ ((\lambda y.\, y)\ (\lambda y.\, y))\\ &\to (\lambda f.\, f\ 7)\ (\lambda y.\, y)\\ &\to (\lambda y.\, y)\ 7\\ &\to 7\end{aligned}$$

5.2.3 Call-by-name

Call-by-name (or CBN) semantics applies the function as soon as possible. The small step operational semantics are a little simpler, as they do not need to ensure that the expression to which a function is applied is a value.

$$\frac{e_1 \to e_1'}{e_1\ e_2 \to e_1'\ e_2} \qquad \frac{}{(\lambda x.\, e_1)\ e_2 \to e_1\{e_2/x\}}\ \beta\text{-Reduction}$$

Let's consider the same examples we used for CBV.

$$\begin{aligned}(\lambda x.\, \lambda y.\, y\ x)\ (5+2)\ \lambda x.\, x+1 &\to (\lambda y.\, y\ (5+2))\ \lambda x.\, x+1\\ &\to (\lambda x.\, x+1)\ (5+2)\\ &\to (5+2)+1\\ &\to 7+1\\ &\to 8\end{aligned}$$

$$\begin{aligned}(\lambda f.\, f\ 7)\ ((\lambda x.\, x\ x)\ \lambda y.\, y) &\to ((\lambda x.\, x\ x)\ \lambda y.\, y)\ 7\\ &\to ((\lambda y.\, y)\ (\lambda y.\, y))\ 7\\ &\to (\lambda y.\, y)\ 7\\ &\to 7\end{aligned}$$

Note that the answers are the same, but the order of evaluation is different. (Later we will see languages where the order of evaluation is important, and may result in different answers.)

5.3 Lambda calculus evaluation

There are many different evaluation strategies for the λ-calculus. The most permissive is *full β reduction*, which allows any *redex*—i.e., any expression of the form $(\lambda x.\, e_1)\ e_2$—to step to $e_1\{e_2/x\}$ at any time. It is defined formally by the following small-step operational semantics rules:

$$\frac{e_1 \to e_1'}{e_1\ e_2 \to e_1'\ e_2} \qquad \frac{e_2 \to e_2'}{e_1\ e_2 \to e_1\ e_2'} \qquad \frac{e_1 \to e_1'}{\lambda x.\, e_1 \to \lambda x.\, e_1'} \qquad \frac{}{(\lambda x.\, e_1)\ e_2 \to e_1\{e_2/x\}}\ \beta$$

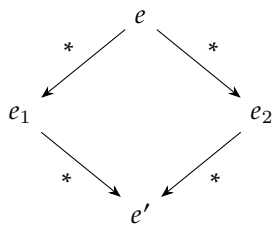
The call-by-value and call-by-name strategies from the previous section are more restrictive. Neither allows evaluation under a λ. CBV only allows an application to reduce after its argument has been reduced to a value (i.e., a λ-abstraction), while CBN allows an application to reduce even when its argument is not a value. Unlike full β reduction, both are deterministic: every expression can take at most one step.

5.4 Confluence

It is not hard to see that the full β reduction strategy is non-deterministic. This raises an interesting question: do the choices made during the evaluation of an expression affect the final result? The answer turns out to be no: full β reduction is *confluent* in the following sense:

Theorem (Church–Rosser). If $e \to^* e_1$ and $e \to^* e_2$ then there exists e' such that $e_1 \to^* e'$ and $e_2 \to^* e'$.

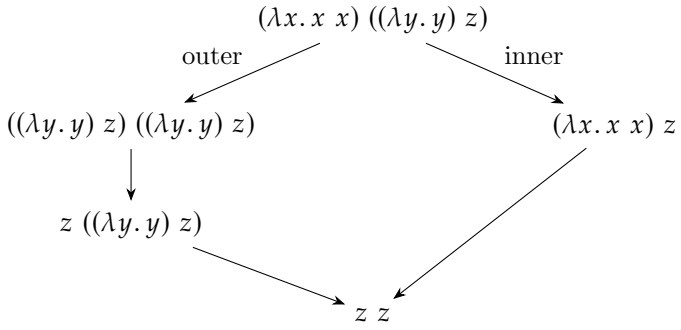
Confluence can be depicted graphically as follows:



This theorem is due to Church and Rosser [**?**], and confluence is often also called the *Church–Rosser property*. An important consequence is that every expression has at most one *normal form*—an expression that cannot step—up to α-equivalence. If $e \to^* e_1$ and $e \to^* e_2$ where e_1 and e_2 are both normal forms, then confluence gives an e' that both reduce to, which must be e_1 and e_2 themselves.

Confluence does not say that every reduction sequence reaches the normal form. For example, $(\lambda x.\, \lambda y.\, y)\ \Omega$, where Ω is the diverging expression discussed in Section 5.7, can step to the normal form $\lambda y.\, y$, but it can also step to itself forever by reducing Ω.

Interestingly, the confluence property does not hold for single steps. For example, consider the expression $e = (\lambda x.\, x\ x)\ ((\lambda y.\, y)\ z)$, which has two redexes:



Reducing the outer redex duplicates the inner one, so the left path needs two more steps to reach $z\ z$ while the right path needs only one. Hence there is no way to complete the diamond with single steps on both sides, and a naive proof of confluence by induction on single steps does not work. The standard proof, due to Tait and Martin-Löf [**?**], instead uses *parallel reduction*, which contracts any set of redexes in an expression at once. Parallel reduction does have the diamond property, and its reflexive-transitive closure coincides with $\to^*$, so confluence follows. Takahashi [**?**] gives a particularly short version of this proof.

5.5 Substitution

Each of the evaluation relations for λ-calculus has a β rule defined in terms of a substitution operation on expressions. Because the expressions involved in the substitution may share some variable names (and because we are working up to α-equivalence) the definition of this operation is slightly subtle and defining it precisely turns out to be trickier than might first appear.

As a first attempt, consider an obvious (but incorrect) definition of the substitution operator. Here we are substituting e for x in some other expression:

$$y\{e/x\} = \begin{cases} e & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$
$$(e_1\ e_2)\{e/x\} = (e_1\{e/x\})\ (e_2\{e/x\})$$
$$(\lambda y.\, e_1)\{e/x\} = \lambda y.\, e_1\{e/x\} \qquad \text{where } y \neq x$$

The intuitive idea is that the last rule relies on α-equivalence to "rewrite" abstractions that use x so they do not conflict. Unfortunately, this definition produces the wrong results when we substitute an expression with free variables under a λ. For example,

$$(\lambda y.\, x)\{y/x\} = (\lambda y.\, y)$$

To fix this problem, we need to revise our definition so that when we substitute under a λ we do not accidentally bind variables in the expression we are substituting. The following definition correctly implements *capture-avoiding substitution*:

$$y\{e/x\} = \begin{cases} e & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$
$$(e_1\ e_2)\{e/x\} = (e_1\{e/x\})\ (e_2\{e/x\})$$
$$(\lambda y.\, e_1)\{e/x\} = \lambda y.\, (e_1\{e/x\}) \qquad \text{where } y \neq x \text{ and } y \notin \mathrm{fv}(e)$$

Note that in the case for λ-abstractions, we require that the bound variable y be different from the variable x we are substituting for and that y not appear in the free variables of e, the expression we are substituting. Because we work up to α-equivalence, we can always pick y to satisfy these side conditions. For example, to calculate $(\lambda z.\, x\ z)\{(w\ y\ z)/x\}$ we first rewrite $\lambda z.\, x\ z$ to $\lambda u.\, x\ u$ and then apply the substitution, obtaining $\lambda u.\, (w\ y\ z)\ u$ as the result.

5.6 λ-calculus encodings

The pure λ-calculus contains only functions as values. It is not exactly easy to write large or interesting programs in the pure λ-calculus. We can however encode objects, such as booleans, and integers.

5.6.1 Booleans

Let us start by encoding constants and operators for booleans. That is, we want to define functions TRUE, FALSE, AND, NOT, IF, and other operators that behave as expected. For example:

$$\text{AND TRUE FALSE} = \text{FALSE}$$
$$\text{NOT FALSE} = \text{TRUE}$$
$$\text{IF TRUE } e_1\ e_2 = e_1$$
$$\text{IF FALSE } e_1\ e_2 = e_2$$

Let's start by defining TRUE and FALSE:

$$\text{TRUE} \triangleq \lambda x.\, \lambda y.\, x$$
$$\text{FALSE} \triangleq \lambda x.\, \lambda y.\, y$$

Thus, both TRUE and FALSE are functions that take two arguments; TRUE returns the first, and FALSE returns the second. We want the function IF to behave like

$$\lambda b.\, \lambda t.\, \lambda f.\ \textbf{if } b = \text{TRUE } \textbf{then } t \textbf{ else } f.$$

The definitions for TRUE and FALSE make this very easy.

$$\text{IF} \triangleq \lambda b.\, \lambda t.\, \lambda f.\, b\ t\ f$$

Definitions of other operators are also straightforward.

$$\begin{aligned} \text{NOT} &\triangleq \lambda b.\, b \text{ FALSE TRUE} \\ \text{AND} &\triangleq \lambda b_1.\, \lambda b_2.\, b_1\ b_2 \text{ FALSE} \\ \text{OR} &\triangleq \lambda b_1.\, \lambda b_2.\, b_1 \text{ TRUE } b_2 \end{aligned}$$

5.6.2 Church numerals

Church numerals encode a number n as a function that takes f and x, and applies f to x n times.

$$\begin{aligned} 0 &\triangleq \lambda f.\, \lambda x.\, x \\ 1 &\triangleq \lambda f.\, \lambda x.\, f\ x \\ 2 &\triangleq \lambda f.\, \lambda x.\, f\ (f\ x) \\ \text{SUCC} &\triangleq \lambda n.\, \lambda f.\, \lambda x.\, f\ (n\ f\ x) \end{aligned}$$

In the definition for SUCC, the expression $n\ f\ x$ applies f to x n times (assuming that variable n is the Church encoding of the natural number n). We then apply f to the result, meaning that we apply f to x $n+1$ times. Given the definition of SUCC, we can easily define addition. Intuitively, the natural number $n_1 + n_2$ is the result of applying the successor function n_1 times to n_2.

$$\text{PLUS} \triangleq \lambda n_1.\, \lambda n_2.\, n_1 \text{ SUCC } n_2$$

Similarly, multiplication adds n_2 to 0, n_1 times, and a test for zero applies the constant function returning FALSE to TRUE n times:

$$\begin{aligned} \text{TIMES} &\triangleq \lambda n_1.\, \lambda n_2.\, n_1 \text{ (PLUS } n_2) \text{ } 0 \\ \text{ISZERO} &\triangleq \lambda n.\, n \text{ } (\lambda z.\, \text{FALSE}) \text{ TRUE} \end{aligned}$$

A predecessor function PRED, with PRED $(n+1) = n$ and PRED $0 = 0$, is also definable, but it is surprisingly tricky.

5.7 Nontermination

Consider the expression $(\lambda x.\, x\ x)\ (\lambda x.\, x\ x)$, which we will refer to as Ω for brevity. Let's try evaluating Ω:

$$\begin{aligned} \Omega &= (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) \\ &\rightarrow (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) \\ &= \Omega \end{aligned}$$

Evaluating Ω never reaches a value! It is an infinite loop!
What happens if we use Ω as an actual argument to a function? Consider this program:

$$(\lambda x.(\lambda y.y))\ \Omega$$

If we use CBV semantics to evaluate the program, we must reduce Ω to a value before we can apply the function. But Ω never evaluates to a value, so we can never apply the function. Thus, under CBV semantics, this program does not terminate. If we use CBN semantics, we can apply the function immediately, without needing to reduce the actual argument to a value:

$$(\lambda x.(\lambda y.y))\ \Omega\ \rightarrow_{\text{CBN}}\ \lambda y.y$$

This difference highlights the fact that evaluation orders, like CBV and CBN, can differ in terms of termination: expressions that diverge under one evaluation order may terminate in another. This does not contradict confluence: the CBV evaluation of this program simply never finishes, so it never reaches a different result.

5.8 Recursion

Ω demonstrates that we can write nonterminating λ-terms, but so far they are not very useful. Can we write recursive functions in the λ-calculus that do actually terminate and produce a value, as we can in most "real" programming languages? It seems difficult at first because all functions are anonymous—there is no name that a function can use to refer to itself. Surprisingly, however, it is possible to make functions that behave recursively. Roughly, our strategy will build on the "self-replicating" behavior of Ω to let functions invoke copies of themselves an unbounded number of times.
Let's consider how we would like to write a factorial function:

$$\text{FACT} \triangleq \lambda n.\, \text{IF (ISZERO } n\text{) 1 (TIMES } n \text{ (FACT (PRED } n\text{)))}$$

In slightly more readable notation, this is just:

$$\text{FACT} \triangleq \lambda n.\ \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times \text{FACT } (n-1)$$

Here, as in the definition above, the name FACT is simply meant to be shorthand for the expression on the right-hand side of the equation. But FACT appears on the right-hand side of the equation as well! Trying to expand the definition would yield an infinite term. This is not a definition, it's a recursive equation.

5.8.1 Recursion Removal Trick

We can perform a "trick" to define a function FACT that satisfies the recursive equation above. First, let's define a new function FACT$'$ that looks like FACT, but takes an additional argument f. We assume that the function f will be instantiated with FACT$'$ itself.

$$\text{FACT}' \triangleq \lambda f.\, \lambda n.\ \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times (f\ f\ (n-1))$$

Note that when we call f, we pass it a copy of itself, preserving the assumption that the actual argument for f will be FACT$'$. Now we can define the factorial function FACT in terms of FACT$'$.

$$\text{FACT} \triangleq \text{FACT}'\ \text{FACT}'$$

Let's try evaluating FACT on an integer.

$$\begin{aligned}
\text{FACT } 3 &= (\text{FACT}'\ \text{FACT}')\ 3 && \text{Definition of FACT} \\
&= ((\lambda f.\, \lambda n.\ \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times (f\ f\ (n-1)))\ \text{FACT}')\ 3 && \text{Definition of FACT}' \\
&\to (\lambda n.\ \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times (\text{FACT}'\ \text{FACT}'\ (n-1)))\ 3 && \text{Application to FACT}' \\
&\to \textbf{if } 3 = 0 \textbf{ then } 1 \textbf{ else } 3 \times (\text{FACT}'\ \text{FACT}'\ (3-1)) && \text{Application to } n \\
&\to 3 \times (\text{FACT}'\ \text{FACT}'\ (3-1)) && \text{Evaluating if} \\
&\to^* \ldots \\
&\to^* 3 \times 2 \times 1 \times 1 \\
&\to^* 6
\end{aligned}$$

So we now have a technique for writing a recursive function f: write a function f' that explicitly takes a copy of itself as an argument, and then define $f \triangleq f'\ f'$.

5.8.2 Fixed point combinators

There is another way of writing recursive functions: we can express the recursive function as the fixed point of some other, higher-order function, and then take its fixed point. We saw this technique earlier in the course when we defined the denotational semantics for while loops.
Let's consider the factorial function again. The factorial function FACT is a fixed point of the following function.

$$G \triangleq \lambda f.\, \lambda n.\ \textbf{if } n = 0 \textbf{ then } 1 \textbf{ else } n \times (f\ (n-1))$$

Recall that if g is a fixed point of G, then we have $G\ g = g$. So if we had some way of finding a fixed point of G, we would have a way of defining the factorial function FACT.

There are a number of "fixed point combinators," and the (infamous) Y combinator is one of them. Thus, we can define the factorial function FACT to be simply $\mathsf{Y}\ G$, the fixed point of G. The Y combinator is defined as

$$\mathsf{Y} \triangleq \lambda f.(\lambda x.\ f\ (x\ x))\ (\lambda x.\ f\ (x\ x)).$$

It was discovered by Haskell Curry, and is one of the simplest fixed-point combinators. Note how similar its definition is to Ω.

Unfortunately, Y does not work under CBV. Evaluating $\mathsf{Y}\ G$ gives

$$\begin{aligned}\mathsf{Y}\ G &\to (\lambda x.\ G\ (x\ x))\ (\lambda x.\ G\ (x\ x)) \\ &\to G\ ((\lambda x.\ G\ (x\ x))\ (\lambda x.\ G\ (x\ x))) \\ &\to G\ (G\ ((\lambda x.\ G\ (x\ x))\ (\lambda x.\ G\ (x\ x)))) \\ &\to \cdots\end{aligned}$$

The argument to G is never a value, so evaluation never terminates. Instead, we'll use a slight variant of the Y combinator, called Z, which works under CBV. It wraps the self-application $x\ x$ in a λ, which delays it until it is needed. The Z combinator is defined as

$$\mathsf{Z} \triangleq \lambda f.(\lambda x.\ f\ (\lambda y.\ x\ x\ y))\ (\lambda x.\ f\ (\lambda y.\ x\ x\ y))$$

Let's see it in action, on our function G. Define FACT to be $\mathsf{Z}\ G$ and calculate as follows:

$$\begin{aligned}&\text{FACT} \\ &= \mathsf{Z}\ G \\ &= (\lambda f.(\lambda x.\ f\ (\lambda y.\ x\ x\ y))\ (\lambda x.\ f\ (\lambda y.\ x\ x\ y)))\ G && \text{Definition of } \mathsf{Z} \\ &\to (\lambda x.\ G\ (\lambda y.\ x\ x\ y))\ (\lambda x.\ G\ (\lambda y.\ x\ x\ y)) \\ &\to G\ (\lambda y.(\lambda x.\ G\ (\lambda y.\ x\ x\ y))\ (\lambda x.\ G\ (\lambda y.\ x\ x\ y))\ y) \\ &= (\lambda f.\lambda n.\ \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (f\ (n-1))) \\ &\quad (\lambda y.(\lambda x.\ G\ (\lambda y.\ x\ x\ y))\ (\lambda x.\ G\ (\lambda y.\ x\ x\ y))\ y) && \text{definition of } G \\ &\to \lambda n.\ \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else} \\ &\quad \textbf{else}\ n \times ((\lambda y.(\lambda x.\ G\ (\lambda y.\ x\ x\ y))\ (\lambda x.\ G\ (\lambda y.\ x\ x\ y))\ y)\ (n-1)) \\ &=_\beta \lambda n.\ \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (\lambda y.(\mathsf{Z}\ G)\ y)\ (n-1) \\ &=_\beta \lambda n.\ \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (\mathsf{Z}\ G\ (n-1)) \\ &= \lambda n.\ \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (\text{FACT}\ (n-1))\end{aligned}$$

There are many (indeed infinitely many) fixed-point combinators. Here's a cute one, due to Jan Willem Klop:

$$\mathsf{Y}_k \triangleq (L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L\ L)$$

where

$$L \triangleq \lambda a\ b\ c\ d\ e\ f\ g\ h\ i\ j\ k\ l\ m\ n\ o\ p\ q\ s\ t\ u\ v\ w\ x\ y\ z\ r.\ (r\ (t\ h\ i\ s\ i\ s\ a\ f\ i\ x\ e\ d\ p\ o\ i\ n\ t\ c\ o\ m\ b\ i\ n\ a\ t\ o\ r))$$

To gain some more intuition for fixed-point combinators, let's derive a fixed-point combinator that was originally discovered by Alan Turing [**?**]. Suppose we have a higher order function f, and want the fixed point of f. We want $\Theta\ f$ to be a fixed point of f, so we need

$$\Theta\ f = f\ (\Theta\ f).$$

This means, that we can write the following recursive equation:

$$\Theta = \lambda f.\ f\ (\Theta\ f).$$

Now we can use the recursion removal trick we described earlier. Define Θ' as $\lambda t.\lambda f.\ f\ (t\ t\ f)$, and Θ as $\Theta'\ \Theta'$. Then we have the following equalities:

$$\begin{aligned}\Theta &= \Theta'\ \Theta' \\ &= (\lambda t.\lambda f.\ f\ (t\ t\ f))\ \Theta' \\ &\to \lambda f.\ f\ (\Theta'\ \Theta'\ f) \\ &= \lambda f.\ f\ (\Theta\ f)\end{aligned}$$

Let's try out the Turing combinator on our higher order function G that we used to define FACT. This time we will use CBN evaluation.

$$
\begin{aligned}
\text{FACT} &= \Theta\ G \\
&= ((\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ (\lambda t.\,\lambda f.\,f\ (t\ t\ f)))\ G \\
&\longrightarrow (\lambda f.\,f\ ((\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ (\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ f))\ G \\
&\longrightarrow G\ ((\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ (\lambda t.\,\lambda f.\,f\ (t\ t\ f))\ G) \\
&= G\ (\Theta\ G) && \text{for brevity} \\
&= (\lambda f.\,\lambda n.\ \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (f\ (n-1)))\ (\Theta\ G) && \text{Definition of } G \\
&\longrightarrow \lambda n.\ \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times ((\Theta\ G)\ (n-1)) \\
&= \lambda n.\ \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (\text{FACT}\ (n-1))
\end{aligned}
$$

Note that the step from $G\ (\Theta\ G)$ passes the non-value $\Theta\ G$ to G, which is only allowed under CBN. Under CBV, $\Theta\ G$ diverges, just like $\mathsf{Y}\ G$. As with Z, we can fix this by delaying the self-application, using $\Theta'_v \triangleq \lambda t.\,\lambda f.\,f\ (\lambda y.\,t\ t\ f\ y)$ in place of Θ'.

5.9 de Bruijn Notation

One way to avoid the tricky interaction between free and bound names in the substitution operator is to pick a representation for expressions that doesn't have any names at all! Intuitively, we can think of a bound variable is just a pointer to the λ that binds it. For example, in $\lambda x.\lambda y.y\ x$, the y points to the first λ and the x points to the second λ.
So-called *de Bruijn* notation uses this idea as the representation for λ expressions. Here is the grammar for λ expressions in de Bruijn notation:

$$e \ ::= \ n \mid \lambda.e \mid e\ e$$

Variables are represented by integers n that refer to (the index of) their binder while lambda-abstractions have the form $\lambda.e$. Note that the the variable bound by the abstraction is not named—i.e., the representation is nameless.
As examples, here are several terms written using standard notation and in de Bruijn notation:

Standard	de Bruijn
$\lambda x.x$	$\lambda.0$
$\lambda z.z$	$\lambda.0$
$\lambda x.\lambda y.x$	$\lambda.\lambda.1$
$\lambda x.\lambda y.\lambda s.\lambda z.x\ s\ (y\ s\ z)$	$\lambda.\lambda.\lambda.3\ 1\ (2\ 1\ 0)$
$(\lambda x.x\ x)\ (\lambda x.x\ x)$	$(\lambda.0\ 0)\ (\lambda.0\ 0)$
$(\lambda x.\lambda x.x)\ (\lambda y.y)$	$(\lambda.\lambda.0)\ (\lambda.0)$

To represent a λ-expression that contains free variables in de Bruijn notation, we need a way to map the free variables to integers. We will work with respect to a map Γ from variables to integers called a *context*. As an example, if Γ maps x to 0 and y to 1, then the de Bruijn representation of $x\ y$ with respect to Γ is 0 1, while the representation of $\lambda z.\ x\ y\ z$ with respect to Γ is $\lambda.\ 1\ 2\ 0$. Note that in this second example, because we have gone under a λ, we have shifted the integers representing x and y up by one to avoid capturing them.
In general, whenever we work de Bruijn representations of expressions containing free variables (i.e., when working with respect to a context Γ) we will need to modify the indices of those variables. For example, when we substitute an expression containing free variables under a λ, we will need to shift the indices up so that they continue to refer to the same numbers with respect to Γ after the substitution as they did before. For example, if we substitute 0 1 for the variable bound by the outermost λ in $\lambda.\lambda.1$ we should get $\lambda.\lambda.2\ 3$, not $\lambda.\lambda.0\ 1$. We will use an auxiliary function that shifts the indices of free variables above a cutoff c up by i:

$$\uparrow_c^i(n) = \begin{cases} n & \text{if } n < c \\ n + i & \text{otherwise} \end{cases}$$

$$\uparrow_c^i(\lambda.e) = \lambda.(\uparrow_{c+1}^i e)$$

$$\uparrow_c^i(e_1\ e_2) = (\uparrow_c^i e_1)\ (\uparrow_c^i e_2)$$

The cutoff keeps track of the variables that were bound in the original expression and so should not be shifted as the shifting operator walks down the structure of an expression. The cutoff is 0 initially.
Using this shifting function, we can define substitution as follows:

$$n\{e/m\} = \begin{cases} e & \text{if } n = m \\ n & \text{otherwise} \end{cases}$$

$$(\lambda.e_1)\{e/m\} = \lambda.\big(e_1\{(\uparrow_0^1 e)/(m+1)\}\big)$$

$$(e_1\ e_2)\{e/m\} = (e_1\{e/m\})\ (e_2\{e/m\})$$

Note that when we go under a λ we increase the index of the variable we are substituting for and shift the free variables in the expression e up by one.
The β rule for terms in de Bruijn notation is as follows:

$$\frac{}{(\lambda.e_1)\ e_2 \ \rightarrow \ \uparrow_0^{-1}\big(e_1\{\uparrow_0^1 e_2/0\}\big)}\ \beta$$

That is, we substitute occurrences of 0, the index of the variable being bound by the λ, with e_2 shifted up by one. Then we shift the result down by one to ensure that any free variables in e_1 continue to refer to the same things after we remove the λ.
To illustrate how this works consider the following example, which we wrote as $(\lambda u.\lambda v.u\ x)\ y$ in standard notation. We will work with respect to a context where $\Gamma(x) = 0$ and $\Gamma(y) = 1$.

$$\begin{aligned}
&(\lambda.\lambda.1\ 2)\ 1\\
&\to \uparrow_0^{-1}\big((\lambda.1\ 2)\{(\uparrow_0^1 1)/0\}\big)\\
&= \uparrow_0^{-1}\big((\lambda.1\ 2)\{2/0\}\big)\\
&= \uparrow_0^{-1}\lambda.\big((1\ 2)\{(\uparrow_0^1 2)/(0+1)\}\big)\\
&= \uparrow_0^{-1}\lambda.\big((1\ 2)\{3/1\}\big)\\
&= \uparrow_0^{-1}\lambda.(1\{3/1\})\ (2\{3/1\})\\
&= \uparrow_0^{-1}\lambda.3\ 2\\
&= \lambda.2\ 1
\end{aligned}$$

which, in standard notation (with respect to Γ), is the same as $\lambda v.\, y\ x$.

5.10 Definitional translation

We have seen how to encode a number of high-level language constructs—booleans, conditionals, natural numbers, and recursion—in λ-calculus. We now consider *definitional translation*, where we define the meaning of language constructs by translation to another language. This is a form of denotational semantics, but instead of the target being mathematical objects, it is a simpler programming language (such as λ-calculus). Note that definitional translation does not necessarily produce clean or efficient code; rather, it defines the meaning of the source language constructs in terms of the target language.
For each language construct, we will define an operational semantics directly, and then give an alternate semantics by translation to a simpler language. We will start by introducing evaluation contexts, which make it easier to present the new language features succinctly.

5.10.1 Evaluation contexts

Recall the syntax and CBV operational semantics for the lambda calculus:

$$\begin{aligned}
e &::= x \mid \lambda x.\, e \mid e_1\ e_2\\
v &::= \lambda x.\, e
\end{aligned}$$

$$\frac{e_1 \to e_1'}{e_1\ e_2 \to e_1'\ e_2} \qquad \frac{e_2 \to e_2'}{v_1\ e_2 \to v_1\ e_2'} \qquad \frac{}{(\lambda x.\, e)\ v \to e\{v/x\}}\ \beta\text{-Reduction}$$

Of the operational semantics rules, only the β-reduction rule told us how to "reduce" an expression; the other two rules tell us the order to evaluate expressions—e.g., evaluate the left hand side of an application to a value first, then evaluate the right hand side of an application to a value. The operational semantics of many of the languages we will consider have this feature: there are two kinds of rules, *congruence rules* that specify evaluation order, and the *computation rules* that specify the "interesting" reductions.
Evaluation contexts are a simple mechanism that separates out these two kinds of rules. An evaluation context E (sometimes written $E[\cdot]$) is an expression with a "hole" in it, that is with a single occurrence of the special symbol $[\cdot]$ (called the "hole") in place of a subexpression. Evaluation contexts are defined using a BNF grammar that is similar to the grammar used to define the language. The following grammar defines evaluation contexts for the pure CBV λ-calculus.

$$E \ ::= \ [\cdot] \mid E\ e \mid v\ E$$

We write $E[e]$ to mean the evaluation context E where the hole has been replaced with the expression e. The

following are examples of evaluation contexts, and evaluation contexts with the hole filled in by an expression.

$$E_1 = [\cdot]\ (\lambda x.\, x) \qquad E_1[\lambda y.\, y\ y] = (\lambda y.\, y\ y)\ \lambda x.\, x$$
$$E_2 = (\lambda z.\, z\ z)\ [\cdot] \qquad E_2[\lambda x.\, \lambda y.\, x] = (\lambda z.\, z\ z)\ (\lambda x.\, \lambda y.\, x)$$
$$E_3 = ([\cdot]\ \lambda x.\, x\ x)\ ((\lambda y.\, y)\ (\lambda y.\, y)) \qquad E_3[\lambda f.\, \lambda g.\, f\ g] = ((\lambda f.\, \lambda g.\, f\ g)\ \lambda x.\, x\ x)\ ((\lambda y.\, y)\ (\lambda y.\, y))$$

Using evaluation contexts, we can define the evaluation semantics for the pure CBV λ-calculus with just two rules, one for evaluation contexts, and one for β-reduction.

$$\frac{e \to e'}{E[e] \to E[e']} \qquad \frac{}{(\lambda x.\, e)\ v \to e\{v/x\}}\ \beta\text{-Reduction}$$

Note that the evaluation contexts for the CBV λ-calculus ensure that we evaluate the left hand side of an application to a value, and then evaluate the right hand side of an application to a value before applying β-reduction.
We can specify the operational semantics of CBN λ-calculus using evaluation contexts:

$$E ::= [\cdot] \mid E\ e$$

$$\frac{e \to e'}{E[e] \to E[e']} \qquad \frac{}{(\lambda x.\, e_1)\ e_2 \to e_1\{e_2/x\}}\ \beta\text{-Reduction}$$

We'll see the benefit of evaluation contexts as we see languages with more syntactic constructs.

5.10.2 Multi-argument functions and currying

Our syntax for functions only allows function with a single argument: $\lambda x.\, e$. We could define a language that allows functions to have multiple arguments.

$$e\ ::=\ x \mid \lambda x_1, \ldots, x_n.\, e \mid e_0\ e_1\ \ldots\ e_n$$

Here, a function $\lambda x_1, \ldots, x_n.\, e$ takes n arguments, with names x_1 through x_n. In a multi argument application $e_0\ e_1\ \ldots\ e_n$, we expect e_0 to evaluate to an n-argument function, and $e_1, \ldots, e_n$ are the arguments that we will give the function.
We can define a CBV operational semantics for the multi-argument λ-calculus as follows.

$$E\ ::=\ [\cdot] \mid v_0\ \ldots\ v_{i-1}\ E\ e_{i+1}\ \ldots\ e_n$$

$$\frac{e \to e'}{E[e] \to E[e']} \qquad \frac{}{(\lambda x_1, \ldots, x_n.\, e_0)\ v_1\ \ldots\ v_n \to e_0\{v_1/x_1\}\{v_2/x_2\}\ldots\{v_n/x_n\}}\ \beta\text{-Reduction}$$

The evaluation contexts ensure that we evaluate a multi-argument application $e_0\ e_1\ \ldots\ e_n$ by evaluating each expression from left to right down to a value.
Now, the multi-argument λ-calculus isn't any more expressive that the pure λ-calculus. We can show this by showing how any multi-argument λ-calculus program can be translated into an equivalent pure λ-calculus program. We define a translation function $\mathcal{T}[\![\cdot]\!]$ that takes an expression in the multi-argument λ-calculus and returns an equivalent expression in the pure λ-calculus. That is, if e is a multi-argument lambda calculus expression, $\mathcal{T}[\![e]\!]$ is a pure λ-calculus expression. We define the translation as follows.

$$\mathcal{T}[\![x]\!] = x$$
$$\mathcal{T}[\![\lambda x_1, \ldots, x_n.\, e]\!] = \lambda x_1.\ \ldots\ \lambda x_n.\, \mathcal{T}[\![e]\!]$$
$$\mathcal{T}[\![e_0\ e_1\ e_2\ \ldots\ e_n]\!] = (\ldots((\mathcal{T}[\![e_0]\!]\ \mathcal{T}[\![e_1]\!])\ \mathcal{T}[\![e_2]\!])\ \ldots\ \mathcal{T}[\![e_n]\!])$$

This process of rewriting a function that takes multiple arguments as a chain of functions that each take a single argument is called *currying*. Consider a mathematical function that takes two arguments, the first from domain A and the second from domain B, and returns a result from domain C. We could describe this function, using mathematical notation for domains of functions, as being an element of $A \times B \to C$. Currying this function produces a function that is an element of $A \to (B \to C)$. That is, the curried version of the function takes an argument from domain A, and returns a function that takes an argument from domain B and produces a result of domain C.

5.11 Products and let

A *product* is a pair of expressions (e_1, e_2). If e_1 and e_2 are both values, then we regard the product as also being a value. (That is, we cannot further evaluate a product if both elements are values.) Given a product, we can access the first or second element using the operators #1 and #2 respectively. That is, $\#1\ (v_1, v_2) \to v_1$ and $\#2\ (v_1, v_2) \to v_2$. (Other common notation for projection includes π_1 and π_2, and fst and snd.)
The syntax of λ-calculus extended with products and let expressions is defined as follows.

$$\begin{aligned} e &::= x \mid \lambda x.\, e \mid e_1\ e_2 \\ &\quad \mid (e_1, e_2) \mid \#1\ e \mid \#2\ e \\ &\quad \mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 \\ v &::= \lambda x.\, e \mid (v_1, v_2) \end{aligned}$$

Note that values in this language are either functions or pairs of values.
We define a small-step CBV operational semantics for the language using evaluation contexts.

$$E \ ::= \ [\cdot] \mid E\ e \mid v\ E \mid (E, e) \mid (v, E) \mid \#1\ E \mid \#2\ E \mid \textbf{let}\ x = E\ \textbf{in}\ e_2$$

$$\frac{e \to e'}{E[e] \to E[e']} \qquad \frac{}{(\lambda x.\, e)\ v \to e\{v/x\}}\ \beta\text{-Reduction}$$

$$\#1\ (v_1, v_2) \to v_1 \qquad \#2\ (v_1, v_2) \to v_2 \qquad \textbf{let}\ x = v\ \textbf{in}\ e \to e\{v/x\}$$

Next, we define an equivalent semantics by translation to the pure CBV λ-calculus.

$$\begin{aligned} \mathcal{T}[\![x]\!] &= x \\ \mathcal{T}[\![\lambda x.\, e]\!] &= \lambda x.\, \mathcal{T}[\![e]\!] \\ \mathcal{T}[\![e_1\ e_2]\!] &= \mathcal{T}[\![e_1]\!]\ \mathcal{T}[\![e_2]\!] \\ \mathcal{T}[\![(e_1, e_2)]\!] &= (\lambda x.\, \lambda y.\, \lambda f.\, f\ x\ y)\ \mathcal{T}[\![e_1]\!]\ \mathcal{T}[\![e_2]\!] \\ \mathcal{T}[\![\#1\ e]\!] &= \mathcal{T}[\![e]\!]\ (\lambda x.\, \lambda y.\, x) \\ \mathcal{T}[\![\#2\ e]\!] &= \mathcal{T}[\![e]\!]\ (\lambda x.\, \lambda y.\, y) \\ \mathcal{T}[\![\textbf{let}\ x = e_1\ \textbf{in}\ e_2]\!] &= (\lambda x.\, \mathcal{T}[\![e_2]\!])\ \mathcal{T}[\![e_1]\!] \end{aligned}$$

Note that we encode a pair (e_1, e_2) as a value that takes a function f, and applies f to v_1 and v_2, where v_1 and v_2 are the result of evaluating e_1 and e_2 respectively. The projection operators pass a function to the encoding of pairs that selects either the first or second element as appropriate. Also note that the expression $\textbf{let}\ x = e_1\ \textbf{in}\ e_2$ is equivalent to the application $(\lambda x.\, e_2)\ e_1$.

5.12 Laziness

In previous lectures we defined semantics for both the call-by-name λ-calculus and the call-by-value λ-calculus. It turns out that we can translate a call-by-name program into a call-by-value program. In CBV, arguments to functions are evaluated before the function is applied; in CBN, functions are applied as soon as possible. In the translation, we delay the evaluation of arguments by wrapping them in a function. This is called a *thunk*: wrapping a computation in a function to delay its evaluation.
Since arguments to functions are turned into thunks, when we want to use an argument in a function body, we need to evaluate the thunk. We do so by applying the thunk (which is simply a function); it doesn't matter what we apply the thunk to, since the thunk's argument is never used.

$$\begin{aligned} \mathcal{T}[\![x]\!] &= x\ (\lambda y.\, y) \\ \mathcal{T}[\![\lambda x.\, e]\!] &= \lambda x.\, \mathcal{T}[\![e]\!] \\ \mathcal{T}[\![e_1\ e_2]\!] &= \mathcal{T}[\![e_1]\!]\ (\lambda z.\, \mathcal{T}[\![e_2]\!]) \qquad\qquad z \text{ is not a free variable of } e_2 \end{aligned}$$

5.13 Adequacy of translation

In each of the previous translations, we defined a semantics for the source language (using evaluation contexts and small-step rules) and the target language (by translation). We would like to be able to show that the translation is correct—that is, that it preserves the meaning of source programs.

More precisely, we would like an expression e in the source language to evaluate to a value v if and only if the translation of e evaluates to a value v' such that v' is "equivalent to" v. What exactly it means for v' to be "equivalent to" v will depend on the translation. Sometimes, it will mean that v' is literally the translation of v; other times, it will mean that v' is merely related to the translation of v by some equivalence.

One tricky issue is that in general, there can be many ways to define equivalences on functions. One way is to say that two functions are equivalent if they agree on the result when applied to any value of a base type (e.g., integers or booleans). The idea is that if two functions disagree when passed a more complex value (say, a function), then we could write a program that uses these functions to produce functions that disagree on values of base types.

There are two criteria for a translation to be adequate: soundness and completeness. For clarity, let's suppose that Exp_{src} is the set of source language expressions, and that $\rightarrow_{\text{src}}$ and $\rightarrow_{\text{trg}}$ are the evaluation relations for the source and target languages respectively. A translation is *sound* if every target evaluation represents a source evaluation:

$$\textbf{Soundness:} \quad \forall e \in \text{Exp}_{\text{src}}.\ \text{if } \mathcal{T}[\![e]\!] \rightarrow^*_{\text{trg}} v' \text{ then } \exists v.\ e \rightarrow^*_{\text{src}} v \text{ and } v' \text{ equivalent to } v$$

A translation is *complete* if every source evaluation has a target evaluation.

$$\textbf{Completeness:} \quad \forall e \in \text{Exp}_{\text{src}}.\ \text{if } e \rightarrow^*_{\text{src}} v \text{ then } \exists v'.\ \mathcal{T}[\![e]\!] \rightarrow^*_{\text{trg}} v' \text{ and } v' \text{ equivalent to } v$$

Chapter 6

Type Systems

A *type* is a collection of computational entities that share some common property. For example, the type **int** represents all expressions that evaluate to an integer, the type **bool** represents all expressions that evaluate to a Boolean, the type $\textbf{int} \to \textbf{int}$ represents all functions from integers to integers, and the type $\textbf{int} \times \textbf{bool}$ represents all pairs of an integer and a Boolean.

You can see types as a *static* approximation of the *dynamic* behaviors of terms and programs. Type *systems* are a lightweight formal method for reasoning about behavior of a program. Uses of type systems include: naming and organizing useful concepts; providing information (to the compiler or programmer) about data manipulated by a program; and ensuring that the run-time behavior of programs meet certain criteria.

In this lecture, we'll consider a type system for the lambda calculus that ensures that values are used correctly; for example, that a program never tries to add an integer to a function. The resulting language (lambda calculus plus the type system) is called the *simply-typed lambda calculus* (STLC).

6.1 Simply-typed lambda calculus

The syntax of the simply-typed lambda calculus is similar to that of untyped lambda calculus, with the exception of abstractions. Since abstractions define functions that take an argument, in the simply-typed lambda calculus, we explicitly state what the type of the argument is. That is, in an abstraction $\lambda x : \tau.\, e$, the τ is the expected type of the argument.

The syntax of the simply-typed lambda calculus is as follows. It includes integer literals n, arithmetic $e_1 \oplus e_2$ (where the operator $\oplus$ is addition, subtraction, or multiplication), and the *unit value* $()$. The unit value is the only value of type **unit**.

$$\begin{array}{rrcl} \text{expressions} & e & ::= & x \mid \lambda x : \tau.\, e \mid e_1\ e_2 \mid n \mid e_1 \oplus e_2 \mid () \\ \text{operators} & \oplus & ::= & + \mid - \mid \times \\ \text{values} & v & ::= & \lambda x : \tau.\, e \mid n \mid () \\ \text{types} & \tau & ::= & \textbf{int} \mid \textbf{unit} \mid \tau_1 \to \tau_2 \end{array}$$

The operational semantics of the simply-typed lambda calculus are the same as the untyped lambda calculus. For completeness, we present the CBV small step operational semantics here.

$$E ::= [\cdot] \mid E\ e \mid v\ E \mid E \oplus e \mid v \oplus E \qquad \frac{e \to e'}{E[e] \to E[e']}\ \textsc{Context}$$

$$\frac{}{(\lambda x : \tau.\, e)\ v \to e\{v/x\}}\ \beta\text{-reduction} \qquad \frac{n = n_1 \oplus n_2}{n_1 \oplus n_2 \to n}\ \textsc{Arith}$$

6.1.1 The typing relation

The presence of types does not alter the evaluation of an expression at all. So what use are types? We will use types to restrict what expressions we will evaluate. Specifically, the type system for the simply-typed lambda calculus will ensure that any well-typed program will not get stuck. A term e is *stuck* if e is not a value and there is no term e' such that $e \to e'$. For example, the expression $42 + \lambda x.\, x$ is stuck: it attempts to add an integer and

a function; it is not a value, and there is no operational rule that allows us to reduce this expression. Another stuck expression is () 47, which attempts to apply the unit value to an integer.

We introduce a relation (or judgment) over typing contexts (or type environments) Γ, expressions e, and types τ. The judgment

$$\Gamma \vdash e : \tau$$

is read as "e has type τ in context Γ".

A typing context is a sequence of variables and their types. In the typing judgment $\Gamma \vdash e : \tau$, we will ensure that if x is a free variable of e, then Γ associates x with a type. We can view a typing context as a partial function from variables to types. We will write $\Gamma, x : \tau$ or $\Gamma[x \mapsto \tau]$ to indicate the typing context that extends Γ by associating variable x with type τ. The empty context is sometimes written $\emptyset$, or often just not written at all. For example, we write $\vdash e : \tau$ to mean that the closed term e has type τ under the empty context.

Given a typing environment Γ and expression e, if there is some τ such that $\Gamma \vdash e : \tau$, we say that e is *well-typed* under context Γ; if Γ is the empty context, we say e is well-typed.

We define the judgment $\Gamma \vdash e : \tau$ inductively.

$$\frac{}{\Gamma \vdash n : \textbf{int}}\ \text{T-Int} \qquad \frac{\Gamma \vdash e_1 : \textbf{int} \qquad \Gamma \vdash e_2 : \textbf{int}}{\Gamma \vdash e_1 \oplus e_2 : \textbf{int}}\ \text{T-Arith} \qquad \frac{}{\Gamma \vdash () : \textbf{unit}}\ \text{T-Unit}$$

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}\ \text{T-Var} \qquad \frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x{:}\tau.\, e : \tau \to \tau'}\ \text{T-Abs} \qquad \frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1\ e_2 : \tau'}\ \text{T-App}$$

An integer n always has type **int**. Expression $e_1 \oplus e_2$ has type **int** if both e_1 and e_2 have type **int**. The unit value () always has type **unit**.

Variable x has whatever type the context associates with x. Note that Γ must contain an association for x in order for the judgment $\Gamma \vdash x : \tau$ to hold, that is, $x \in \text{dom}(\Gamma)$. The abstraction $\lambda x : \tau.\, e$ has the function type $\tau \to \tau'$ if the function body e has type τ' under the assumption that x has type τ. Finally, an application $e_1\ e_2$ has type τ' provided that e_1 is a function of type $\tau \to \tau'$, and e_2 is an argument of the expected type, i.e., of type τ.

To type check an expression e, we attempt to construct a derivation of the judgment $\vdash e : \tau$, for some type τ. For example, consider the program $(\lambda x{:}\textbf{int}.\, x + 40)\ 2$. The following is a proof that $(\lambda x{:}\textbf{int}.\, x + 40)\ 2$ is well-typed.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{x : \textbf{int} \vdash x : \textbf{int}}\ \text{T-Var} \qquad \dfrac{}{x : \textbf{int} \vdash 40 : \textbf{int}}\ \text{T-Int}}{x : \textbf{int} \vdash x + 40 : \textbf{int}}\ \text{T-Arith}}{\vdash \lambda x{:}\textbf{int}.\, x + 40 : \textbf{int} \to \textbf{int}}\ \text{T-Abs} \qquad \dfrac{}{\vdash 2 : \textbf{int}}\ \text{T-Int}}{\vdash (\lambda x{:}\textbf{int}.\, x + 40)\ 2 : \textbf{int}}\ \text{T-App}$$

6.1.2 Type soundness

We mentioned above that the type system ensures that any well-typed program does not get stuck. We can state this property formally.

> **Theorem 6.1.1** Type soundness
>
> If $\vdash e : \tau$ and $e \to^* e'$ and $e' \not\to$ then e' is a value and $\vdash e' : \tau$.

We will prove this theorem using two lemmas: preservation and progress. Intuitively, preservation says that if an expression e is well-typed, and e can take a step to e', then e' is well-typed. That is, evaluation preserves well-typedness. Progress says that if an expression e is well-typed, then either e is a value, or there is an e' such that e can take a step to e'. That is, well-typedness means that the expression cannot get stuck.

Together, these two lemmas suffice to prove type soundness. Given the preservation lemma, a trivial induction on the number of steps taken to reach e' from e establishes that $\vdash e' : \tau$. Then the progress lemma ensures that, if e' cannot take a step, then it must be a value.

6.1.3 Preservation

To prove preservation, we will need some extra tiny lemmas.

Lenma 6.1.1 Substitution

If $\Gamma, x : \tau' \vdash e : \tau$ and $\vdash v : \tau'$ then $\Gamma \vdash e\{v/x\} : \tau$.

Lenma 6.1.2 Context

If $\vdash E[e] : \tau$, where the derivation of this judgment gives e the type τ', and $\vdash e' : \tau'$, then $\vdash E[e'] : \tau$.

We'll assume these without proof. (They're not difficult, but the proof of substitution can get rather long. It is by induction on the typing derivation, which is why the lemma is stated for an arbitrary context Γ: the case for $\lambda y : \tau_1 . e_1$ needs the induction hypothesis for the body e_1, in the larger context $\Gamma, y : \tau_1$. Preservation only uses the case where Γ is empty.) Equipped with these little lemmas, we're ready to move on to the main proof of preservation.

A quick note on proof strategy: to prove preservation, it's possible to induct either on the typing relation or on the small-step relation. Both have their advantages and disadvantages; we'll use the small-step relation here.

Lenma 6.1.3 Preservation

If $\vdash e : \tau$ and $e \longrightarrow e'$ then $\vdash e' : \tau$.

Proof. Assume $\vdash e : \tau$ and $e \longrightarrow e'$. We need to show $\vdash e' : \tau$. We will do this by induction on the derivation of $e \longrightarrow e'$.

- ARITH
  Here, $e \equiv n_1 \oplus n_2$, and $e' = n$ where $n = n_1 \oplus n_2$.

  There is only one typing rule that applies to arithmetic expressions, T-ARITH, from which we know $\tau =$ **int**.

  By the typing rule T-INT, we have $\vdash e' :$ **int** as required.

- β-REDUCTION
  Here, $e \equiv (\lambda x : \tau' . e_1)\ v$ and $e' \equiv e_1\{v/x\}$.

  Since e is well-typed by assumption, we have derivations showing $\vdash \lambda x : \tau' . e_1 : \tau' \longrightarrow \tau$ and $\vdash v : \tau'$. There is only one typing rule for abstractions, T-ABS, from which we know $x : \tau' \vdash e_1 : \tau$.

  By our substitution lemma above, we have $\vdash e_1\{v/x\} : \tau$ as required.

- CONTEXT
  Here, we have some context E such that $e = E[e_1]$ and $e' = E[e_2]$ for some e_1 and e_2 such that $e_1 \longrightarrow e_2$.

  Since e is well-typed, we can show by induction on the structure of E that $\vdash e_1 : \tau_1$ for some τ_1. (This simple sub-induction is left as an exercise.)

  By the induction hypothesis and because we know $e_1 \longrightarrow e_2$, we have $\vdash e_2 : \tau_1$. (Put intuitively, e_2 has the same type as the one we just established for e_1.)

  By our context lemma above, we have $\vdash E[e_2] : \tau$ as required.



6.1.4 Progress

To prove our progress lemma, we'll need one extra lemma that gives us the syntax forms for closed terms.

Lenma 6.1.4 Canonical Forms

If $\vdash v : \tau$, then

1. If τ is **int**, then v is an integer literal, i.e., some n.
2. If τ is $\tau_1 \longrightarrow \tau_2$, then v is an abstraction, i.e., $\lambda x : \tau_1 . e$ for some x and e.

Proof. The proof is by inspection of the typing rules.

i If τ is **int**, then the only rule which lets us give a value this type is T-Int.

ii If τ is $\tau_1 \to \tau_2$, then the only rule which lets us give a value this type is T-Abs.

Now we're ready to prove progress.

Lenma 6.1.5 Progress

If $\vdash e : \tau$ then either e is a value or there exists an e' such that $e \to e'$.

Proof. We proceed by induction on the derivation of $\vdash e : \tau$.

- T-Var
  This case is impossible, since a variable is not well-typed in the empty environment.
- T-Unit, T-Int, T-Abs
  In all of these cases, e is a value.
- T-Arith
  Here $e \equiv e_1 \oplus e_2$ and $\vdash e_1 :$ **int** and $\vdash e_2 :$ **int**. By the inductive hypothesis, for $i \in \{1, 2\}$ (i.e., for both e_1 and e_2), either e_i is a value or there is an e_i' such that $e_i \to e_i'$.

  If e_1 is not a value, we have from above that $e_1 \to e_1'$. Therefore, by the Context rule, $e_1 \oplus e_2 \to e_1' \oplus e_2$.

  Otherwise, e_1 is a value. If e_2 is not a value, then by Context again, $e_1 \oplus e_2 \to e_1 \oplus e_2'$.

  Otherwise, both e_1 and e_2 are values. By our canonical forms lemma, $e_1 = n_1$ and $e_2 = n_2$ are both integer literals. By the Arith rule, we have $e_1 \oplus e_2 \to n$ where $n = n_1 \oplus n_2$.
- T-App
  Here $e \equiv e_1\ e_2$ and $\vdash e_1 : \tau' \to \tau$ and $\vdash e_2 : \tau'$. By the inductive hypothesis, for $i \in \{1, 2\}$, either e_i is a value or there is an e_i' such that $e_i \to e_i'$.

  If e_1 is not a value, then by the above and by applying the Context rule, $e_1\ e_2 \to e_1'\ e_2$.

  Otherwise, e_1 is a value. If e_2 is not a value, then by Context, $e_1\ e_2 \to e_1\ e_2'$.

  If e_1 and e_2 are values, then, by our canonical forms lemma, e_1 is an abstraction $\lambda x : \tau'.\, e'$. Therefore, by β-reduction, we have $e_1\ e_2 \to e'\{e_2/x\}$.

6.1.5 Uniqueness of Types

Our typing rules have another useful property: every well-typed expression has exactly one type.

Lenma 6.1.6 Uniqueness of types

If $\Gamma \vdash e : \tau$ and $\Gamma \vdash e : \tau'$ then $\tau = \tau'$.

Proof sketch. By induction on e. The typing rules are syntax-directed: for each form of expression, exactly one typing rule can be used to conclude its type. For example, if $e \equiv \lambda x : \tau_1.\, e_1$, then both derivations must end with T-Abs, so $\tau = \tau_1 \to \tau_2$ and $\tau' = \tau_1 \to \tau_2'$, where $\Gamma, x : \tau_1 \vdash e_1 : \tau_2$ and $\Gamma, x : \tau_1 \vdash e_1 : \tau_2'$. By the induction hypothesis, $\tau_2 = \tau_2'$. Note that the annotation τ_1 on the abstraction is essential here. The other cases are similar.

Because the rules are syntax-directed, they also give a simple algorithm for type checking: a recursive function over the syntax of e that computes the unique type of e, or reports that e is not well-typed.

6.1.6 Type Completeness?

Not all expressions in the untyped lambda calculus are well-typed. Type soundness implies that any lambda calculus program that gets stuck is not well-typed.
But are there programs that do not get stuck that are not well-typed? In other words, does our type system unjustly rule out legal programs?
Unfortunately, the answer is yes. In particular, because the simply-typed lambda calculus requires us to specify a type for function arguments, any given function can only take arguments of one type. Consider, for example, the identity function $\lambda x.\, x$. This function may be applied to any argument, and it will not get stuck. However, we must provide a type for the argument. If we specify $\lambda x : \mathbf{int}.\, x$, then this function can only accept integers, and the program $(\lambda x : \mathbf{int}.\, x)\ ()$ is not well-typed, even though it does not get stuck. Indeed, in the simply-typed lambda calculus, there is a different identity function for each type.

6.1.7 Termination

The type system also rules out some programs that do not get stuck but instead run forever. Consider $\Omega \triangleq (\lambda x.\, x\ x)\ (\lambda x.\, x\ x)$. To write Ω in the simply-typed lambda calculus, we would need a type τ such that $\lambda x : \tau.\, x\ x$ is well-typed. By T-App, the application $x\ x$ requires x to have a function type $\tau_1 \to \tau_2$, and also to have the argument type τ_1. By T-Var, both of these are τ, so

$$\tau_1 = \tau = \tau_1 \to \tau_2.$$

But no type satisfies this equation, since τ_1 would have to be a proper part of itself, and types are finite. So Ω is not well-typed, and neither are the fixed-point combinators Y and Z, which use self-application in the same way. In fact, every well-typed program terminates.

> **Theorem 6.1.2** Strong normalization
>
> If $\vdash e : \tau$, then every sequence of reductions starting from e is finite.

The proof uses a technique called *logical relations*, due to Tait, which we will not cover here. One consequence is that the simply-typed lambda calculus is not Turing complete: since every well-typed program terminates, it cannot express all computable functions, and in particular it cannot express a function that runs forever on some inputs. This is another way in which the type system is incomplete. To recover general recursion, we will add a fixed-point operator to the language as a primitive below.

6.2 Advanced Types

In this lecture we will extend the simply-typed λ-calculus with several features we saw earlier in the course, including products and sums, as well as some new ones.

6.2.1 Booleans and Conditionals

We add the Boolean constants true and false, conditionals, and a comparison operator on integers.

$$\begin{array}{lcl} e & ::= & \cdots \mid \mathbf{true} \mid \mathbf{false} \mid \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 \mid e_1 = e_2 \\ v & ::= & \cdots \mid \mathbf{true} \mid \mathbf{false} \end{array}$$

The evaluation contexts evaluate the guard of a conditional, but neither branch: only the branch that is selected is evaluated.

$$E ::= \cdots \mid \mathbf{if}\ E\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 \mid E = e \mid v = E$$

$$\mathbf{if\ true\ then}\ e_2\ \mathbf{else}\ e_3 \to e_2 \qquad \mathbf{if\ false\ then}\ e_2\ \mathbf{else}\ e_3 \to e_3$$

$$\frac{n_1 = n_2}{n_1 = n_2 \to \mathbf{true}} \qquad \frac{n_1 \neq n_2}{n_1 = n_2 \to \mathbf{false}}$$

We add a new type **bool** of Booleans.

$$\tau ::= \cdots \mid \mathbf{bool}$$

The typing rules are as follows. Both branches of a conditional must have the same type, which is the type of the whole conditional.

$$\frac{}{\Gamma \vdash \textbf{true} : \textbf{bool}}\ \text{T-True} \qquad \frac{}{\Gamma \vdash \textbf{false} : \textbf{bool}}\ \text{T-False}$$

$$\frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 : \tau}\ \text{T-If}$$

$$\frac{\Gamma \vdash e_1 : \textbf{int} \qquad \Gamma \vdash e_2 : \textbf{int}}{\Gamma \vdash e_1 = e_2 : \textbf{bool}}\ \text{T-Eq}$$

6.2.2 What Goes Wrong?

It is instructive to see how the type soundness proof fails if we get the typing rules wrong. Suppose we changed T-If so that the type of a conditional is the type of its first branch, and the second branch may have any type:

$$\frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau'}{\Gamma \vdash \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 : \tau}\ \text{T-If?}$$

Then preservation fails: $\vdash$ **if false then** 1 **else** () : **int**, but it steps to (), which has type **unit**. As a result, soundness fails as well: (**if false then** 1 **else** ()) + 1 has type **int**, but it steps to () + 1, which is stuck.
Now suppose instead that we allowed the first argument of addition to be a Boolean:

$$\frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \textbf{int}}{\Gamma \vdash e_1 + e_2 : \textbf{int}}\ \text{T-Plus?}$$

Then progress fails: $\vdash$ **true** + 1 : **int**, but **true** + 1 is stuck, because no evaluation rule adds a Boolean to an integer. Mistakes in the operational semantics can break soundness too. For example, if we forgot the evaluation context $v \oplus E$, then $1 + (2 + 3)$ would be well-typed but stuck: 1 is already a value, so the context $E \oplus e$ does not apply, and there is no way to evaluate $2 + 3$. In general, the typing rules, the evaluation rules, and the proofs of progress and preservation must all fit together.

6.2.3 Products

We have previously seen how to encode products into untyped λ-calculus.

$$\begin{array}{rcl} e & ::= & \cdots \mid (e_1, e_2) \mid \#1\ e \mid \#2\ e \\ v & ::= & \cdots \mid (v_1, v_2) \end{array}$$

We defined congruence rules that determine the order of evaluation, using the following evaluation contexts.

$$E ::= \cdots \mid (E, e) \mid (v, E) \mid \#1\ E \mid \#2\ E$$

We also defined two computation rules that determine how the pairing constructor and destructors interact.

$$\#1\ (v_1, v_2) \to v_1 \qquad \#2\ (v_1, v_2) \to v_2$$

In simply-typed λ-calculus, the type of a product expression (or a product type) is a pair of types, written $\tau_1 \times \tau_2$. The typing rules for the product constructors and destructors are as follows:

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2} \qquad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \#1\ e : \tau_1} \qquad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \#2\ e : \tau_2}$$

Note the similarities between these rules and the proof rules for conjunction in natural deduction. We will examine this relationship closely later in the course.

6.2.4 Derived Forms

Some useful constructs can be defined in terms of the ones we already have. These *derived forms* (or *syntactic sugar*) need no new evaluation or typing rules. For example, we can define local definitions and sequencing:

$$\begin{aligned}\textbf{let } x{:}\tau = e_1 \textbf{ in } e_2 &\triangleq (\lambda x{:}\tau.\, e_2)\; e_1 \\ e_1; e_2 &\triangleq (\lambda x{:}\textbf{unit}.\, e_2)\; e_1 \quad \text{where } x \notin fv(e_2)\end{aligned}$$

Under CBV, $\textbf{let } x{:}\tau = e_1 \textbf{ in } e_2$ evaluates e_1 to a value and then substitutes it for x in e_2, and $e_1; e_2$ evaluates e_1, discards its value, and then evaluates e_2. Their typing rules can be derived from T-Abs and T-App:

$$\frac{\Gamma \vdash e_1 : \tau \qquad \Gamma, x : \tau \vdash e_2 : \tau'}{\Gamma \vdash \textbf{let } x{:}\tau = e_1 \textbf{ in } e_2 : \tau'} \qquad \frac{\Gamma \vdash e_1 : \textbf{unit} \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1; e_2 : \tau}$$

For example, the first rule follows by applying T-Abs to the second premise, which gives $\Gamma \vdash \lambda x{:}\tau.\, e_2 : \tau \to \tau'$, and then T-App with the first premise. The second rule is similar, using the fact that adding $x : \textbf{unit}$ to Γ does not change the type of e_2, since x is not free in e_2. We often omit the annotation on **let**, since τ must be the type of e_1.

6.2.5 Sums

Our next example, sums, is dual to products. Intuitively, a product holds two values, one of type τ_1, and one of type τ_2, while a sum holds a single value that is either of type τ_1 or of type τ_2. The type of a sum is written $\tau_1 + \tau_2$. There are two constructors for sums, corresponding to whether we are constructing a sum with a value of τ_1 or a value of τ_2.

$$\begin{aligned} e &::= \cdots \mid \mathsf{inl}_{\tau_1+\tau_2}\, e \mid \mathsf{inr}_{\tau_1+\tau_2}\, e \mid \textbf{case } e_1 \textbf{ of } e_2 \mid e_3 \\ v &::= \cdots \mid \mathsf{inl}_{\tau_1+\tau_2}\, v \mid \mathsf{inr}_{\tau_1+\tau_2}\, v \end{aligned}$$

There are congruence rules that determine the order of evaluation, as defined by the following evaluation contexts.

$$E ::= \cdots \mid \mathsf{inl}_{\tau_1+\tau_2}\, E \mid \mathsf{inr}_{\tau_1+\tau_2}\, E \mid \textbf{case } E \textbf{ of } e_2 \mid e_3$$

There are also two computation rules that show how the constructors and destructors interact.

$$\textbf{case } \mathsf{inl}_{\tau_1+\tau_2}\, v \textbf{ of } e_2 \mid e_3 \longrightarrow e_2\; v \qquad \textbf{case } \mathsf{inr}_{\tau_1+\tau_2}\, v \textbf{ of } e_2 \mid e_3 \longrightarrow e_3\; v$$

The type of a sum expression (or a sum type) is written $\tau_1 + \tau_2$. The typing rules for the sum constructors and destructor are the following.

$$\frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \mathsf{inl}_{\tau_1+\tau_2}\, e : \tau_1 + \tau_2} \qquad \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \mathsf{inr}_{\tau_1+\tau_2}\, e : \tau_1 + \tau_2}$$

$$\frac{\Gamma \vdash e : \tau_1 + \tau_2 \qquad \Gamma \vdash e_1 : \tau_1 \to \tau \qquad \Gamma \vdash e_2 : \tau_2 \to \tau}{\Gamma \vdash \textbf{case } e \textbf{ of } e_1 \mid e_2 : \tau}$$

Let's see an example of a program that uses sum types.

$$\begin{aligned}&\textbf{let } f = \lambda a{:}\textbf{int} + (\textbf{int} \to \textbf{int}).\, \textbf{case } a \textbf{ of } (\lambda y{:}\textbf{int}.\, y + 1) \mid (\lambda g{:}\textbf{int} \to \textbf{int}.\, g\; 35) \textbf{ in} \\ &\textbf{let } h = \lambda x{:}\textbf{int}.\, x + 7 \textbf{ in} \\ &f\; (\mathsf{inr}_{\textbf{int}+(\textbf{int}\to\textbf{int})}\, h)\end{aligned}$$

The function f takes argument a, which is a sum—that is, the actual argument for a will either be a value of type **int** or a value of type $\textbf{int} \to \textbf{int}$. We destruct the sum value with a case statement, which must be prepared to take either of the two kinds of values that the sum may contain. In this instance, we end up applying f to a value of type $\textbf{int} \to \textbf{int}$ (i.e., a value injected into the right type of the sum), so the entire program ends up evaluating to 42.

6.2.6 Extending the Proofs

When we extend the language, we must also extend the proof of type soundness. Most of the work is in the canonical forms lemma, which gains a case for each new type of values.

Lenma 6.2.1 Canonical Forms, continued

If $\vdash v : \tau$, then

1. If τ is **bool**, then v is **true** or **false**.
2. If τ is $\tau_1 \times \tau_2$, then v is a pair (v_1, v_2) with $\vdash v_1 : \tau_1$ and $\vdash v_2 : \tau_2$.
3. If τ is $\tau_1 + \tau_2$, then v is $\mathsf{inl}_{\tau_1+\tau_2}\, v'$ or $\mathsf{inr}_{\tau_1+\tau_2}\, v'$.

Each new typing rule then adds a case to the proofs of preservation and progress. For example, here is the case of the progress proof for conditionals.

- T-If
  Here $e \equiv$ **if** e_1 **then** e_2 **else** e_3 and $\vdash e_1 :$ **bool**. By the inductive hypothesis, either e_1 is a value or there is an e_1' such that $e_1 \longrightarrow e_1'$.
  If $e_1 \longrightarrow e_1'$, then by the Context rule with the context **if** $[\cdot]$ **then** e_2 **else** e_3, we have $e \longrightarrow$ **if** e_1' **then** e_2 **else** e_3.
  Otherwise, e_1 is a value, so by canonical forms it is **true** or **false**, and e steps to e_2 or e_3 by one of the evaluation rules for conditionals.

Note that we did not need the inductive hypothesis for e_2 or e_3, because neither branch is evaluated before the guard. The corresponding cases of preservation follow directly from T-If: if $e \longrightarrow e_2$, then $\vdash e_2 : \tau$ is one of its premises, and similarly for e_3.

6.2.7 Fixed Points

As we saw above, fixed-point combinators such as Y and Z are not well-typed, and every well-typed program terminates. To obtain fixed points in the simply-typed lambda calculus, we can instead add a new primitive **fix** to the language. The evaluation rules for the new primitive mimic the behavior of the fixed-point combinators we saw previously.
We extend the syntax with the new primitive operator. Intuitively, $\mathsf{fix}\, e$ is the fixed-point of the function e. Note that $\mathsf{fix}\, v$ is not a value.

$$e ::= \cdots \mid \mathsf{fix}\, e$$

We extend the operational semantics for the new operator. There is a new evaluation context, and a new axiom.

$$E ::= \cdots \mid \mathsf{fix}\, E \qquad\qquad \mathsf{fix}\, \lambda x{:}\tau.\, e \longrightarrow e\{(\mathsf{fix}\, \lambda x{:}\tau.\, e)/x\}$$

Note that we can define the **letrec** $x{:}\tau = e_1$ **in** e_2 construct in terms of the **fix** operator.

$$\textbf{letrec}\ x{:}\tau = e_1\ \textbf{in}\ e_2 \triangleq \textbf{let}\ x = \mathsf{fix}\, \lambda x{:}\tau.\, e_1\ \textbf{in}\ e_2$$

The typing rule for **fix** says that $\mathsf{fix}\, e$ is a fixed point of a function e from τ to τ, so it has type τ:

$$\frac{\Gamma \vdash e : \tau \longrightarrow \tau}{\Gamma \vdash \mathsf{fix}\, e : \tau}\ \text{T-Fix}$$

Returning to our trusty factorial example, the following program implements the factorial function using the **fix** operator.

$$\text{FACT} \triangleq \mathsf{fix}\, \lambda f{:}\textbf{int} \longrightarrow \textbf{int}.\, \lambda n{:}\textbf{int}.\, \textbf{if}\ n = 0\ \textbf{then}\ 1\ \textbf{else}\ n \times (f\ (n-1))$$

Note that we can write non-terminating computations for any type: the expression $\mathsf{fix}\, \lambda x{:}\tau.\, x$ has type τ, and does not terminate.
Although the **fix** operator is normally used to define recursive functions, it can be used to find fixed points of any type. For example, consider the following expression.

$$\mathsf{fix}\, \lambda x{:}(\textbf{int} \longrightarrow \textbf{bool}) \times (\textbf{int} \longrightarrow \textbf{bool}).\, (\lambda n{:}\textbf{int}.\, \textbf{if}\ n = 0\ \textbf{then true else}\ (\#2\ x)\ (n-1),\ \lambda n{:}\textbf{int}.\, \textbf{if}\ n = 0\ \textbf{then false else}\ (\#1\ x)\ (n-1))$$

This expression defines a pair of mutually recursive functions; the first function returns true if and only if its argument is even; the second function returns true if and only if its argument is odd.

6.2.8 Exceptions

Many programming languages provide support for throwing and catching exceptions. We can model an extremely simple form of exceptions by extending the simply-typed λ-calculus with a single exception representing an error. We first extend the syntax of the language:

$$e ::= \cdots \mid \textbf{error} \mid \textbf{try}\ e_1\ \textbf{with}\ e_2$$

We do not add **try** expressions to our evaluation contexts—doing so would allow exceptions to "jump over" handlers. Instead, we add a special propagation rule for **try**:

$$\frac{e_1 \to e_1'}{\textbf{try}\ e_1\ \textbf{with}\ e_2 \to \textbf{try}\ e_1'\ \textbf{with}\ e_2}$$

and rules for propagating and catching exceptions. (The side condition $E \neq [\cdot]$ ensures that **error** by itself does not step.)

$$\frac{E \neq [\cdot]}{E[\textbf{error}] \to \textbf{error}} \qquad \textbf{try error with}\ e \to e \qquad \textbf{try}\ v\ \textbf{with}\ e \to v$$

The typing rule for exceptions allows them to take any type, while the typing rule for try-with expressions requires both sub-expressions to have the same type:

$$\frac{}{\Gamma \vdash \textbf{error} : \tau} \qquad \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \textbf{try}\ e_1\ \textbf{with}\ e_2 : \tau}$$

The first typing rule is extremely flexible, allowing errors to be thrown anywhere in a program. Note that it breaks the uniqueness of types: for example, both $\vdash$ **error** : **int** and $\vdash$ **error** : **bool** hold. However, it is not hard to see that it causes the progress lemma to become false: the expression **error** is not a value but is stuck. Fortunately, we can prove the following weaker version, which is still strong enough to prove a useful form of type soundness.

Lenma 6.2.2 Progress

If $\vdash e : \tau$ then e is a value or e is **error** or there exists e' such that $e \to e'$.

The preservation lemma remains unchanged.
The actual soundness theorem is as follows:

Theorem 6.2.1 Soundness

If $\vdash e : \tau$ and $e \to^* e'$ and $e' \not\to$ then either e' is a value or e' is **error**.

6.3 Reference Types

We extend the language with OCaml-style mutable references. The syntax and semantics are:

$$
\begin{aligned}
e &::= \cdots \mid \texttt{ref}\ e \mid !e \mid e_1 := e_2 \mid \ell \\
v &::= \cdots \mid \ell \\
E &::= \cdots \mid \texttt{ref}\ E \mid !E \mid E := e \mid v := E
\end{aligned}
$$

$$
\frac{\ell \notin \mathrm{dom}(\sigma)}{\langle \texttt{ref}\ v, \sigma\rangle \to \langle \ell, \sigma[\ell \mapsto v]\rangle}\ \textsc{Alloc} \qquad \frac{\sigma(\ell) = v}{\langle !\ell, \sigma\rangle \to \langle v, \sigma\rangle}\ \textsc{Deref}
$$

$$
\frac{\ell \in \mathrm{dom}(\sigma)}{\langle \ell := v, \sigma\rangle \to \langle (), \sigma[\ell \mapsto v]\rangle}\ \textsc{Assign}
$$

We add a type τ **ref** for locations holding values of type τ. For instance, $\texttt{ref}\ 7 : \textbf{int ref}$. Assignment is performed only for its effect, so it has type **unit**.

$$
\tau ::= \cdots \mid \tau\ \textbf{ref}
$$

$$
\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \texttt{ref}\ e : \tau\ \textbf{ref}} \qquad \frac{\Gamma \vdash e : \tau\ \textbf{ref}}{\Gamma \vdash !e : \tau} \qquad \frac{\Gamma \vdash e_1 : \tau\ \textbf{ref} \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 := e_2 : \textbf{unit}}
$$

Store typings

There is no rule yet for typing a location ℓ. Its type should be τ **ref** where τ is the type of the value stored at ℓ, but inspecting the store is inefficient and may be inconclusive (e.g. if $\sigma(\ell) = \ell$). Instead we use *store typings* Σ, which are partial functions from locations to types. The typing judgment now has four components and is written $\Gamma, \Sigma \vdash e : \tau$. The reference rules become (other rules are modified to carry Σ in the obvious way):

$$
\frac{\Gamma, \Sigma \vdash e : \tau}{\Gamma, \Sigma \vdash \texttt{ref}\ e : \tau\ \textbf{ref}} \qquad \frac{\Gamma, \Sigma \vdash e : \tau\ \textbf{ref}}{\Gamma, \Sigma \vdash !e : \tau}
$$

$$
\frac{\Gamma, \Sigma \vdash e_1 : \tau\ \textbf{ref} \qquad \Gamma, \Sigma \vdash e_2 : \tau}{\Gamma, \Sigma \vdash e_1 := e_2 : \textbf{unit}} \qquad \frac{\Sigma(\ell) = \tau}{\Gamma, \Sigma \vdash \ell : \tau\ \textbf{ref}}
$$

Type soundness

The earlier soundness statement must be adapted to account for the store and the store typing.

Definition. A store σ is *well-typed* with respect to Γ and Σ, written $\Gamma, \Sigma \vdash \sigma$, if $\mathrm{dom}(\sigma) = \mathrm{dom}(\Sigma)$ and for all $\ell \in \mathrm{dom}(\sigma)$ we have $\Gamma, \Sigma \vdash \sigma(\ell) : \Sigma(\ell)$.

> **Theorem 6.3.1** Type soundness
>
> If $\cdot, \Sigma \vdash e : \tau$ and $\cdot, \Sigma \vdash \sigma$ and $\langle e, \sigma\rangle \to^* \langle e', \sigma'\rangle$, then either e' is a value, or there exist e'' and σ'' such that $\langle e', \sigma'\rangle \to \langle e'', \sigma''\rangle$.

The proof uses progress and preservation as before. Progress adapts easily. Preservation is subtler: Alloc grows the store, so $\mathrm{dom}(\sigma') \neq \mathrm{dom}(\Sigma)$, and the store typing must be allowed to grow too.

> **Lenma 6.3.1** Preservation
>
> If $\Gamma, \Sigma \vdash e : \tau$ and $\Gamma, \Sigma \vdash \sigma$ and $\langle e, \sigma\rangle \to \langle e', \sigma'\rangle$, then there exists $\Sigma' \supseteq \Sigma$ such that $\Gamma, \Sigma' \vdash e' : \tau$ and $\Gamma, \Sigma' \vdash \sigma'$.

Here $\Sigma' \supseteq \Sigma$ means $\Sigma(\ell) = \Sigma'(\ell)$ for all $\ell \in \mathrm{dom}(\Sigma)$; viewing Σ as the set of pairs $\{(\ell, \tau) \mid \ell \in \mathrm{dom}(\Sigma) \wedge \Sigma(\ell) = \tau\}$, this is set inclusion. Intuitively, Σ' is either Σ or Σ extended with a newly allocated location.

Landin's knot

References recover recursion (and hence Turing completeness): create a cell holding a dummy function and "backpatch" it with the real definition.

```
let r = ref (fun x:int. 0) in
let f = fun x:int. if x = 0 then 1 else x * (!r) (x - 1) in
let a = (r := f) in
f 5
```

After the assignment, $!r$ is f itself, so $f\ 5$ evaluates to 120.

6.4 Recursive Types

Languages commonly allow recursive data types, e.g. in OCaml

```
type tree = Leaf | Node of int * tree * tree
```

We want *tree* to satisfy $\mathit{tree} = \mathbf{unit} + \mathbf{int} \times \mathit{tree} \times \mathit{tree}$, i.e. to be a solution of

$$\alpha = \mathbf{unit} + \mathbf{int} \times \alpha \times \alpha,$$

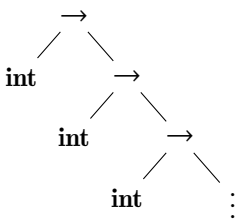
but no such type exists among the types seen so far. There are two standard approaches: *equirecursive* and *isorecursive*.

6.5 Equirecursive Types

Repeatedly unwinding the equation gives

$$\begin{aligned} \alpha &= \mathbf{unit} + \mathbf{int} \times \alpha \times \alpha \\ &= \mathbf{unit} + \mathbf{int} \times (\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha) \times (\mathbf{unit} + \mathbf{int} \times \alpha \times \alpha) \\ &= \cdots \end{aligned}$$

a sequence of ever deeper finite trees, each a substitution instance of the previous one. Taking the limit yields an infinite labeled tree whose nodes are labeled by the constructors $\times, +, \mathbf{int}, \mathbf{unit}$; this infinite tree is the solution, and it is what we take as the type *tree*.
Finite types over constructors such as $\to, \times, +, \mathbf{unit}, \mathbf{int}$ are finite labeled trees (e.g. $\mathbf{int} \to \mathbf{int} \to \mathbf{int}$). Infinite types are infinite labeled trees respecting arities: binary constructors have exactly two children, nullary ones are leaves.



A (finite or infinite) tree with only finitely many distinct subtrees (up to isomorphism) is called *regular*. The tree above is regular: its only subtrees are itself and **int**. The limit defining *tree* is also regular, with five subtrees: *tree*, **unit**, $\mathbf{int} \times \mathit{tree} \times \mathit{tree}$, $\mathit{tree} \times \mathit{tree}$, and **int**.
Regular trees suffice for finite systems of equations

$$\alpha_1 = \tau_1, \quad \ldots, \quad \alpha_n = \tau_n,$$

where each τ_i is a finite type over the constructors and $\alpha_1, \ldots, \alpha_n$. Such a system has a solution $\sigma_1, \ldots, \sigma_n$ of regular trees, and the solution is unique if no τ_i is a bare variable.

6.5.1 The μ constructor

The solution of $\alpha = \tau$ (with $\tau \neq \alpha$) is finite if $\alpha \notin FV(\tau)$ and infinite otherwise; we denote it $\mu\alpha.\,\tau$. The operator μ binds α just as λ binds variables in terms (same scope, free/bound variables, α-conversion, substitution). Since $\mu\alpha.\,\tau$ solves $\alpha = \tau$,

$$\mu\alpha.\,\tau = \tau\{\mu\alpha.\,\tau/\alpha\}.$$

For example,

$$\mathit{tree} \triangleq \mu\alpha.\ \mathbf{unit} + \mathbf{int} \times \alpha \times \alpha.$$

Solutions of mutually recursive systems are also expressible with μ. If neither τ_1 nor τ_2 is a variable, the system $\alpha_1 = \tau_1,\ \alpha_2 = \tau_2$ has the unique solution

$$\sigma_1 = \mu\alpha_1.\ \tau_1\{\mu\alpha_2.\,\tau_2/\alpha_2\} \qquad \sigma_2 = \mu\alpha_2.\ \tau_2\{\mu\alpha_1.\,\tau_1/\alpha_1\}.$$

Mutual recursion arises in practice, e.g. Java classes `Node` and `Edge` that refer to each other.

6.5.2 Typing rules

Since $\mu\alpha.\,\tau = \tau\{\mu\alpha.\,\tau/\alpha\}$, the rules are simple (equivalently: allow substituting equals for equals in types):

$$\frac{\Gamma \vdash e : \tau\{\mu\alpha.\,\tau/\alpha\}}{\Gamma \vdash e : \mu\alpha.\,\tau}\ \mu\text{-Intro} \qquad \frac{\Gamma \vdash e : \mu\alpha.\,\tau}{\Gamma \vdash e : \tau\{\mu\alpha.\,\tau/\alpha\}}\ \mu\text{-Elim}$$

6.6 Isorecursive Types

Here there are no infinite types: $\mu\alpha.\,\tau$ is itself a type, and it is *distinct but isomorphic* to $\tau\{\mu\alpha.\,\tau/\alpha\}$. Substituting $\mu\alpha.\,\tau$ for α is *unfolding*; the reverse is *folding*, and both are explicit operations:

$$\begin{aligned} \mathtt{unfold}_{\mu\alpha.\,\tau} &: \mu\alpha.\,\tau \to \tau\{\mu\alpha.\,\tau/\alpha\} \\ \mathtt{fold}_{\mu\alpha.\,\tau} &: \tau\{\mu\alpha.\,\tau/\alpha\} \to \mu\alpha.\,\tau \end{aligned}$$

Subscripts are often suppressed.

6.6.1 Typing rules

$$\frac{\Gamma \vdash e : \tau\{\mu\alpha.\,\tau/\alpha\}}{\Gamma \vdash \mathtt{fold}\ e : \mu\alpha.\,\tau}\ \mu\text{-Intro} \qquad \frac{\Gamma \vdash e : \mu\alpha.\,\tau}{\Gamma \vdash \mathtt{unfold}\ e : \tau\{\mu\alpha.\,\tau/\alpha\}}\ \mu\text{-Elim}$$

6.6.2 Operational semantics

Under CBV, `fold` v is a value, and arguments of `fold` and `unfold` are evaluated first:

$$\begin{aligned} v &::= \cdots \mid \mathtt{fold}\ v \\ E &::= \cdots \mid \mathtt{fold}\ E \mid \mathtt{unfold}\ E \end{aligned}$$

with the single new rule

$$\mathtt{unfold}\ (\mathtt{fold}\ v) \to v.$$

A value of type $\mu\alpha.\,\tau$ can only have been created by a `fold`, so unfolding it always succeeds.

6.6.3 Example

Let $\mathit{intlist} \triangleq \mu\alpha.\ \mathbf{unit} + \mathbf{int} \times \alpha$. To sum a list $l : \mathit{intlist}$ with a recursive function f, we must unfold l to expose the sum type before casing:

$$\mathtt{case}\ (\mathtt{unfold}\ l)\ \mathtt{of}\ (\lambda u : \mathbf{unit}.\ 0)\ \mid\ (\lambda p : \mathbf{int} \times \mathit{intlist}.\ (\#1\ p) + f\ (\#2\ p)).$$

This is the OCaml code with the folds and unfolds that OCaml hides made explicit.

6.7 Equirecursive vs. Isorecursive

Java and Modula-3 are equirecursive: folded and unfolded types are equal and `fold`/`unfold` act as the identity, so e.g. a class `E` with a field `E e` permits `e.e.e.e.x`. The ML family, CLU, and C are isorecursive: $\mu\alpha.\,\tau$ and $\tau\{\mu\alpha.\,\tau/\alpha\}$ are different types and explicit casts are needed (CLU calls them "up" and "down"). In OCaml, `match` and `let` perform `unfold` implicitly, and datatype constructors act as `fold`:

```
# type tree = Leaf | Node of int * tree * tree;;
# Node (4, Leaf, Leaf);;
- : tree = Node (4, Leaf, Leaf)
```

6.8 Numbers as a Recursive Type

bool can be encoded as **unit** + **unit** (with *true* and *false* the left and right injections). With recursive types, **int** need not be primitive either. Natural numbers are

$$\begin{aligned} \mathit{nat} &\triangleq \mu\alpha.\ \mathbf{unit} + \alpha \\ 0 &\triangleq \texttt{fold}\ (\texttt{inl}_{\mathbf{unit}+\mathit{nat}}\ ()) \\ 1 &\triangleq \texttt{fold}\ (\texttt{inr}_{\mathbf{unit}+\mathit{nat}}\ 0) \\ 2 &\triangleq \texttt{fold}\ (\texttt{inr}_{\mathbf{unit}+\mathit{nat}}\ 1), \end{aligned}$$

and so on. Arithmetic can be coded over nat, e.g. successor:

$$(\lambda x : \mathit{nat}.\ \texttt{fold}\ (\texttt{inr}_{\mathbf{unit}+\mathit{nat}}\ x)) : \mathit{nat} \to \mathit{nat}.$$

Thus **unit**, functions, products, sums, and recursive types suffice to build integers, lists, trees, floats, etc.

6.9 Self-Application and Ω

Recall $\omega \triangleq \lambda x.\, x\, x$ and $\Omega \triangleq \omega\, \omega$. In ω, x is applied as a function, so $x : \sigma \to \tau$, but it is also its own argument, so $\sigma = \sigma \to \tau$. This suggests $\mu\alpha.\,(\alpha \to \tau)$ for any τ. Unfolding gives

$$\texttt{unfold}\ x : (\mu\alpha.\,(\alpha \to \tau)) \to \tau,$$

so $(\texttt{unfold}\ x)\ x : \tau$ and

$$\omega \triangleq (\lambda x : \mu\alpha.\,(\alpha \to \tau).\ (\texttt{unfold}\ x)\ x) : (\mu\alpha.\,(\alpha \to \tau)) \to \tau.$$

Folding gives $\texttt{fold}\ \omega : \mu\alpha.\,(\alpha \to \tau)$, hence

$$\omega\ (\texttt{fold}\ \omega) : \tau,$$

which is Ω with explicit folds and unfolds. In OCaml:

```
# type u = Fold of (u -> u);;
# let omega = fun x -> match x with Fold f -> f x;;
val omega : u -> u = <fun>
# omega (Fold omega);;   (* runs forever *)
```

So recursive types alone, without `fix` or references, allow non-terminating yet well-typed programs.

6.10 Untyped to Typed λ-Calculus

With recursive types every pure untyped λ-term can be typed by

$$U \triangleq \mu\alpha.\ \alpha \to \alpha,$$

via the translation

$$\begin{aligned} [\![x]\!] &\triangleq x \\ [\![e_0\ e_1]\!] &\triangleq (\texttt{unfold}\ [\![e_0]\!])\ [\![e_1]\!] \\ [\![\lambda x.\, e]\!] &\triangleq \texttt{fold}\ (\lambda x : U.\ [\![e]\!]). \end{aligned}$$

Every untyped term maps to a term of type U.

Chapter 7

Polymorphism

7.1 Parametric Polymorphism

In the simply-typed λ-calculus every argument carries a type annotation, so each type needs its own identity function: $\lambda x : \mathbf{int}.\, x$, $\lambda x : \mathbf{bool}.\, x$, $\lambda x : \mathbf{int} \to \mathbf{int}.\, x$, and so on. *Polymorphism* is the ability to use code with values of different types. Three common kinds:

- *Subtype polymorphism*: a subsumption rule lets a value of type τ be used as type τ' whenever τ is a subtype of τ'.
- *Ad-hoc polymorphism*: code that looks polymorphic but is not (e.g. overloading); the argument types dispatch, at compile time or run time, to one of several separate implementations.
- *Parametric polymorphism*: code written without knowledge of the actual argument types (polymorphic functions in OCaml; generics in Java, Scala, Rust). This is the subject of this lecture.

7.2 System F

System F extends the simply-typed λ-calculus with abstraction over types. It was invented independently by Girard (1972, "System F") and Reynolds (1974, "polymorphic λ-calculus"). A *type abstraction* $\Lambda X.\, e$ is a function from types to terms; a *type application* $e\,[\tau]$ instantiates it at τ. For example, the polymorphic identity is

$$ID \triangleq \Lambda X.\ \lambda x : X.\ x.$$

7.2.1 Syntax and operational semantics

$$\begin{aligned} e &::= x \mid \lambda x : \tau.\, e \mid e_1\, e_2 \mid () \mid n \mid e_1 \oplus e_2 \mid \texttt{true} \mid \texttt{false} \mid \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \mid \Lambda X.\, e \mid e\,[\tau] \\ v &::= \lambda x : \tau.\, e \mid () \mid n \mid \texttt{true} \mid \texttt{false} \mid \Lambda X.\, e \\ \tau &::= \mathbf{unit} \mid \mathbf{int} \mid \mathbf{bool} \mid \tau_1 \to \tau_2 \mid X \mid \forall X.\, \tau \end{aligned}$$

Here $\oplus \in \{+, -, \times\}$. Both $\Lambda X.\, e$ and $\forall X.\, \tau$ bind X, giving the usual free/bound type variables and α-equivalence (e.g. $\forall X.\, X \to X$ and $\forall Y.\, Y \to Y$ are the same type). Evaluation is CBV as before, with one new context and one new rule:

$$E ::= \cdots \mid E\,[\tau] \qquad\qquad \frac{}{(\Lambda X.\, e)\,[\tau] \to e\{\tau/X\}}\ \textsc{Type-}\beta$$

Note $\Lambda X.\, e$ is a value: we do not evaluate under Λ (just as not under λ). For example,

$$ID\,[\mathbf{int}]\ 5 = (\Lambda X.\, \lambda x : X.\, x)\,[\mathbf{int}]\ 5 \to (\lambda x : \mathbf{int}.\, x)\ 5 \to 5,$$

and $ID\,[\mathbf{int} \to \mathbf{int}]$ steps to $\lambda x : \mathbf{int} \to \mathbf{int}.\, x$.

Type substitution must be capture-avoiding. E.g. in $\Lambda Y.\, (\Lambda X.\, \Lambda Y.\, \lambda x : X.\, \lambda y : Y.\, x)\,[Y]$ we must rename the inner bound variable:

$$(\forall Y.\, X \to Y \to X)\{Y/X\} = \forall Y'.\, Y \to Y' \to Y,$$

not $\forall Y.\, Y \to Y \to Y$, which would capture the outer Y.

7.2.2 Type system

$\Lambda X.\, e$ has type $\forall X.\, \tau$ where τ is the type of e (which may mention X). Typing now tracks, besides the term context Γ, a set Δ of type variables in scope, so judgments are $\Delta, \Gamma \vdash e : \tau$; e.g. $\lambda x : X.\, x$ is ill-typed on its own. An auxiliary judgment $\Delta \vdash \tau\ ok$ says every free type variable of τ is in Δ. The rules of the simply-typed fragment carry Δ unchanged, and annotations must be well-formed:

$$\frac{\Gamma(x) = \tau}{\Delta, \Gamma \vdash x : \tau}\ \text{T-Var} \qquad \frac{\Delta \vdash \tau\ ok \qquad \Delta, \Gamma, x : \tau \vdash e : \tau'}{\Delta, \Gamma \vdash \lambda x : \tau.\, e : \tau \to \tau'}\ \text{T-Abs}$$

$$\frac{\Delta, \Gamma \vdash e_1 : \tau \to \tau' \qquad \Delta, \Gamma \vdash e_2 : \tau}{\Delta, \Gamma \vdash e_1\, e_2 : \tau'}\ \text{T-App} \qquad \frac{\Delta, \Gamma \vdash e_1 : \mathbf{bool} \qquad \Delta, \Gamma \vdash e_2 : \tau \qquad \Delta, \Gamma \vdash e_3 : \tau}{\Delta, \Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau}\ \text{T-If}$$

Rules for (), integers, arithmetic, and Booleans are similar. The two new rules:

$$\frac{\Delta \cup \{X\}, \Gamma \vdash e : \tau}{\Delta, \Gamma \vdash \Lambda X.\, e : \forall X.\, \tau}\ \text{T-TAbs} \qquad \frac{\Delta, \Gamma \vdash e : \forall X.\, \tau' \qquad \Delta \vdash \tau\ ok}{\Delta, \Gamma \vdash e\,[\tau] : \tau'\{\tau/X\}}\ \text{T-TApp}$$

In T-TAbs we choose $X \notin \Delta$ by α-renaming. Well-formedness of types:

$$\frac{X \in \Delta}{\Delta \vdash X\ ok} \qquad \Delta \vdash \mathbf{unit}\ ok \qquad \Delta \vdash \mathbf{int}\ ok \qquad \Delta \vdash \mathbf{bool}\ ok \qquad \frac{\Delta \vdash \tau_1\ ok \qquad \Delta \vdash \tau_2\ ok}{\Delta \vdash \tau_1 \to \tau_2\ ok} \qquad \frac{\Delta \cup \{X\} \vdash \tau\ ok}{\Delta \vdash \forall X.\, \tau\ ok}$$

Derivation that the polymorphic identity has type $\forall X.\, X \to X$:

$$\dfrac{\dfrac{\dfrac{X \in \{X\}}{\{X\} \vdash X\ ok} \qquad \dfrac{(x : X)(x) = X}{\{X\}, x : X \vdash x : X}\ \text{T-Var}}{\{X\}, \cdot \vdash \lambda x : X.\, x : X \to X}\ \text{T-Abs}}{\emptyset, \cdot \vdash \Lambda X.\, \lambda x : X.\, x : \forall X.\, X \to X}\ \text{T-TAbs}$$

By T-TApp, $ID\,[\mathbf{int}]$ has type $(X \to X)\{\mathbf{int}/X\} = \mathbf{int} \to \mathbf{int}$.

7.3 Programming in System F

7.3.1 Self application

The term $\lambda x.\, x\, x$ is untypable in the simply-typed λ-calculus, but in System F we can give x a polymorphic type and instantiate it at that same type:

$$\vdash \lambda x : \forall X.\, X \to X.\ x\,[\forall X.\, X \to X]\ x\ :\ (\forall X.\, X \to X) \to (\forall X.\, X \to X).$$

Applied to ID it returns ID. We still cannot type Ω, since every well-typed System F program terminates (Section 7.5).

7.3.2 Church numerals

All Church numerals share one type:

$$NAT \triangleq \forall X.\, (X \to X) \to X \to X.$$

$$\begin{aligned} 0 &\triangleq \Lambda X.\ \lambda f : X \to X.\ \lambda x : X.\ x \\ 1 &\triangleq \Lambda X.\ \lambda f : X \to X.\ \lambda x : X.\ f\, x \\ SUCC &\triangleq \lambda n : NAT.\ \Lambda X.\ \lambda f : X \to X.\ \lambda x : X.\ f\,(n\,[X]\ f\ x) \end{aligned}$$

$SUCC$ must instantiate n at X before applying it to f and x. Note that *twice* is exactly 2.

7.3.3 Products and sums

Products and sums are encoded with no new types. A pair waits for a consumer p and hands it both components, with the consumer choosing the result type R:

$$\begin{aligned}
\tau_1 \times \tau_2 &\triangleq \forall R.\, (\tau_1 \to \tau_2 \to R) \to R \\
\mathit{pair} &\triangleq \Lambda T_1.\, \Lambda T_2.\, \lambda v_1 : T_1.\, \lambda v_2 : T_2.\, \Lambda R.\, \lambda p : T_1 \to T_2 \to R.\, p\ v_1\ v_2 \\
\mathit{fst} &\triangleq \Lambda T_1.\, \Lambda T_2.\, \lambda v : T_1 \times T_2.\, v\,[T_1]\ (\lambda x : T_1.\, \lambda y : T_2.\, x) \\
\mathit{snd} &\triangleq \Lambda T_1.\, \Lambda T_2.\, \lambda v : T_1 \times T_2.\, v\,[T_2]\ (\lambda x : T_1.\, \lambda y : T_2.\, y)
\end{aligned}$$

A sum value waits for one handler per case and calls the appropriate one:

$$\begin{aligned}
\tau_1 + \tau_2 &\triangleq \forall R.\, (\tau_1 \to R) \to (\tau_2 \to R) \to R \\
\mathit{inl} &\triangleq \Lambda T_1.\, \Lambda T_2.\, \lambda v : T_1.\, \Lambda R.\, \lambda b_1 : T_1 \to R.\, \lambda b_2 : T_2 \to R.\, b_1\, v \\
\mathit{inr} &\triangleq \Lambda T_1.\, \Lambda T_2.\, \lambda v : T_2.\, \Lambda R.\, \lambda b_1 : T_1 \to R.\, \lambda b_2 : T_2 \to R.\, b_2\, v \\
\mathit{case} &\triangleq \Lambda T_1.\, \Lambda T_2.\, \Lambda R.\, \lambda v : T_1 + T_2.\, \lambda b_1 : T_1 \to R.\, \lambda b_2 : T_2 \to R.\, v\,[R]\ b_1\ b_2
\end{aligned}$$

Even **unit** and **bool** are unnecessary: $\mathbf{unit} \triangleq \forall R.\, R \to R$ (whose only value is ID) and $\mathbf{bool} \triangleq \mathbf{unit} + \mathbf{unit}$.

7.4 Existential Types

A universal type $\forall X.\, \tau$ means the *user* chooses X. Dually, an existential type $\exists X.\, \tau$ means there is *some* type X with a value of type τ: the *creator* chooses X and the user does not know it. This hides an implementation behind an interface, as modules do in OCaml. For example, an interface for counters (an initial counter, an increment, and a read):

$$\mathit{COUNTER} \triangleq \exists C.\, C \times (C \to C) \times (C \to \mathbf{int}).$$

(We write (e_1, e_2, e_3) for tuples and $\#i\ e$ for projections; these may be added directly or encoded as above.)

7.4.1 Syntax and semantics

$$\begin{aligned}
e &::= \cdots \mid \texttt{pack}\ \{\tau_1, e\}\ \texttt{as}\ \exists X.\, \tau_2 \mid \texttt{unpack}\ \{X, x\} = e_1\ \texttt{in}\ e_2 \\
v &::= \cdots \mid \texttt{pack}\ \{\tau_1, v\}\ \texttt{as}\ \exists X.\, \tau_2 \\
\tau &::= \cdots \mid \exists X.\, \tau
\end{aligned}$$

$\texttt{pack}\ \{\tau_1, e\}\ \texttt{as}\ \exists X.\, \tau_2$ pairs a witness type τ_1 with a value of type $\tau_2\{\tau_1/X\}$. $\texttt{unpack}\ \{X, x\} = e_1\ \texttt{in}\ e_2$ opens the package, binding the hidden type to X and the value to x in e_2 (so $\exists X.\, \tau$ binds X in τ, and **unpack** binds X and x in e_2). Semantics:

$$E ::= \cdots \mid \texttt{pack}\ \{\tau_1, E\}\ \texttt{as}\ \exists X.\, \tau_2 \mid \texttt{unpack}\ \{X, x\} = E\ \texttt{in}\ e$$

$$\texttt{unpack}\ \{X, x\} = (\texttt{pack}\ \{\tau_1, v\}\ \texttt{as}\ \exists Y.\, \tau_2)\ \texttt{in}\ e \ \longrightarrow\ e\{v/x\}\{\tau_1/X\}$$

7.4.2 Typing

$$\frac{\Delta \vdash \tau_1\ \mathit{ok} \qquad \Delta, \Gamma \vdash e : \tau_2\{\tau_1/X\}}{\Delta, \Gamma \vdash \texttt{pack}\ \{\tau_1, e\}\ \texttt{as}\ \exists X.\, \tau_2 : \exists X.\, \tau_2}\ \text{T-Pack}$$

$$\frac{\Delta, \Gamma \vdash e_1 : \exists X.\, \tau_1 \qquad \Delta \cup \{X\}, \Gamma, x : \tau_1 \vdash e_2 : \tau_2 \qquad \Delta \vdash \tau_2\ \mathit{ok}}{\Delta, \Gamma \vdash \texttt{unpack}\ \{X, x\} = e_1\ \texttt{in}\ e_2 : \tau_2}\ \text{T-Unpack}$$

In T-Pack the annotation $\exists X.\, \tau_2$ says which occurrences of the witness are hidden (otherwise a value of type $\mathbf{int} \to \mathbf{int}$ could be typed $C \to C$, $C \to \mathbf{int}$, $\mathbf{int} \to C$, or $\mathbf{int} \to \mathbf{int}$). In T-Unpack, e_2 is checked with X an opaque new type variable ($X \notin \Delta$), and $\Delta \vdash \tau_2\ \mathit{ok}$ ensures the hidden type does not escape through the result type.

7.4.3 Example: counters

$$\mathit{counter} \triangleq \texttt{pack}\ \{\mathbf{int},\ (0,\ \lambda i : \mathbf{int}.\, i+1,\ \lambda i : \mathbf{int}.\, i)\}\ \texttt{as}\ \exists C.\ C \times (C \to C) \times (C \to \mathbf{int})$$

A client:

$$\texttt{unpack}\ \{C, c\} = \mathit{counter}\ \texttt{in}\ (\#3\ c)\ ((\#2\ c)\ ((\#2\ c)\ (\#1\ c))) \to^* 2.$$

Inside the `unpack`, the client only knows $c : C \times (C \to C) \times (C \to \mathbf{int})$; e.g. $(\#1\ c) + 1$ is ill-typed, and $\texttt{unpack}\ \{C, c\} = \mathit{counter}\ \texttt{in}\ \#1\ c$ is rejected by T-Unpack because its type C would escape. Hence implementations are interchangeable, e.g. one that counts down internally:

$$\mathit{counter}' \triangleq \texttt{pack}\ \{\mathbf{int},\ (0,\ \lambda i : \mathbf{int}.\, i-1,\ \lambda i : \mathbf{int}.\, 0-i)\}\ \texttt{as}\ \exists C.\ C \times (C \to C) \times (C \to \mathbf{int}).$$

No client can distinguish *counter* from *counter*′ (*representation independence*, a consequence of parametricity).

7.4.4 Encoding existentials

Existentials are definable from universals: a package takes a result type Y and a client that works for any hidden type, and calls it with its witness and value.

$$\exists X.\, \tau \triangleq \forall Y.\, (\forall X.\, \tau \to Y) \to Y$$
$$\texttt{pack}\ \{\tau_1, e\}\ \texttt{as}\ \exists X.\, \tau_2 \triangleq \Lambda Y.\ \lambda f : \forall X.\, \tau_2 \to Y.\ f\,[\tau_1]\ e$$
$$\texttt{unpack}\ \{X, x\} = e_1\ \texttt{in}\ e_2 \triangleq e_1\,[\tau_2]\ (\Lambda X.\, \lambda x : \tau_1.\, e_2)$$

where $e_1 : \exists X.\, \tau_1$ and $e_2 : \tau_2$. The side condition that X not occur free in τ_2 appears here as well: the client $\Lambda X.\, \lambda x : \tau_1.\, e_2$ must have type $\forall X.\, \tau_1 \to \tau_2$, which only makes sense if τ_2 does not mention X.

7.5 Metatheory

System F is type sound, with the same proof structure as for the simply-typed λ-calculus (progress and preservation).

> **Theorem 7.5.1** Type soundness
>
> If $\emptyset, \cdot \vdash e : \tau$ and $e \to^* e'$ and $e' \not\to$, then e' is a value.

The canonical forms lemma gains a case: if $\emptyset, \cdot \vdash v : \forall X.\, \tau$ then $v = \Lambda X.\, e$ for some e. Preservation for Type-β needs a new lemma:

> **Lenma 7.5.1** Type substitution
>
> If $\Delta \cup \{X\}, \Gamma \vdash e : \tau$ and $\Delta \vdash \tau'$ *ok*, then $\Delta, \Gamma\{\tau'/X\} \vdash e\{\tau'/X\} : \tau\{\tau'/X\}$.

> **Theorem 7.5.2** Strong normalization (Girard, 1972)
>
> If $\emptyset, \cdot \vdash e : \tau$, then every reduction sequence starting from e is finite.

The proof is much harder than for the simply-typed case, since the type of $\Lambda X.\, e$ quantifies over all types, including $\forall X.\, \tau$ itself; Girard's proof uses *reducibility candidates*. Consequently System F is not Turing complete, and Ω, Y, and Z remain untypable. It is nonetheless very expressive: the functions on naturals definable in System F are exactly those provably total in second-order arithmetic, far beyond the primitive recursive functions.

7.6 Type Erasure

Types never affect the result of a program, so they can be erased before running:

$$\begin{aligned} \mathit{erase}(x) &= x \\ \mathit{erase}(\lambda x : \tau.\, e) &= \lambda x.\, \mathit{erase}(e) \\ \mathit{erase}(e_1\, e_2) &= \mathit{erase}(e_1)\ \mathit{erase}(e_2) \\ \mathit{erase}(\Lambda X.\, e) &= \lambda z.\, \mathit{erase}(e) \qquad \text{where } z \text{ is fresh for } e \\ \mathit{erase}(e\,[\tau]) &= \mathit{erase}(e)\ () \end{aligned}$$

The remaining cases erase types in subexpressions. A type abstraction becomes a function of a dummy argument (so it is still a value), and a type application passes (), so each TYPE-β step becomes an ordinary β step:

$$\mathit{erase}((\Lambda X.\, e)\,[\tau]) = (\lambda z.\, \mathit{erase}(e))\ () \longrightarrow \mathit{erase}(e) = \mathit{erase}(e\{\tau/X\}).$$

Theorem 7.6.1 Erasure

For all expressions e and e', $e \longrightarrow e'$ if and only if $\mathit{erase}(e) \longrightarrow \mathit{erase}(e')$.

So we can type check with full types and run without them, as OCaml does.

7.7 Parametricity

A function of type $\forall X.\, X \to X$ knows nothing about X: it cannot inspect, create, or modify a value of type X, so it must be the identity. Likewise a function of type $\forall X.\, X \to X \to X$ can only return one of its arguments, so there are exactly two (up to equivalence): the Church Booleans. And there is no closed term of type $\forall X.\, \forall Y.\, X \to Y$, since no value of an arbitrary type Y can be produced.

This is *parametricity*: Reynolds (1983) showed polymorphic functions treat type arguments uniformly, and Wadler (1989) showed how to derive "theorems for free" from types. For instance, any $f : \forall X.\, X\ \mathit{list} \to X\ \mathit{list}$ can only rearrange, drop, or duplicate elements, hence for every g,

$$\mathit{map}\ g\ (f\ l) = f\ (\mathit{map}\ g\ l).$$

7.8 Polymorphism in OCaml

In System F, programmers write type abstractions, type applications, and type annotations explicitly; e.g. using the polymorphic *twice* at **int** requires $\mathit{twice}\,[\mathbf{int}]\ (\lambda n : \mathbf{int}.\, n + 1)\ 7$. Since reconstructing such annotations is undecidable (Wells 1994), a compiler cannot infer them in general. Yet in OCaml no types need be written and we still get polymorphism:

```
let twice f x = f (f x)
```

is inferred to have type `('a -> 'a) -> 'a -> 'a`, which corresponds to the System F type $\forall X.\,(X \to X) \to X \to X$. Then `twice (fun n -> n + 1) 7` and `twice not true` both work, with the type variable instantiated automatically at **int** or **bool**.
OCaml can do this because its polymorphism is more restricted than System F's. We first see how to infer types for the simply-typed λ-calculus via constraint generation and *unification*, then extend this to *let-polymorphism*, the restricted polymorphism of OCaml and the ML family.

7.9 Type Inference

In the simply-typed λ-calculus each argument has an annotation $\lambda x : \tau.\, e$. Without annotations, can we still type check by *inferring* (or *reconstructing*) the missing types? Yes. Consider

$$\lambda a.\, \lambda b.\, \lambda c.\ \texttt{if}\ a\,(b+1)\ \texttt{then}\ b\ \texttt{else}\ c.$$

b is used in an addition, so $b : \mathbf{int}$. a is applied to $b+1$, so a is a function whose argument type is **int**. The result of $a\,(b+1)$ is a guard, so a returns **bool**. Both branches have the same type, so c has the type of b, namely **int**:

$$\lambda a : \mathbf{int} \to \mathbf{bool}.\ \lambda b : \mathbf{int}.\ \lambda c : \mathbf{int}.\ \texttt{if}\ a\,(b+1)\ \texttt{then}\ b\ \texttt{else}\ c.$$

7.9.1 Constraint-based typing

Given Γ and e, the algorithm produces a type together with a set of *constraints* (equations between types) that must hold for e to be well-typed. We extend types with type variables $X, Y, \ldots$ (placeholders for unknown types, *not* bound by any $\forall$, unlike System F's) and allow abstractions with or without annotations:

$$\begin{aligned} e &::= x \mid \lambda x : \tau.\, e \mid \lambda x.\, e \mid e_1\, e_2 \mid () \mid n \mid e_1 \oplus e_2 \mid \texttt{true} \mid \texttt{false} \mid \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \\ \tau &::= \mathbf{unit} \mid \mathbf{int} \mid \mathbf{bool} \mid \tau_1 \to \tau_2 \mid X \end{aligned}$$

The judgment $\Gamma \vdash e : \tau \mid C$ means e has type τ in Γ provided every constraint in C is satisfied. Read bottom to top, the rules compute τ and C from Γ and e.

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau \mid \emptyset}\ \text{CT-Var} \qquad \frac{}{\Gamma \vdash n : \mathbf{int} \mid \emptyset}\ \text{CT-Int} \qquad \frac{}{\Gamma \vdash \texttt{true} : \mathbf{bool} \mid \emptyset}\ \text{CT-True}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \qquad \Gamma \vdash e_2 : \tau_2 \mid C_2}{\Gamma \vdash e_1 \oplus e_2 : \mathbf{int} \mid C_1 \cup C_2 \cup \{\tau_1 = \mathbf{int},\ \tau_2 = \mathbf{int}\}}\ \text{CT-Arith}$$

Note:

Could we replace $tao1$ with int? It would mean we can't derive a type judgement, whereas with the constraints we always have a derivation. However, there will be places where we don't know the type of e, so we want to be more flexible and add the conditon in the constraints.

$$\frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \qquad \Gamma \vdash e_2 : \tau_2 \mid C_2 \qquad \Gamma \vdash e_3 : \tau_3 \mid C_3}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau_2 \mid C_1 \cup C_2 \cup C_3 \cup \{\tau_1 = \mathbf{bool},\ \tau_2 = \tau_3\}}\ \text{CT-If}$$

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \mid C}{\Gamma \vdash \lambda x : \tau_1.\, e : \tau_1 \to \tau_2 \mid C}\ \text{CT-AbsAnn} \qquad \frac{\Gamma, x : X \vdash e : \tau \mid C \qquad X \text{ fresh}}{\Gamma \vdash \lambda x.\, e : X \to \tau \mid C}\ \text{CT-Abs}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \qquad \Gamma \vdash e_2 : \tau_2 \mid C_2 \qquad X \text{ fresh}}{\Gamma \vdash e_1\, e_2 : X \mid C_1 \cup C_2 \cup \{\tau_1 = \tau_2 \to X\}}\ \text{CT-App}$$

The rules for () and `false` are like CT-True. In CT-AbsAnn the annotation gives the argument type; in CT-Abs we invent a fresh X; in CT-App we invent a fresh X for the result and require e_1 to be a function from the type of e_2 to X. "Fresh" means X appears nowhere else in the derivation.
For our example, CT-Abs gives a, b, c fresh types X_a, X_b, X_c. CT-Arith on $b + 1$ gives **int** with constraints $\{X_b = \textbf{int}, \textbf{int} = \textbf{int}\}$. CT-App on $a\,(b + 1)$ gives a fresh Y and the constraint $X_a = \textbf{int} \to Y$. CT-If gives type X_b and adds $Y = \textbf{bool}$ and $X_b = X_c$. So the whole expression has type

$$X_a \to X_b \to X_c \to X_b$$

subject to

$$C = \{X_b = \textbf{int},\ \textbf{int} = \textbf{int},\ X_a = \textbf{int} \to Y,\ Y = \textbf{bool},\ X_b = X_c\}.$$

7.9.2 Unification

A *type substitution* σ is a finite map from type variables to types, e.g. $[X \mapsto \textbf{int}, Y \mapsto \textbf{int} \to \textbf{int}]$. Applying it replaces type variables simultaneously:

$$\sigma(X) = \begin{cases} \tau & \text{if } X \mapsto \tau \in \sigma \\ X & \text{if } X \text{ is not in the domain of } \sigma \end{cases} \qquad \sigma(\tau_1 \to \tau_2) = \sigma(\tau_1) \to \sigma(\tau_2),$$

and $\sigma(\tau) = \tau$ for **unit**, **int**, **bool**. As nothing binds these type variables, capture is not a concern. Composition is $(\sigma_1 \circ \sigma_2)(\tau) = \sigma_1(\sigma_2(\tau))$.
σ *unifies* a constraint $\tau_1 = \tau_2$ if $\sigma(\tau_1) = \sigma(\tau_2)$, and *satisfies* (unifies) a set C if it unifies every constraint in C. E.g. $[X \mapsto \textbf{int}, Y \mapsto \textbf{int} \to \textbf{int}]$ unifies $X \to (X \to \textbf{int}) = \textbf{int} \to Y$, as both sides become $\textbf{int} \to (\textbf{int} \to \textbf{int})$.
If $\Gamma \vdash e : \tau \mid C$ and σ satisfies C, then e has type $\sigma(\tau)$ in $\sigma(\Gamma)$; if no substitution satisfies C, e is not typable. So we need a substitution satisfying C:

$$\mathit{unify}(\emptyset) = [\,] \quad \text{(the empty substitution)}$$

$$\mathit{unify}(\{\tau_1 = \tau_2\} \cup C) = \begin{cases} \mathit{unify}(C) & \text{if } \tau_1 = \tau_2 \\ \mathit{unify}(C\{\tau_2/X\}) \circ [X \mapsto \tau_2] & \text{if } \tau_1 = X \text{ and } X \notin \mathit{ftv}(\tau_2) \\ \mathit{unify}(C\{\tau_1/X\}) \circ [X \mapsto \tau_1] & \text{if } \tau_2 = X \text{ and } X \notin \mathit{ftv}(\tau_1) \\ \mathit{unify}(C \cup \{\tau_1' = \tau_2',\ \tau_1'' = \tau_2''\}) & \text{if } \tau_1 = \tau_1' \to \tau_1'' \text{ and } \tau_2 = \tau_2' \to \tau_2'' \\ \text{fail} & \text{otherwise} \end{cases}$$

Note:

It's important to compose instead of simply "smashing" the unification and substitution (first we substitute X in our term, then we apply the other constraints with X substituted) (kinda?)

Here $\mathit{ftv}(\tau)$ is the set of type variables occurring in τ. The algorithm fails when asked to unify two different base types, or a base type with a function type.
On our example: $X_b = \textbf{int}$ yields $[X_b \mapsto \textbf{int}]$ and turns $X_b = X_c$ into $\textbf{int} = X_c$; $\textbf{int} = \textbf{int}$ is already satisfied; $X_a = \textbf{int} \to Y$ yields $[X_a \mapsto \textbf{int} \to Y]$; $Y = \textbf{bool}$ yields $[Y \mapsto \textbf{bool}]$; finally $\textbf{int} = X_c$ yields $[X_c \mapsto \textbf{int}]$. Applying the composition to $X_a \to X_b \to X_c \to X_b$ gives

$$(\textbf{int} \to \textbf{bool}) \to \textbf{int} \to \textbf{int} \to \textbf{int},$$

matching the informal result. The algorithm is due to Robinson (1965), applied to type inference by Hindley (1969) and Milner (1978).

Theorem 7.9.1 Unification

The algorithm $\mathit{unify}(C)$ always terminates. It succeeds if and only if C has a solution, and in that case $\mathit{unify}(C)$ is a most general solution: for every solution σ' of C there is a substitution σ'' such that $\sigma' = \sigma'' \circ \mathit{unify}(C)$.

Thus the inferred type is the most general one: every other type of the expression is an instance of it. For $\lambda x.\, x$ we get $X \to X$ with no constraints, so *unify* returns the empty substitution; every other type, e.g. **int** $\to$ **int**, is an instance of $X \to X$.

7.9.3 The occurs check

The check $X \notin \mathit{ftv}(\tau)$ before mapping X to τ is the *occurs check*. It prevents cyclic substitutions such as $[X \mapsto X \to Y]$, which make sense only with infinite types. For $\lambda x.\, x\, x$, CT-Abs gives x the type X and CT-App generates $X = X \to Y$; the occurs check makes *unify* fail, so $\lambda x.\, x\, x$ is not typable (as in the simply-typed λ-calculus). This is the source of OCaml's error "The type variable 'a occurs inside 'a -¿ 'b" for `fun x -> x x`.

7.10 Let-Polymorphism

Unification finds the most general type, which may contain unconstrained type variables; e.g. $\lambda x.\, x : X \to X$. We would like to use such a function at several types:

$$\texttt{let}\ \mathit{id} = \lambda x.\, x\ \texttt{in if}\ \mathit{id}\ \texttt{true then}\ \mathit{id}\ 1\ \texttt{else}\ 0$$

(where $\texttt{let}\ x = e_1\ \texttt{in}\ e_2$ abbreviates $(\lambda x.\, e_2)\, e_1$). With the rules so far, *id* has the single type $X \to X$, and its two uses generate $X =$ **bool** and $X =$ **int**, which are unsatisfiable together.
The idea: at a `let` we know the whole definition, so we can generalize its type to $\forall X.\, X \to X$ and instantiate it with fresh type variables at each use. To keep inference decidable, $\forall$ is only allowed at the outside. A *type scheme* is

$$s ::= \tau \mid \forall X.\, s,$$

and contexts map variables to type schemes. The types τ contain no $\forall$ and are called *monotypes*.

7.10.1 Typing rules

For variables, abstractions, and `let` (other rules as in the simply-typed λ-calculus):

$$\frac{\Gamma(x) = \forall X_1 \ldots X_n.\, \tau}{\Gamma \vdash x : \tau\{\tau_1/X_1, \ldots, \tau_n/X_n/\}}\ \text{T-Var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x.\, e : \tau_1 \to \tau_2}\ \text{T-Abs}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \mathit{gen}(\Gamma, \tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}\ \text{T-Let}$$

where $\mathit{gen}(\Gamma, \tau) = \forall X_1 \ldots X_n.\, \tau$ and $X_1, \ldots, X_n$ are the type variables free in τ but not free in Γ. We cannot generalize over variables free in Γ: they stand for types of variables still in scope, which are not yet known. T-Var instantiates the quantified variables with arbitrary monotypes, and T-Abs gives the argument a monotype. So *id* gets $\forall X.\, X \to X$ and its two uses instantiate X with **bool** and **int**; the whole expression has type **int**.
`let` is no longer just an abbreviation:

$$(\lambda \mathit{id}.\ \texttt{if}\ \mathit{id}\ \texttt{true then}\ \mathit{id}\ 1\ \texttt{else}\ 0)\ (\lambda x.\, x)$$

is not typable, since the λ-bound *id* must have a single monotype. It is well-typed in System F, but only at $(\forall X.\, X \to X) \to$ **int**, with a $\forall$ to the left of an arrow. Restricting $\forall$ to the outside of type schemes is *prenex polymorphism*; it makes inference decidable, and every typable expression has a *principal type scheme* of which all its other types are instances.

7.10.2 Inference

Combine constraint generation with unification. To infer $\texttt{let}\ x = e_1\ \texttt{in}\ e_2$ in Γ:

1. Generate constraints for e_1: $\Gamma \vdash e_1 : \tau_1 \mid C_1$.
2. Solve them immediately: $\sigma = \mathit{unify}(C_1)$.
3. Generalize: give x the type scheme $\mathit{gen}(\sigma(\Gamma), \sigma(\tau_1))$.

4. Infer e_2 in $\sigma(\Gamma)$ extended with this scheme for x.

At each use of a variable with scheme $\forall X_1 \ldots X_n.\, \tau$, replace $X_1, \ldots, X_n$ by fresh type variables. Constraints for e_1 must be solved before generalizing, otherwise we would not know which variables are truly unconstrained. This is essentially Hindley–Milner inference (Algorithm W), the core of type inference in OCaml, Haskell, and others. It is fast in practice, though in the worst case nested `let`s can produce types of size exponential in the program.

7.10.3 The value restriction

With references, generalizing every `let` is unsound:

```
let r = ref (fun x -> x) in
r := (fun n -> n + 1);
(!r) true
```

If r were generalized to $\forall X.\, (X \to X)$ **ref**, the assignment could use it at **int** and the dereference at **bool**, applying an integer function to a Boolean. OCaml's *value restriction* generalizes `let` x `=` e_1 `in` e_2 only when e_1 is a syntactic value (such as a function). Otherwise the type variables stay *weak* and the first use fixes them: above, the assignment fixes r : **(int → int) ref** and the application to `true` is rejected.

7.10.4 Weak type variables

OCaml prints weak variables as `'_weak1`, `'_weak2`, ...: whereas `'a` means "for all types", `'_weak1` means one particular, not yet determined type.

```
# let r = ref (fun x -> x);;
val r : ('_weak1 -> '_weak1) ref
# r := (fun n -> n + 1);;
- : unit = ()
# r;;
- : (int -> int) ref
# (!r) true;;
Error: This expression has type bool but an expression was expected of type int
```

In terms of our algorithm, the value restriction solves the constraints for e_1 as usual but skips *gen*. A weak type variable is an unsolved type variable free in the typing context; later constraints can solve it, but only once, since it stands for a single type. This makes the restriction sound: a reference can only ever hold values of one type. The restriction also applies when nothing is mutable, since an application is not a value:

```
# let ids = List.map (fun x -> x);;
val ids : '_weak1 list -> '_weak1 list
```

The fix is to η-expand, making the definition a function (hence a value):

```
# let ids l = List.map (fun x -> x) l;;
val ids : 'a list -> 'a list
```

A weak variable cannot escape a compilation unit: if nothing determines it, the compiler reports that the type contains type variables that cannot be generalized. OCaml in fact uses a *relaxed value restriction* (Garrigue, 2004), which also generalizes variables appearing only in covariant positions; e.g. `let x = List.rev []` has type `'a list`, since `'a` does not appear under `ref` or to the left of an arrow.

7.11 Summary

System F gives fully general polymorphism but requires type annotations, as inference is undecidable. For the simply-typed λ-calculus we infer types by generating constraints and solving them by unification. Let-polymorphism extends this to a restricted polymorphism with $\forall$ only at the outside of type schemes, generalized at `let` and instantiated at each use. Inference is decidable and finds principal types, which is why OCaml can infer polymorphic types for us.

For other types of inference, checkout local or bidirectional methods.